目 录

1 逻辑控制指令 1

1.1 比较指令 1
1.1.1 UEQB：8 位无符号类型等于比较指令 1
1.1.2 UEQW：16 位无符号类型等于比较指令 1
1.1.3 UEQD：32 位无符号类型等于比较指令 1
1.1.4 ULEB：8 位无符号类型小于等于比较指令 2
1.1.5 ULEW：16 位无符号类型小于等于比较指令 2
1.1.6 ULED：32 位无符号类型小于等于比较指令 2
1.1.7 ULTB：8 位无符号类型小于比较指令 2
1.1.8 ULTW：16 位无符号类型小于比较指令 3
1.1.9 ULTD：32 位无符号类型小于比较指令 3
1.1.10 UGEB：8 位无符号类型大于等于比较指令 3
1.1.11 UGEW：16 位无符号类型大于等于比较指令 4
1.1.12 UGED：32 位无符号类型大于等于比较指令 4
1.1.13 UGTB：8 位无符号类型大于比较指令 4
1.1.14 UGTW：16 位无符号类型大于比较指令 4
1.1.15 UGTD：32 位无符号类型大于比较指令 5
1.1.16 UNEB：8 位无符号类型不等于比较指令 5
1.1.17 UNEW：16 位无符号类型不等于比较指令 5
1.1.18 UNED：32 位无符号类型不等于比较指令 6
1.1.19 UZCPB：8 位无符号类型区间比较指令 6
1.1.20 UZCPW：16 位无符号类型区间比较指令 6
1.1.21 UZCPD：32 位无符号类型区间比较指令 7
1.1.22 EQB：8 位有符号类型等于比较指令 7
1.1.23 EQW：16 位有符号类型等于比较指令 7
1.1.24 EQD：32 位有符号类型等于比较指令 7
1.1.25 LEB：8 位有符号类型小于等于比较指令 8
1.1.26 LEW：16 位有符号类型小于等于比较指令 8
1.1.27 LED：32 位有符号类型小于等于比较指令 8
1.1.28 LTB：8 位有符号类型小于比较指令 9
1.1.29 LTW：16 位有符号类型小于比较指令 9
1.1.30 LTD：32 位有符号类型小于比较指令 9
1.1.31 GEB：8 位有符号类型大于等于比较指令 9
1.1.32 GEW：16 位有符号类型大于等于比较指令 10
1.1.33 GED：32 位有符号类型大于等于比较指令 10
1.1.34 GTB：8 位有符号类型大于比较指令 10
1.1.35 GTW：16 位有符号类型大于比较指令 10
1.1.36 GTD：32 位有符号类型大于比较指令 11
1.1.37 NEB：8 位有符号类型不等于比较指令 11
1.1.38 NEW：16 位有符号类型不等于比较指令 11
1.1.39 NED：32 位有符号类型不等于比较指令 12
1.1.40 ZCPB：8 位有符号类型区间比较指令 12
1.1.41 ZCPW：16 位有符号类型区间比较指令 12
1.1.42 ZCPD：32 位有符号类型区间比较指令 12
1.1.43 EEQ：浮点数类型等于比较指令 13

1.1.44 ELE：浮点数类型小于等于比较指令 13
1.1.45 ELT：浮点数类型小于比较指令 13
1.1.46 EGE：浮点数类型大于等于比较指令 14
1.1.47 EGT：浮点数类型大于比较指令 14
1.1.48 ENE：浮点数类型不等于比较指令 14
1.1.49 EZCP：浮点数类型区间比较指令 15
1.2 时钟指令 15
1.2.1 TON：通电延时定时器指令 15
1.2.2 TP：脉冲发生器指令 15
1.2.3 TOF：延时断开计时器指令 16
1.2.4 CTD：减计数指令 16
1.2.5 CTU：加计数指令 16
1.2.6 CTUD：可增可减计数指令 17
1.2.7 F_TRIG：下降沿功能块指令 17
1.2.8 R_TRIG：上升沿功能块指令 17
1.2.9 SR：置位优先触发器指令 18
1.2.10 RS：复位优先触发器指令 18
1.3 传送指令 18
1.3.1 MOVB：8 位传送指令 18
1.3.2 MOVW：16 位传送指令 18
1.3.3 MOVD：32 位传送指令 19
1.3.4 MOVR：浮点数传送指令 19
1.3.5 MOVB_3：字节传送指令 19
1.3.6 MOVB_4：传送指令 19
1.3.7 MOVB_5：传送指令 20
1.3.8 FMOVB：传送指令 20
1.3.9 FMOVW：传送指令 20
1.3.10 FMOVD：传送指令 21
1.3.11 XCHB：8 位数据交换指令 21
1.3.12 XCHW：16 位数据交换指令 21
1.3.13 XCHD：32 位数据交换指令 22
1.4 移位指令 22
1.4.1 USHLB：8 位无符号逻辑左移指令 22
1.4.2 USHLW：16 位无符号逻辑左移指令 22
1.4.3 USHLD：32 位无符号逻辑左移指令 22
1.4.4 USHRB：8 位无符号逻辑右移指令 23
1.4.5 USHRW：16 位无符号逻辑右移指令 23
1.4.6 USHRD：32 位无符号逻辑右移指令 23
1.4.7 UROLB：8 位无符号循环左移指令 24
1.4.8 UROLW：16 位无符号循环左移指令 24
1.4.9 UROLD：32 位无符号循环左移指令 24
1.4.10 URORB：8 位无符号循环右移指令 25
1.4.11 URORW：16 位无符号循环右移指令 25
1.4.12 URORD：32 位无符号循环右移指令 25
1.4.13 URCLB：8 位无符号带进位循环左移指令 25
1.4.14 URCLW：16 位无符号带进位循环左移指令 26
1.4.15 URCLD：32 位无符号带进位循环左移指令 26
1.4.16 URCRB：8 位无符号带进位循环右移指令 26

1.4.17 URCRW：16 位无符号带进位循环右移指令 26
1.4.18 URCRD：32 位无符号带进位循环右移指令 27
1.4.19 SHLB：8 位有符号逻辑左移指令 27
1.4.20 SHLW：16 位有符号逻辑左移指令 27
1.4.21 SHLD：32 位有符号逻辑左移指令 28
1.4.22 SHRB：8 位有符号逻辑右移指令 28
1.4.23 SHRW：16 位有符号逻辑右移指令 28
1.4.24 SHRD：32 位有符号逻辑右移指令 29
1.5 逻辑运算 29
1.5.1 NEGB：8 位补码操作指令 29
1.5.2 NEGW：16 位补码操作指令 29
1.5.3 NEGD：32 位补码操作指令 29
1.5.4 ANDB：8 位按位与操作指令 30
1.5.5 ANDW：16 位按位与操作指令 30
1.5.6 ANDD：32 位按位与操作指令 30
1.5.7 ORBB：8 位按位或操作指令 30
1.5.8 ORW：16 位按位或操作指令 31
1.5.9 ORD：32 位按位或操作指令 31
1.5.10 EORB：8 位按位异或操作指令 31
1.5.11 EORW：16 位按位异或操作指令 31
1.5.12 EORD：32 位按位异或操作指令 32
1.5.13 NOTB：8 位按位非操作指令 32
1.5.14 NOTW：16 位按位非操作指令 32
1.5.15 NOTD：32 位按位非操作指令 32
1.6 数据处理 33
1.6.1 SELB：8 位选择指令 33
1.6.2 SELW：16 位选择指令 33
1.6.3 SELD：32 位选择指令 33
1.6.4 MUXB：8 位数据选择指令 34
1.6.5 MUXW：16 位数据选择指令 34
1.6.6 MUXD：32 位数据选择指令 34
1.6.7 LIMITB：8 位上限/下限控制指令 34
1.6.8 LIMITW：16 位上限/下限控制指令 35
1.6.9 LIMITD：32 位上限/下限控制指令 35
1.6.10 MAXB：8 位取大值指令 35
1.6.11 MAXW：16 位取大值指令 35
1.6.12 MAXD：32 位取大值指令 36
1.6.13 MINB：8 位取小值指令 36
1.6.14 MINW：16 位取小值指令 36
1.6.15 MIND：32 位取小值指令 36
1.6.16 MEANB：字节型平均数指令 37
1.6.17 MEANW：字型平均数指令 37
1.6.18 MEAND：双字型平均数指令 37
1.6.19 BON：ON 位判断指令 38
1.6.20 SQRB：字节型开方指令 38
1.6.21 SQRW：字型开方指令 38
1.6.22 SQRD：双字型开方指令 38
1.6.23 SUMB：字节型置 ON 位总和指令 39

1.6.24 SUMW：字型置 ON 位总和指令 39
1.6.25 SUMD：双字型置 ON 位总和指令 39
1.6.26 DECO：译码指令 39
1.6.27 ENCO：编码指令 40
1.6.28 ASCI：ASCII 转换指令 40
1.6.29 HEX：十六进制转换指令 40
1.7 算术运算 40
1.7.1 UADDB：无符号字节型加法指令 40
1.7.2 UADDW：无符号字型加法指令 41
1.7.3 UADDD：无符号双字型加法指令 41
1.7.4 UDECB：无符号字节型自减指令 41
1.7.5 UDECW：无符号字型自减指令 41
1.7.6 UDECD：无符号双字型自减指令 42
1.7.7 UDIVB：无符号字节型除法指令 42
1.7.8 UDIVW：无符号字型除法指令 42
1.7.9 UDIVD：无符号双字型除法指令 42
1.7.10 UINCB：无符号字节型自加指令 43
1.7.11 UINCW：无符号字型自加指令 43
1.7.12 UINCD：无符号双字型自加指令 43
1.7.13 UMODB：无符号字节型取模指令 43
1.7.14 UMODW：无符号字型取模指令 44
1.7.15 UMODD：无符号双字型取模指令 44
1.7.16 UMULB：无符号字节型乘法指令 44
1.7.17 UMULW：无符号字型乘法指令 44
1.7.18 UMULD：无符号双字节型乘法指令 45
1.7.19 USUBB：无符号字节型减法指令 45
1.7.20 USUBW：无符号字型减法指令 45
1.7.21 USUBD：无符号双字型减法指令 45
1.7.22 ADDB：有符号字节型加法指令 46
1.7.23 ADDW：有符号字型加法指令 46
1.7.24 ADDD：有符号双字型加法指令 46
1.7.25 DECB：有符号字节型自减指令 46
1.7.26 DECW：有符号字型自减指令 47
1.7.27 DECD：有符号双字型自减指令 47
1.7.28 DIVB：有符号字节型除法指令 47
1.7.29 DIVW：有符号字型除法指令 47
1.7.30 DIVD：有符号双字型除法指令 47
1.7.31 INCB：有符号字节型自加指令 48
1.7.32 INCW：有符号字型自加指令 48
1.7.33 INCD：有符号双字型自加指令 48
1.7.34 MODB：有符号字节型取模指令 48
1.7.35 MODW：有符号字型取模指令 49
1.7.36 MODD：有符号双字型取模指令 49
1.7.37 MULB：有符号字节型乘法指令 49
1.7.38 MULW：有符号字型乘法指令 49
1.7.39 MULD：有符号双字型乘法指令 50
1.7.40 SUBB：有符号字节型减法指令 50
1.7.41 SUBW：有符号字型减法指令 50

1.7.42 SUBD：有符号双字型减法指令 50
1.7.43 EADD：浮点数加法指令 51
1.7.44 ESUB：浮点数减法指令 51
1.7.45 EMUL：浮点数乘法指令 51
1.7.46 EDIV：浮点数除法指令 51
1.7.47 ESQR：浮点数开方指令 52
1.7.48 ELN：浮点数 e 为底对数指令 52
1.7.49 ELOG：浮点数 10 为底对数指令 52
1.7.50 EEXP：浮点数 e 为底幂指令 52
1.7.51 ESIN：浮点数正弦指令 52
1.7.52 ECOS：浮点数余弦指令 53
1.7.53 ETAN：浮点数正切指令 53
1.7.54 EASIN：浮点数反正弦指令 53
1.7.55 EACOS：浮点数反余弦指令 53
1.7.56 EATAN：浮点数反正切指令 54
1.7.57 EEXPT：浮点数幂指令 54
1.8 无类型运算 54
1.8.1 ADD：无类型加法指令 54
1.8.2 SUB：无类型减法指令 54
1.8.3 MUL：无类型乘法指令 55
1.8.4 DIV：无类型除法指令 55
1.8.5 EQ：无类型比较等于指令 55
1.8.6 LE：无类型比较小于等于指令 55
1.8.7 LT：无类型比较小于指令 56
1.8.8 GE：无类型比较大于等于指令 56
1.8.9 GT：无类型比较大于指令 56
1.8.10 NE：无类型比较不等于指令 56
1.8.11 MOD：无类型取模指令 57
1.8.12 SHL：无类型左移指令 57
1.8.13 SHR：无类型右移指令 57
1.8.14 MOVE：无类型传送指令 58
1.9 外部设备 I/O 58
1.9.1 SEGD：7 段译码器指令 58
1.10 数据类型转换 58
1.10.1 SINT_TO_USINT：有符号字节型转无符号字节型指令 58
1.10.2 SINT_TO_UINT：有符号字节型转无符号字型指令 58
1.10.3 SINT_TO_UDINT：有符号字节型转无符号双字型指令 59
1.10.4 SINT_TO_INT：有符号字节型转有符号字型指令 59
1.10.5 SINT_TO_DINT：有符号字节型转无符号双字型指令 59
1.10.6 SINT_TO_REAL：有符号字节型转浮点型指令 59
1.10.7 INT_TO_USINT：有符号字型转无符号字节型指令 59
1.10.8 INT_TO_UINT：有符号字型转无符号字型指令 60
1.10.9 INT_TO_UDINT：有符号字型转无符号双字型指令 60
1.10.10 INT_TO_SINT：有符号字型转有符号字节型指令 60
1.10.11 INT_TO_DINT：有符号字型转有符号双字型指令 60
1.10.12 INT_TO_REAL：有符号字型转浮点型指令 61
1.10.13 DINT_TO_USINT：有符号双字型转无符号字节型指令 61
1.10.14 DINT _TO_UINT：有符号双字型转无符号字型指令 61

1.10.15 DINT _TO_UDINT：有符号双字型转无符号双字型指令 61
1.10.16 DINT _TO_SINT：有符号双字型转有符号字节型指令 61
1.10.17 DINT _TO_INT：有符号双字型转有符号字型指令 62
1.10.18 DINT _TO_REAL：有符号双字型转浮点型指令 62
1.10.19 USINT _TO_UINT：无符号字节型转无符号字型指令 62
1.10.20 USINT _TO_UDINT：无符号字节型转无符号双字型指令 62
1.10.21 USINT _TO_SINT：无符号字节型转有符号字节型指令 62
1.10.22 USINT _TO_INT：无符号字节型转有符号字型指令 63
1.10.23 USINT _TO_DINT：无符号字节型转有符号双字型指令 63
1.10.24 USINT _TO_REAL：无符号字节型转浮点型指令 63
1.10.25 UINT_TO_USINT：无符号字型转无符号字节型指令 63
1.10.26 UINT_TO_UDINT：无符号字型转无符号双字型指令 64
1.10.27 UINT_TO_SINT：无符号字型转有符号字节型指令 64
1.10.28 UINT_TO_INT：无符号字型转有符号字型指令 64
1.10.29 UINT_TO_DINT：无符号字型转有符号双字型指令 64
1.10.30 UINT_TO_REAL：无符号字型转浮点型指令 64
1.10.31 UDINT_TO_USINT：无符号双字型转无符号字节型指令 65
1.10.32 UDINT _TO_UINT：无符号双字型转无符号字型指令 65
1.10.33 UDINT _TO_SINT：无符号双字型转有符号字节型指令 65
1.10.34 UDINT _TO_INT：无符号双字型转有符号字型指令 65
1.10.35 UDINT _TO_DINT：无符号双字型转有符号双字型指令 65
1.10.36 UDINT _TO_REAL：无符号双字型转浮点型指令 66
1.10.37 REAL_TO_USINT：浮点型转无符号字节型指令 66
1.10.38 REAL _TO_UINT：浮点型转无符号字型指令 66
1.10.39 REAL _TO_UDINT：浮点型转无符号双字型指令 66
1.10.40 REAL _TO_SINT：浮点型转有符号字节型指令 67
1.10.41 REAL _TO_INT：浮点型转有符号字型指令 67
1.10.42 REAL _TO_DINT：浮点型转有符号双字型指令 67
1.11 模拟量控制 67
1.11.1 PIDAT：无符号字节转 BCD 指令 67
1.11.2 TimeproportionOut：时间比例输出指令 68
1.11.3 LimitAlarm：超限报警指令 68
1.11.4 LimitAlarmDv：超偏差报警指令 69
1.11.5 LimitAlarmDvStbSeq：稳态超偏差报警指令 69
1.11.6 ScaleTrans：范围转换指令 70
1.12 BCD 码转换 70
1.12.1 USINT_TO_BCD：无符号字节转 BCD 指令 70
1.12.2 UINT_TO_BCD：无符号字转 BCD 指令 70
1.12.3 UDINT_TO_BCD：无符号双字转 BCD 指令 70
1.12.4 SINT_TO_BCD：有符号字节转 BCD 指令 71
1.12.5 INT_TO_BCD：有符号字转 BCD 指令 71
1.12.6 DINT_TO_BCD：有符号双字转 BCD 指令 71
1.12.7 BCD_TO_USINT：BCD 转无符号字节指令 71
1.12.8 BCD_TO_UINT：BCD 转无符号字指令 72
1.12.9 BCD_TO_UDINT：BCD 转无符号双字指令 72
1.12.10 BCD_TO_SINT：BCD 转有符号字节指令 72
1.12.11 BCD_TO_INT：BCD 转有符号字指令 72
1.12.12 BCD_TO_DINT：BCD 转有符号双字指令 73

1.13 时间和日期 74
1.13.1 ADD_TIME：TIME 变量的加法指令 74
1.13.2 ADD_TOD_TIME：TOD 变量和 TIME 变量的加法指令 74
1.13.3 ADD_DT_TIME：DT 变量和 TIME 变量的加法指令 74
1.13.4 SUB_TIME：TIME 减法指令 75
1.13.5 SUB_TOD_TIME：TOD 变量和 TIME 变量的减法指令 75
1.13.6 SUB_TOD_ TOD：TOD 变量的减法指令 75
1.13.7 SUB_DATE_DATE：DATE 变量的减法指令 76
1.13.8 SUB_DT_DT：DT 变量的减法指令 76
1.13.9 SUB _DT_TIME：DT 变量和 TIME 变量的减法指令 76
1.13.10 MULTIME：TIME 变量和数字的乘法指令 76
1.13.11 DIVTIME：TIME 变量和数字的除法指令 77
1.13.12 CONCAT_DATE_TOD：DATE 变量和 TOD 变量的连接指令 77
1.13.13 DT_TO_TOD：DT 变量转 TOD 变量指令 77
1.13.14 DT_TO_DATE：DT 变量转 DATE 变量指令 77
1.13.15 DtToSec：DT 变量转秒数指令 78
1.13.16 DateToSec：Date 变量转秒数指令 78
1.13.17 TodToSec：Tod 变量转秒数指令 78
1.13.18 SecToDt：秒数转 DT 变量指令 78
1.13.19 SecToDate：秒数转 DATE 变量指令 78
1.13.20 SecToTod：秒数转 TOD 变量指令 79
1.13.21 TimeToMilliSec：TIME 变量转毫秒数指令 79
1.13.22 TimeToSec：TIME 变量转秒数指令 79
1.13.23 MilliSecToTime：毫秒数转 TIME 变量指令 79
1.13.24 SecToTime：秒数转 TIME 变量指令 80
1.13.25 ChkLeapYear：判断闰年指令 80
1.13.26 GetDaysOfMonth：获取天数指令 80
1.13.27 DaysToMonth：计算月份指令 80
1.13.28 GetDayOfWeek：获取星期几指令 81
1.13.29 GetWeekOfYear：获取第几周指令 81
1.13.30 GetTime：获取系统时间指令 81
1.13.31 SetTime：设置系统时间指令 81

2 运动控制指令 83

2.1 单轴模块 84
2.1.1 MC_Power 84
2.1.2 MC_Stop 85
2.1.3 MC_Halt 85
2.1.4 MC_MoveAbsolute 86
2.1.5 MC_MoveRelative 87
2.1.6 MC_MoveAdditive 88
2.1.7 MC_MoveSuperimposed 90
2.1.8 MC_HaltSuperimposed 91
2.1.9 MC_MoveVelocity 92
2.1.10 MC_MoveContinuousAbsolute 93
2.1.11 MC_MoveContinuousRelative 94
2.1.12 MC_TorqueControl 95
2.1.13 MC_PositionProfile 96

2.1.14 MC_VelocityProfile 97
2.1.15 MC_AccelerationProfile 98
2.1.16 MC_SetPosition 99
2.1.17 MC_ReadParameter 100
2.1.18 MC_ReadBoolParameter 101
2.1.19 MC_WriteParameter 101
2.1.20 MC_WriteBoolParameter 102
2.1.21 MC_ReadDigitalInput 103
2.1.22 MC_ReadDigitalOutput 103
2.1.23 MC_WriteDigitalOutput 104
2.1.24 MC_ReadActualPosition 105
2.1.25 MC_ReadActualVelocity 105
2.1.26 MC_ReadActualTorque 106
2.1.27 MC_ReadStatus 106
2.1.28 MC_ReadAxisInfo 107
2.1.29 MC_ReadAxisError 108
2.1.30 MC_Reset 109
2.1.31 MC_TouchProbe 109
2.1.32 MC_AbortTrigger 110
2.1.33 MC_CamTableSelect 111
2.1.34 MC_CamIn 112
2.1.35 MC_CamOut 114
2.1.36 MC_GearIn 114
2.1.37 MC_GearOut 115
2.1.38 MC_PhasingAbsolute 116
2.1.39 MC_PhasingRelative 117
2.2 轴组模块 118
2.2.1 MC_AddAxisToGrououp 118
2.2.2 MC_RemoveAxisFromGroup 119
2.2.3 MC_UngroupAllAxes 120
2.2.4 MC_GroupReadConfiguration 120
2.2.5 MC_GroupEnable 121
2.2.6 MC_GroupDisable 122
2.2.7 MC_GroupHome 122
2.2.8 MC_SetKinTransform 123
2.2.9 MC_SetCartesianTransform 124
2.2.10 MC_SetCoordinateTransform 125
2.2.11 MC_ReadKinTransform 127
2.2.12 MC_ReadCartesianTransform 127
2.2.13 MC_ReadCoordinateTransform 128
2.2.14 MC_GroupSetPosition 129
2.2.15 MC_GroupReadActualPosition 130
2.2.16 MC_GroupReadActualVelocity 131
2.2.17 MC_GroupReadActualAcceleration 132
2.2.18 MC_GroupStop 133
2.2.19 MC_GroupHalt 134
2.2.20 MC_GroupInterrupt 135
2.2.21 MC_GroupContinue 136

2.2.22 MC_GroupReadStatus 136
2.2.23 MC_GroupReadError 137
2.2.24 MC_GroupReset 138
2.2.25 MC_MoveLinearAbsolute 139
2.2.26 MC_MoveLinearRelative 140
2.2.27 MC_MoveCircularAbsolute 142
2.2.28 MC_MoveCircularRelative 143
2.2.29 MC_MoveDirectAbsolute 145
2.2.30 MC_MoveDirectRelative 146
2.2.31 MC_PathSelect 148
2.2.32 MC_MovePath 149
2.2.33 MC_GroupSetOverrid 150
2.2.34 MC_SyncAxisToGroup 151
2.2.35 MC_SyncGroupToAxis 152

3 通信协议指令 154

3.1 通讯基础指令 154
3.1.1 StkUDPCreate 154
3.1.2 StkUDPRcv 154
3.1.3 StkUDPSend 155
3.1.4 SerialSend 156
3.1.5 SerialRcv 156
3.1.6 SerialInit 157
3.1.7 StkTCPConnect 158
3.1.8 StkTCPSend 158
3.1.9 StkTCPRcv 159
3.1.10 StkTCPClose 160
3.1.11 StkTCPAccept 160
3.1.12 StkGetTCPStatus 161
3.1.13 AryLRC_BYTE 162
3.1.14 AryLRC_WORD 162
3.1.15 AryLRC_DWORD 163
3.1.16 SUM_BYTE 163
3.1.17 SUM_WORD 163
3.1.18 SUM_DWORD 164
3.1.19 CRC16_MODBUS 164
3.1.20 CRC16_CCITT 165
3.1.21 CANSend 165
3.1.22 CANRecv 166
3.1.23 CANInit 166
3.1.24 MQTTRecv 167
3.1.25 MQTTSend 168
3.1.26 MQTTSubscribe 169
3.1.27 MQTTCreate 169

4 安全算法指令 ..... 错误！未定义书签。

4.1 RC4 对称加密算法指令 ..... **错误！未定义书签。**
4.1.1 RC4En ..... **错误！未定义书签。**

4.1.2 RC4De ..... **错误！未定义书签。**
4.2 RSA 非对称加密算法指令 ..... **错误！未定义书签。**
4.2.1 RSAEn ..... **错误！未定义书签。**
4.2.2 RSADe ..... **错误！未定义书签。**
4.2.3 getPrime ..... **错误！未定义书签。**
4.2.4 getRSAKey ..... **错误！未定义书签。**
4.2.5 Verify ..... **错误！未定义书签。**

1 逻辑控制指令

1.1 比较指令

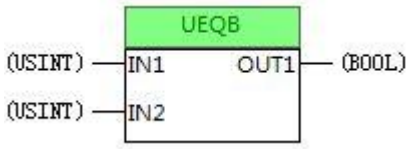
1.1.1 UEQB：8 位无符号类型等于比较指令

接口设计：输入变量 IN1，IN2，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：UEQB (IN1 := , IN2 := , OUT1 =>);

控件图示：



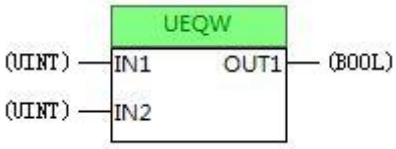
1.1.2 UEQW：16 位无符号类型等于比较指令

接口设计：输入变量 IN1，IN2，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：UEQW (IN1 := , IN2 := , OUT1 =>);

控件图示：



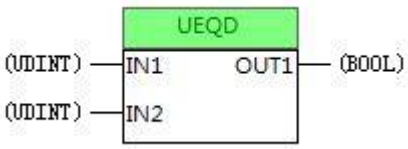
1.1.3 UEQD：32 位无符号类型等于比较指令

接口设计：输入变量 IN1，IN2，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：UEQD (IN1 := , IN2 := , OUT1 =>);

控件图示：



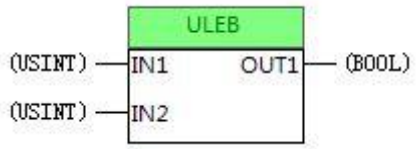
1.1.4 ULEB：8 位无符号类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：ULEB (IN1 := , IN2 := , OUT1 =>);

控件图示：



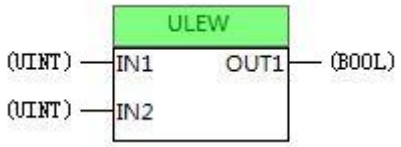
1.1.5 ULEW：16 位无符号类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：ULEW (IN1 := , IN2 := , OUT1 =>);

控件图示：



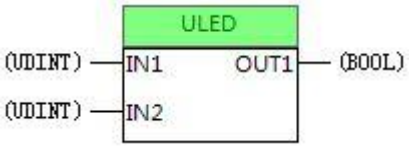
1.1.6 ULED：32 位无符号类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := ULED (IN1 := , IN2 := , OUT1 =>);

控件图示：



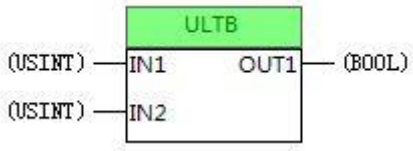
1.1.7 ULTB：8 位无符号类型小于比较指令

接口设计：输入变量 IN1，IN2，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：ULTB (IN1 := , IN2 := , OUT1 =>);

控件图示：



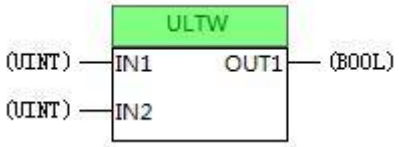
1.1.8 ULTW：16 位无符号类型小于比较指令

接口设计：输入变量 IN1，IN2，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：ULTW (IN1 := , IN2 := , OUT1 =>);

控件图示：



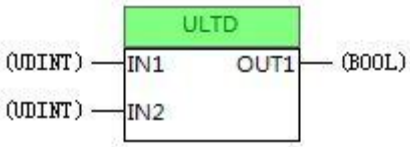
1.1.9 ULTD：32 位无符号类型小于比较指令

接口设计：输入变量 IN1，IN2，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：ULTD (IN1, IN2);

控件图示：



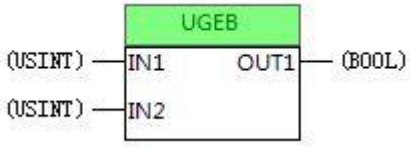
1.1.10 UGEB：8 位无符号类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UGEB (IN1, IN2);

控件图示：



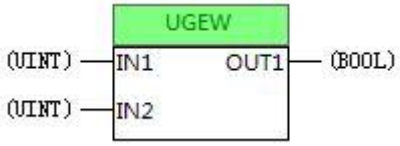
1.1.11 UGEW：16 位无符号类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UGEW (IN1, IN2);

控件图示：



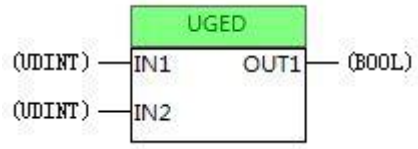
1.1.12 UGED：32 位无符号类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UGED (IN1, IN2);

控件图示：



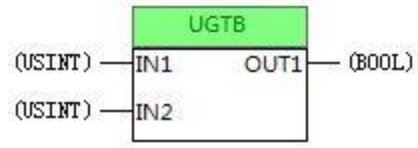
1.1.13 UGTB：8 位无符号类型大于比较指令

接口设计：输入变量 IN1，IN2，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UGTB (IN1, IN2);

控件图示：



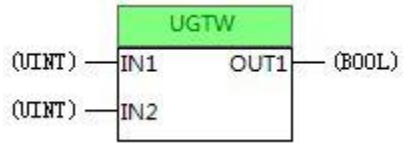
1.1.14 UGTW：16 位无符号类型大于比较指令

接口设计：输入变量 IN1，IN2，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。.

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UGTW (IN1, IN2);

控件图示：



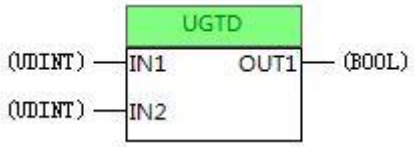
1.1.15 UGTD：32 位无符号类型大于比较指令

接口设计：输入变量 IN1，IN2，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UGTD (IN1, IN2);

控件图示：



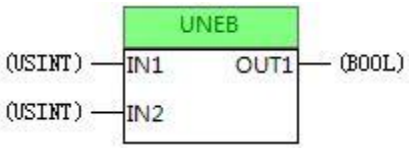
1.1.16 UNEB：8 位无符号类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UNEB (IN1, IN2);

控件图示：



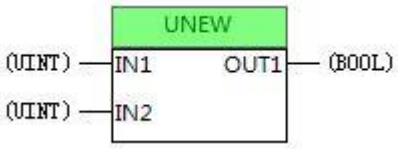
1.1.17 UNEW：16 位无符号类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UNEW (IN1, IN2);

控件图示：



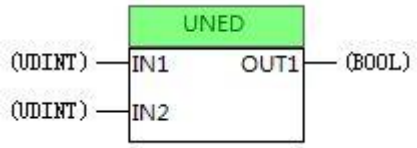
1.1.18 UNED：32 位无符号类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UNED (IN1, IN2);

控件图示：



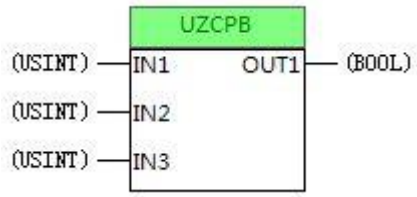
1.1.19 UZCPB：8 位无符号类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 USINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UZCPB (IN1, IN2, IN3);

控件图示：



1.1.20 UZCPW：16 位无符号类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := UZCPW (IN1, IN2, IN3);

控件图示：



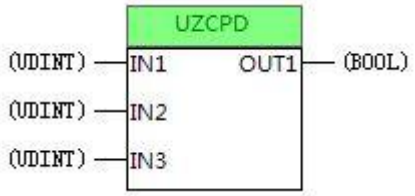
1.1.21 UZCPD：32 位无符号类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 UDINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE; 否则 OUT1 = FALSE。

控件文本表示：OUT1 := UZCPD (IN1, IN2, IN3);

控件图示：



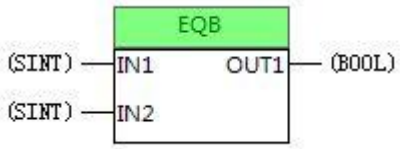
1.1.22 EQB：8 位有符号类型等于比较指令

接口设计：输入变量 IN1，IN2，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := EQB (IN1, IN2);

控件图示：



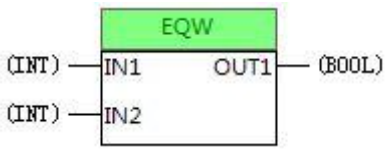
1.1.23 EQW：16 位有符号类型等于比较指令

接口设计：输入变量 IN1，IN2，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := EQW (IN1, IN2);

控件图示：



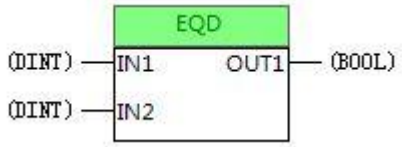
1.1.24 EQD：32 位有符号类型等于比较指令

接口设计：输入变量 IN1，IN2，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := EQD (IN1, IN2);

控件图示：



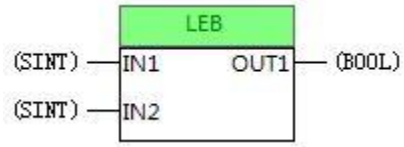
1.1.25 LEB：8 位有符号类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := LEB (IN1, IN2);

控件图示：



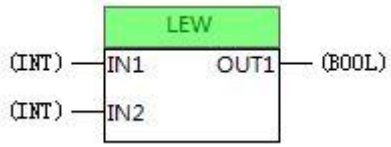
1.1.26 LEW：16 位有符号类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := LEW (IN1, IN2);

控件图示：



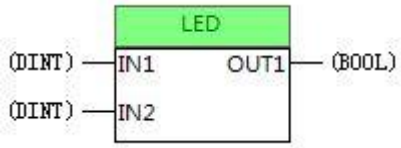
1.1.27 LED：32 位有符号类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := LED (IN1, IN2);

控件图示：



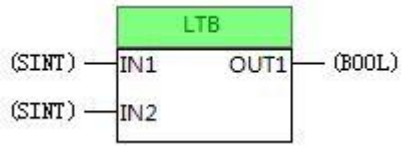
1.1.28 LTB：8 位有符号类型小于比较指令

接口设计：输入变量 IN1，IN2，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := LTB (IN1, IN2);

控件图示：



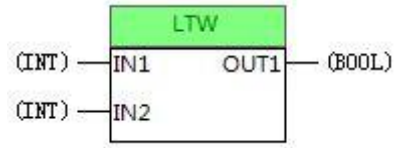
1.1.29 LTW：16 位有符号类型小于比较指令

接口设计：输入变量 IN1，IN2，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := LTW (IN1, IN2);

控件图示：



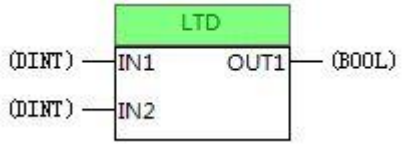
1.1.30 LTD：32 位有符号类型小于比较指令

接口设计：输入变量 IN1，IN2，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := LTD (IN1, IN2);

控件图示：



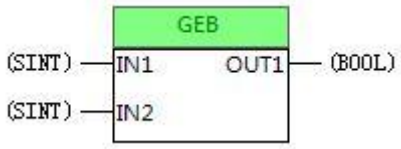
1.1.31 GEB：8 位有符号类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := GEB (IN1, IN2);

控件图示：



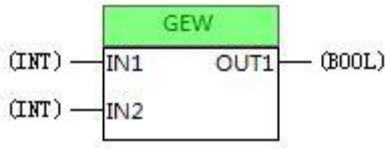
1.1.32 GEW：16 位有符号类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := GEW (IN1, IN2);

控件图示：



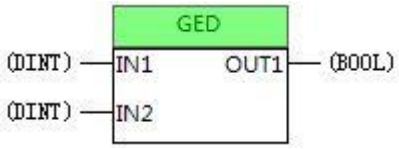
1.1.33 GED：32 位有符号类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := GED (IN1, IN2);

控件图示：



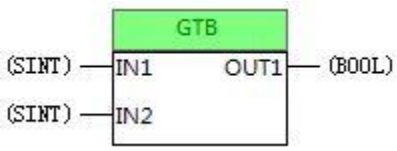
1.1.34 GTB：8 位有符号类型大于比较指令

接口设计：输入变量 IN1，IN2，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := GTB (IN1, IN2);

控件图示：



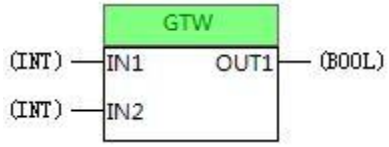
1.1.35 GTW：16 位有符号

接口设计：输入变量 IN1，IN2，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := GTW (IN1, IN2);

控件图示：



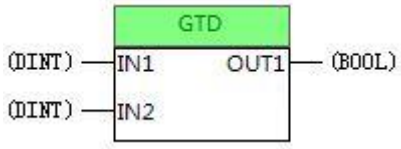
1.1.36 GTD：32 位有符号类型大于比较指令

接口设计：输入变量 IN1，IN2，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := GTD (IN1, IN2);

控件图示：



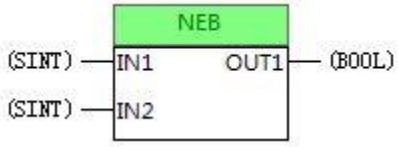
1.1.37 NEB：8 位有符号类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := NEB (IN1, IN2);

控件图示：



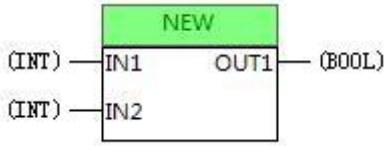
1.1.38 NEW：16 位有符号类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := NEW (IN1, IN2);

控件图示：



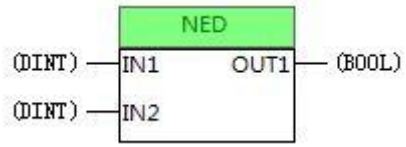
1.1.39 NED：32 位有符号类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := NED (IN1, IN2);

控件图示：



1.1.40 ZCPB：8 位有符号类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 SINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE; 否则 OUT1 = FALSE。

控件文本表示：OUT1 := ZCPB (IN1, IN2);

控件图示：



1.1.41 ZCPW：16 位有符号类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 INT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE; 否则 OUT1 = FALSE。

控件文本表示：OUT1 := ZCPW (IN1, IN2);

控件图示：



1.1.42 ZCPD：32 位有符号类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 DINT 类型的变量；输出变量 OUT1，为 BOOL 类

型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE; 否则 OUT1 = FALSE。

控件文本表示：OUT1 := ZCPD (IN1, IN2);

控件图示：



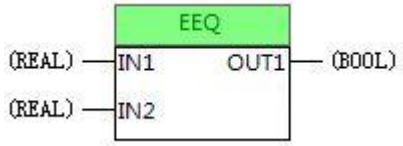
1.1.43 EEQ：浮点数类型等于比较指令

接口设计：输入变量 IN1，IN2，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := EEQ (IN1, IN2);

控件图示：



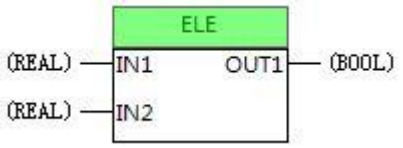
1.1.44 ELE：浮点数类型小于等于比较指令

接口设计：输入变量 IN1，IN2，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := ELE (IN1, IN2);

控件图示：



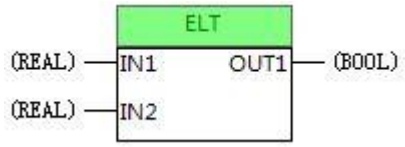
1.1.45 ELT：浮点数类型小于比较指令

接口设计：输入变量 IN1，IN2，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := ELT (IN1, IN2);

控件图示：



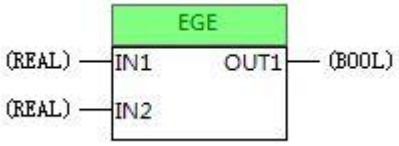
1.1.46 EGE：浮点数类型大于等于比较指令

接口设计：输入变量 IN1，IN2，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := EGE (IN1, IN2);

控件图示：



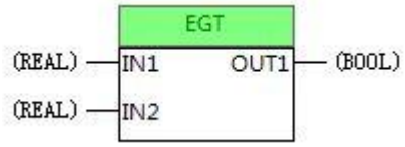
1.1.47 EGT：浮点数类型大于比较指令

接口设计：输入变量 IN1，IN2，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := EGT (IN1, IN2);

控件图示：



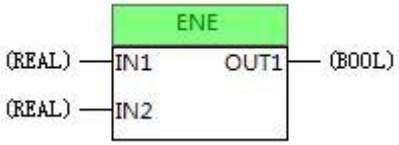
1.1.48 ENE：浮点数类型不等于比较指令

接口设计：输入变量 IN1，IN2，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则 OUT1 = FALSE。

控件文本表示：OUT1 := ENE (IN1, IN2);

控件图示：



1.1.49 EZCP：浮点数类型区间比较指令

接口设计：输入变量 IN1，IN2，IN3，为 REAL 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：如果 MIN(IN2, IN3) < IN1 < MAX(IN2, IN3), 则 OUT1 = TRUE; 否则 OUT1 = FALSE。

控件文本表示：OUT1 := EZCP (IN1, IN2);

控件图示：



1.2 时钟指令

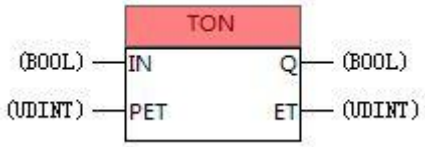
1.2.1 TON：通电延时定时器指令

接口设计：输入变量 IN，PET；输出变量 Q，ET。IN，Q 为 BOOL 类型的变量；PET，ET 为 UDINT 类型的变量。

功能描述：在 IN 端经过一个上升沿且保持为真的情况下，经过 PET 毫秒后，置 Q 为 1。当 IN 为 0 时，Q 和 ET 置 0；当 IN 为 1 时，ET 每毫秒计数加 1，当 ET 与 PET 相等时，Q 置 1。TON 内部存储上一次 IN 的值。

控件文本表示：(Q, ET) := TON (IN, PET);

控件图示：



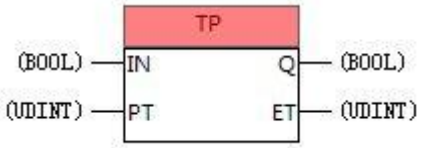
1.2.2 TP：脉冲发生器指令

接口设计：输入变量 IN，PT；输出变量 Q，ET。IN，Q 为 BOOL 类型的变量；PET，ET 为 UDINT 类型的变量。

功能描述：当 IN 端接收到一个上升沿时，Q 置为真且持续 PT 毫秒。TP 内部存储上一次 IN 的值。

控件文本表示：(Q, ET) := TP (IN, PT);

控件图示：



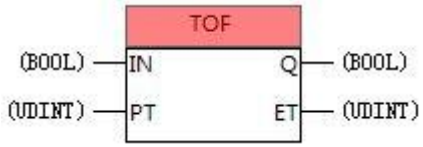
1.2.3 TOF：延时断开计时器指令

接口设计：输入变量 IN，PT；输出变量 Q，ET。IN，Q 为 BOOL 类型的变量；PT，ET 为 UDINT 类型的变量。

功能描述：当 IN 端接为真的情况下，Q 置为真，且在 PT 毫秒后，Q 置为假。TOF 内部存储上一次 IN 的值。

控件文本表示：(Q, ET) := TOF (IN, PT);

控件图示：



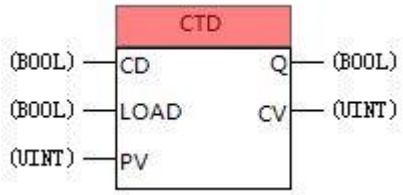
1.2.4 CTD：减计数指令

接口设计：输入变量 CD，LOAD，PV；输出变量 Q，CV。IN，LOAD，Q 为 BOOL 类型的变量；PV，CV 为 UINT 类型的变量。

功能描述：当 CD 端接收到一个上升沿时，CV 减 1；当 CV 为 0 时，Q 置 1。CV 初始值为 PV。LOAD 为复位信号，当 LOAD 为 1 时，Q 置 0，CV 等于 PV。

控件文本表示：(Q, CV) := CTD (CD, LOAD, PV);

控件图示：



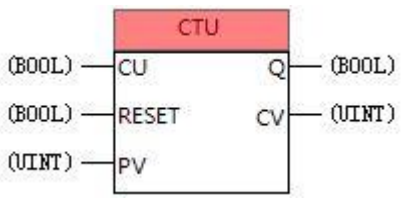
1.2.5 CTU：加计数指令

接口设计：输入变量 CU，RESET，PV；输出变量 Q，CV。CU，RESET，Q 为 BOOL 类型的变量；PV，CV 为 UINT 类型的变量。

功能描述：当 CU 端接收到一个上升沿时，CV 加 1；当 CV=PV 时，Q 置 1。CV 初始值为 0。RESET 为复位信号，当 RESET 为 1 时，Q 置 0，CV 置 0。

控件文本表示：(Q, CV) := CTU (CU, RESET, PV);

控件图示：



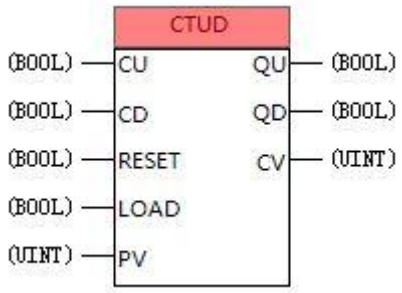
1.2.6 CTUD：可增可减计数指令

接口设计：输入变量 CU，CD，RESET，LOAD，PV；输出变量 QU，QD，CV。CU，CD，RESET，LOAD，QU，QD 为 BOOL 类型的变量；PV，CV 为 UINT 类型的变量。

功能设计：当 CD 端接收到一个上升沿时，CV 减 1；当 CU 端接收到一个上升沿时，CV 加 1；当 CU 端和 CV 端同时接收到上升沿时，无变化。当 CV=0 时，QD 置 1；当 CV=PV 时，QU 置 1。LOAD 为减复位信号，当 LOAD 为 1 时，QD 置 0，CV 等于 PV。RESET 为加复位信号，当 RESET 为 1 时，QU 置 0，CV 置 0。

控件文本表示：(QU, QD, CV) := CTUD(CU, CD, RESET, LOAD, PV);

控件图示：



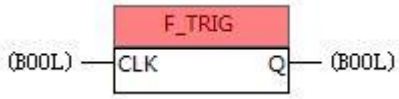
1.2.7 F_TRIG：下降沿功能块指令

接口设计：输入变量 CLK，输出变量 Q，均为 BOOL 类型的变量。

功能描述：当 CLK 从 TRUE 变为 FALSE 时，Q=TRUE，否则 Q=FALSE。F_TRIG 内部保存上一次 CLK 的值。

控件文本表示：Q := F_ TRIG (CLK);

控件图示：



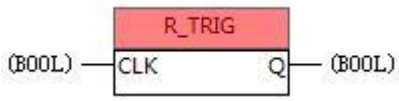
1.2.8 R_TRIG：上升沿功能块指令

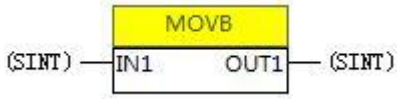
接口设计：输入变量 CLK，输出变量 Q，均为 BOOL 类型的变量。

功能描述：当 CLK 从 FALSE 变为 TRUE 时，Q=TRUE，否则 Q=FALSE。R_TRIG 内部保存上一次 CLK 的值。

控件文本表示：Q := R_ TRIG (CLK);

控件图示：





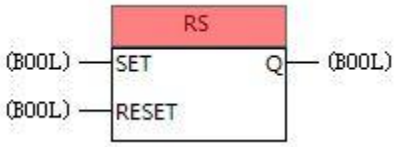
1.2.9 SR：置位优先触发器指令

接口设计：输入变量 SET，RESET，输出变量 Q，均为 BOOL 类型的变量。

功能描述：当置位信号 SET 和复位信号 RESET 都为真时，复位信号优先，Q 置 0。

控件文本表示：Q := SR (SET, RESET);

控件图示：



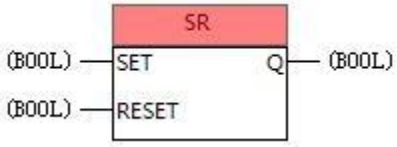
1.2.10 RS：复位优先触发器指令

接口设计：输入变量 SET，RESET，输出变量 Q，均为 BOOL 类型的变量。

功能设计：当置位信号 SET 和复位信号 RESET 都为真时，置位信号优先，Q 置 1。

控件文本表示：Q := RS (SET, RESET);

控件图示：



1.3 传送指令

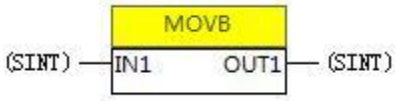
1.3.1 MOVB：8 位传送指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：将 IN1 寄存器的值传送给 OUT1 寄存器。

控件文本表示：OUT1 := MOVB (IN1);

控件图示：



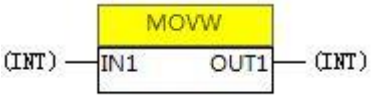
1.3.2 MOVW：16 位传送指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：将 IN1 寄存器的值传送给 OUT1 寄存器。

控件文本表示：OUT1 := MOVW (IN1);

控件图示：



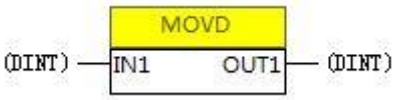
1.3.3 MOVD：32 位传送指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：将 IN1 寄存器的值传送给 OUT1 寄存器。

控件文本表示：OUT1 := MOVD (IN1);

控件图示：



1.3.4 MOVR：浮点数传送指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：将 IN1 寄存器的值传送给 OUT1 寄存器。

控件文本表示：OUT1 := MOVR (IN1);

控件图示：



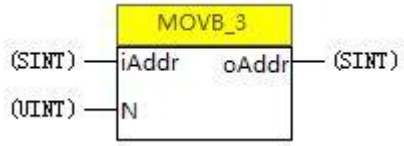
1.3.5 MOVB_3：字节传送指令

接口设计：输入变量 iAddr，N，输出变量 oAddr。iAddr，oAddr 为 SINT 类型的变量，N 为 UINT 类型的变量。

功能描述：将 iAddr 为起始地址的 N 个 Byte 的数据传送到以 oAddr 起始地址的 N 个 Byte 中。

控件文本表示：oAddr := MOVB_3 (iAddr, N);

控件图示：



1.3.6 MOVB_4：传送指令

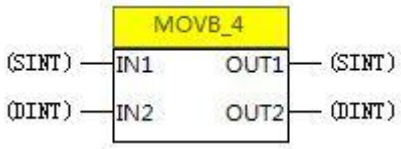
接口设计：输入变量 IN1，IN2，输出变量 OUT1，OUT2。IN1，OUT1 为 SINT 类型的变量，IN2，OUT2 为 DINT 类型的变量。

功能描述：将以 IN1 的地址加上偏移量 IN2 的地址后的一个 Byte 的数据，至 OUT1 地址加

上偏移量 OUT2 地址后的一个 Byte 中。输入地址可以是整个 PLC 空间，输出地址为除了输入 IO 的地址控件。

控件文本表示：(OUT1, OUT2) := MOVB_4(IN1, IN2);

控件图示：



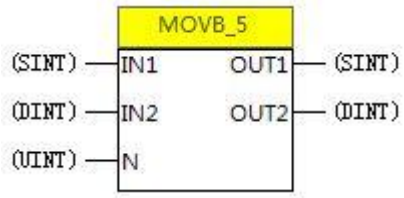
1.3.7 MOVB_5：传送指令

接口设计：输入变量 IN1，IN2，N，输出变量 OUT1，OUT2。IN1，OUT1 为 SINT 类型的变量，IN2，OUT2 为 DINT 类型的变量，N 为 UINT 类型的变量。

功能描述：将以 IN1 的地址加上偏移量 IN2 的地址后的 N 个 Byte 的数据，至 OUT1 地址加上偏移量 OUT2 地址后的 N 个 Byte 中。输入地址可以是整个 PLC 空间，输出地址为除了输入 IO 的地址控件。

控件文本表示：(OUT1, OUT2) := MOVB_5(IN1, IN2, N);

控件图示：



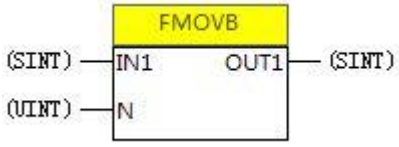
1.3.8 FMOVB：传送指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，OUT1 为 SINT 类型的变量，N 为 UINT 类型的变量。

功能描述：将数据 IN1 传送到 IN2 个以 OUT 为起始地址的连续地址，每个地址的大小为 1 Byte。

控件文本表示：OUT1 := FMOVB (IN1, N);

控件图示：



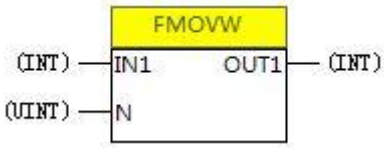
1.3.9 FMOVW：传送指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，OUT1 为 INT 类型的变量，N 为 UINT 类型的变量。

功能描述：将数据 IN1 传送到 IN2 个以 OUT 为起始地址的连续地址，每个地址的大小为 2 Byte。

控件文本表示：OUT1 := FMOVW (IN1, N);

控件图示：



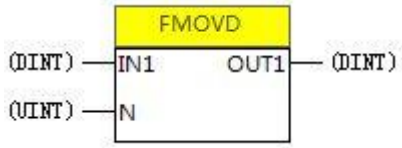
1.3.10 FMOVD：传送指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，OUT1 为 DINT 类型的变量，N 为 UINT 类型的变量。

功能描述：将数据 IN1 传送到 IN2 个以 OUT 为起始地址的连续地址，每个地址的大小为 4 Byte。

控件文本表示：OUT1 := FMOVD (IN1, N);

控件图示：



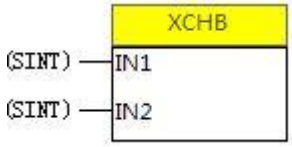
1.3.11 XCHB：8 位数据交换指令

接口设计：输入变量 IN1，IN2，均为 SINT 类型的变量。

功能描述：交换 IN1 和 IN2 的值。

控件文本表示：XCHB (IN1, IN2);

控件图示：



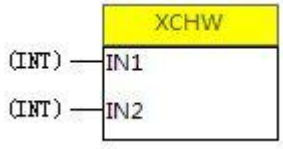
1.3.12 XCHW：16 位数据交换指令

接口设计：输入变量 IN1，IN2，均为 INT 类型的变量。

功能描述：交换 IN1 和 IN2 的值。

控件文本表示：XCHW (IN1, IN2);

控件图示：



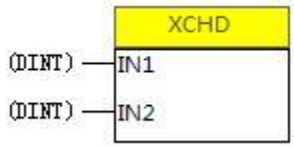
1.3.13 XCHD：32 位数据交换指令

接口设计：输入变量 IN1，IN2，均为 DINT 类型的变量。

功能描述：交换 IN1 和 IN2 的值。

控件文本表示：XCHD (IN1, IN2);

控件图示：



1.4 移位指令

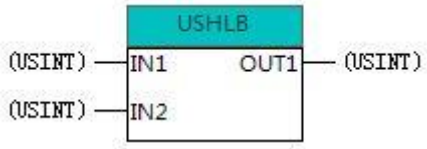
1.4.1 USHLB：8 位无符号逻辑左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑左移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 << IN2。

控件文本表示：OUT1 := USHLB (IN1, IN2);

控件图示：



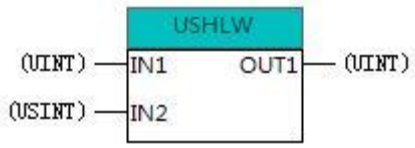
1.4.2 USHLW：16 位无符号逻辑左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑左移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 << IN2。

控件文本表示：OUT1 := USHLW (IN1, IN2);

控件图示：



1.4.3 USHLD：32 位无符号逻辑左移指令

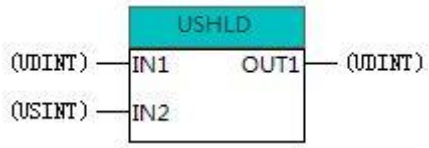
接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UDINT 类型的变量，IN2

为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑左移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 << IN2。

控件文本表示：OUT1 := USHLD (IN1, IN2);

控件图示：



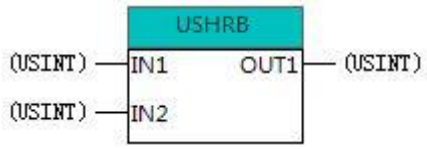
1.4.4 USHRB：8 位无符号逻辑右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑右移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 >> IN2。

控件文本表示：OUT1 := USHRB (IN1, IN2);

控件图示：



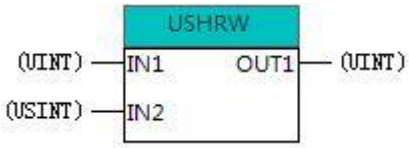
1.4.5 USHRW：16 位无符号逻辑右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑右移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 >> IN2。

控件文本表示：OUT1 := USHLW (IN1, IN2);

控件图示：



1.4.6 USHRD：32 位无符号逻辑右移指令

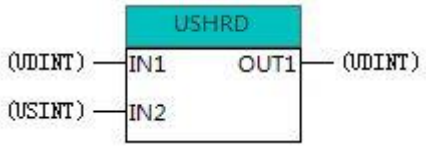
接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UDINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑右移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 >>

IN2。

控件文本表示：OUT1 := USHRD (IN1, IN2);

控件图示：



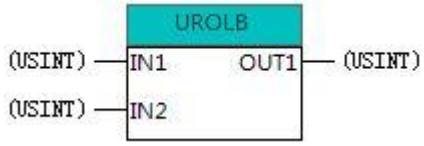
1.4.7 UROLB：8 位无符号循环左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值循环左移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := UROLB (IN1, IN2);

控件图示：



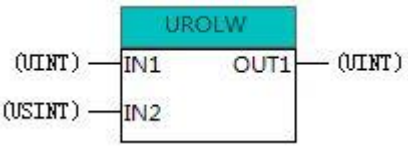
1.4.8 UROLW：16 位无符号循环左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值循环左移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := UROLW (IN1, IN2);

控件图示：



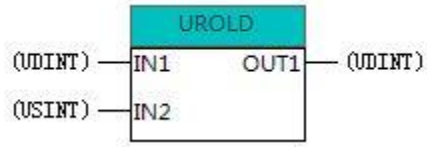
1.4.9 UROLD：32 位无符号循环左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UDINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值循环左移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := UROLD (IN1, IN2);

控件图示：



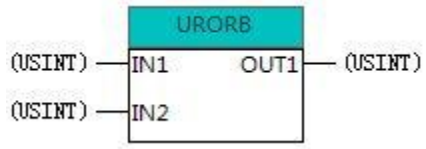
1.4.10 URORB：8 位无符号循环右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值循环右移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URORB (IN1, IN2);

控件图示：



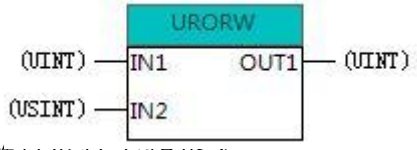
1.4.11 URORW：16 位无符号循环右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值循环右移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URORW (IN1, IN2);

控件图示：



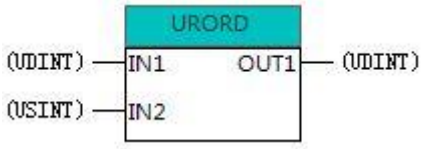
1.4.12 URORD：32 位无符号循环右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UDINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值循环右移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URORD (IN1, IN2);

控件图示：



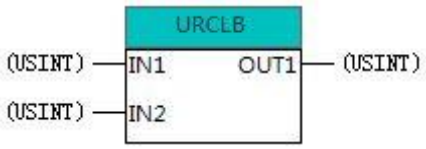
1.4.13 URCLB：8 位无符号带进位循环左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值带进位循环左移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URCLB (IN1, IN2);

控件图示：



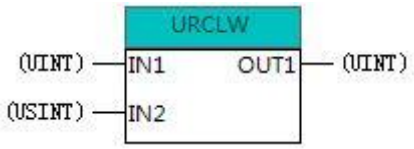
1.4.14 URCLW：16 位无符号带进位循环左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值带进位循环左移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URCLW (IN1, IN2);

控件图示：



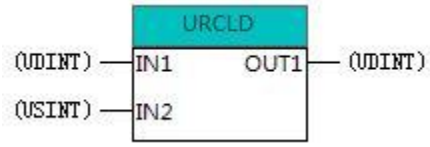
1.4.15 URCLD：32 位无符号带进位循环左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UDINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值带进位循环左移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URCLD (IN1, IN2);

控件图示：



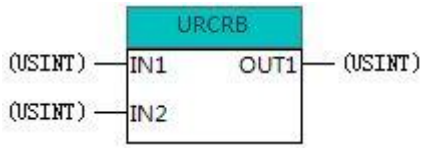
1.4.16 URCRB：8 位无符号带进位循环右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值带进位循环右移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URCRB (IN1, IN2);

控件图示：



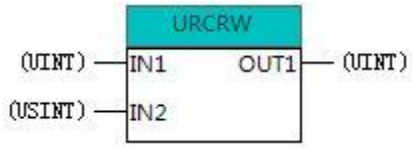
1.4.17 URCRW：16 位无符号带进位循环右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值带进位循环右移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URCRW (IN1, IN2);

控件图示：



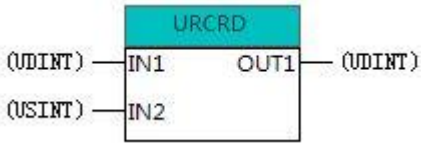
1.4.18 URCRD：32 位无符号带进位循环右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 UDINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值带进位循环右移 IN2 位，并赋值给输出变量 OUT1。

控件文本表示：OUT1 := URCRD (IN1, IN2);

控件图示：



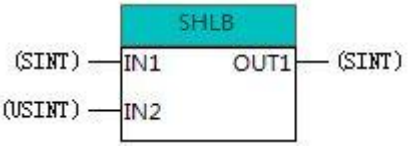
1.4.19 SHLB：8 位有符号逻辑左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 SINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑左移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 << IN2。

控件文本表示：OUT1 := SHLB (IN1, IN2);

控件图示：



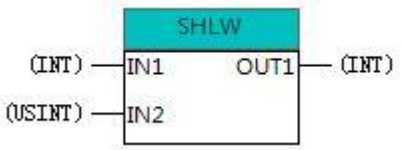
1.4.20 SHLW：16 位有符号

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 INT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑左移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 << IN2。

控件文本表示：OUT1 := SHLW (IN1, IN2);

控件图示：



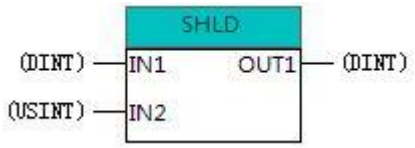
1.4.21 SHLD：32 位有符号逻辑左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 DINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑左移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 << IN2。

控件文本表示：OUT1 := SHLD (IN1, IN2);

控件图示：



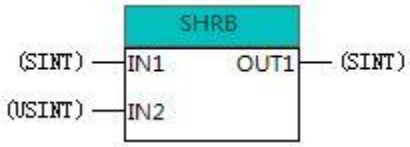
1.4.22 SHRB：8 位有符号逻辑右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 SINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑右移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 >> IN2。

控件文本表示：OUT1 := SHRB (IN1, IN2);

控件图示：



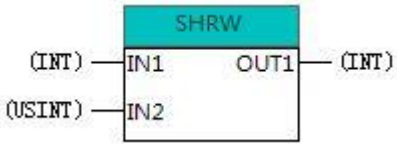
1.4.23 SHRW：16 位有符号逻辑右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 INT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑右移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 >> IN2。

控件文本表示：OUT1 := SHRW (IN1, IN2);

控件图示：



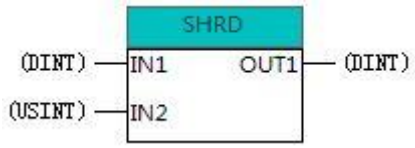
1.4.24 SHRD：32 位有符号逻辑右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 DINT 类型的变量，IN2 为 USINT 类型的变量。

功能描述：将输入变量 IN1 的值逻辑右移 IN2 位，并赋值给输出变量 OUT1，即 OUT1 = IN1 >> IN2。

控件文本表示：OUT1 := SHRD (IN1, IN2);

控件图示：



1.5 逻辑运算

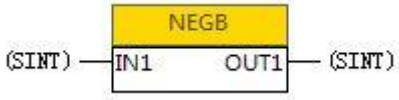
1.5.1 NEGB：8 位补码操作指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 的补码。

控件文本表示：OUT1 := NEGB (IN1);

控件图示：



1.5.2 NEGW：16 位补码操作指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 的补码。

控件文本表示：OUT1 := NEGW (IN1);

控件图示：



1.5.3 NEGD：32 位补码操作指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 的补码。

控件文本表示：OUT1 := NEGD (IN1);

控件图示：



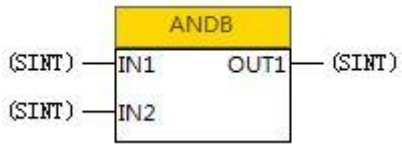
1.5.4 ANDB：8 位按位与操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位与的结果。

控件文本表示：OUT1 := ANDB (IN1, IN2);

控件图示：



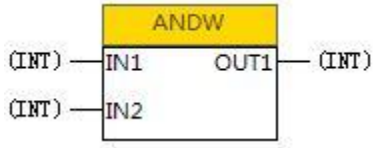
1.5.5 ANDW：16 位按位与操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位与的结果。

控件文本表示：OUT1 := ANDW (IN1, IN2);

控件图示：



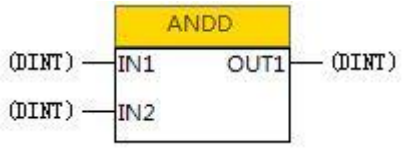
1.5.6 ANDD：32 位按位与操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位与的结果。

控件文本表示：OUT1 := ANDD (IN1, IN2);

控件图示：



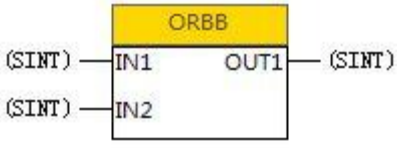
1.5.7 ORBB：8 位按位或操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位或的结果。

控件文本表示：OUT1 := ORBB (IN1, IN2);

控件图示：



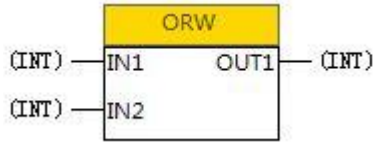
1.5.8 ORW：16 位按位或操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位或的结果。

控件文本表示：OUT1 := ORW (IN1, IN2);

控件图示：



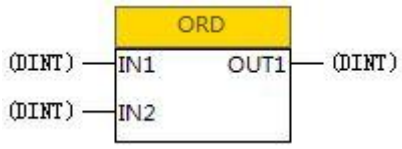
1.5.9 ORD：32 位按位或操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位或的结果。

控件文本表示：OUT1 := ORD (IN1, IN2);

控件图示：



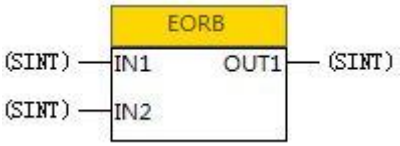
1.5.10 EORB：8 位按位异或操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位异或的结果。

控件文本表示：OUT1 := EORB (IN1, IN2);

控件图示：



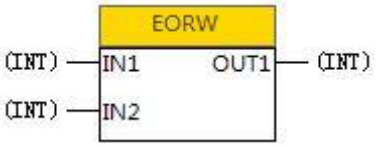
1.5.11 EORW：16 位按位异或操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位异或的结果。

控件文本表示：OUT1 := EORW (IN1, IN2);

控件图示：



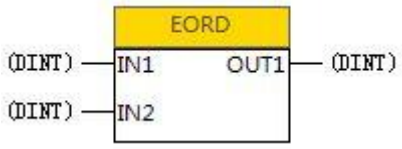
1.5.12 EORD：32 位按位异或操作指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 和 IN2 按位异或的结果。

控件文本表示：OUT1 := EORD (IN1, IN2);

控件图示：



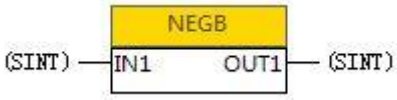
1.5.13 NOTB：8 位按位非操作指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 的按位非的结果。

控件文本表示：OUT1 := NOTB (IN1);

控件图示：



1.5.14 NOTW：16 位按位非操作指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 的按位非的结果。

控件文本表示：OUT1 := NOTW (IN1);

控件图示：



1.5.15 NOTD：32 位按位非操作指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 的按位非的结果。

控件文本表示：OUT1 := NOTD (IN1);

控件图示：



1.6 数据处理

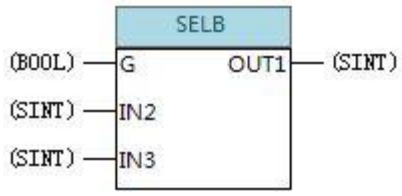
1.6.1 SELB：8 位选择指令

接口设计：输入变量 G，IN1，IN2，输出变量 OUT1。G 为 BOOL 类型的变量；IN1，IN2，OUT1 为 SINT 类型的变量。

功能描述：若 G 为 TRUE，OUT1 = IN1；否则，OUT1 = IN2。

控件文本表示：OUT1 := SELB (G, IN1, IN2);

控件图示：



1.6.2 SELW：16 位选择指令

接口设计：输入变量 G，IN1，IN2，输出变量 OUT1。G 为 BOOL 类型的变量；IN1，IN2，OUT1 为 INT 类型的变量。

功能描述：若 G 为 TRUE，OUT1 = IN1；否则，OUT1 = IN2。

控件文本表示：OUT1 := SELW (G, IN1, IN2);

控件图示：



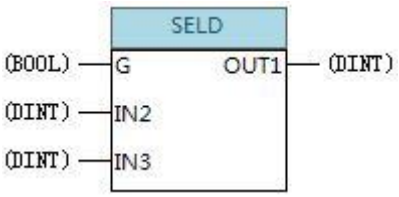
1.6.3 SELD：32 位选择指令

接口设计：输入变量 G，IN1，IN2，输出变量 OUT1。G 为 BOOL 类型的变量；IN1，IN2，OUT1 为 DINT 类型的变量。

功能描述：若 G 为 TRUE，OUT1 = IN1；否则，OUT1 = IN2。

控件文本表示：OUT1 := SELD (G, IN1, IN2);

控件图示：



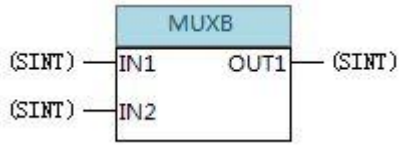
1.6.4 MUXB：8 位数据选择指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述： OUT1 = （起始地址 IN2 + 偏移量 IN1）所指的地址的值。

控件文本表示：OUT1 := MUXB (IN1, IN2);

控件图示：



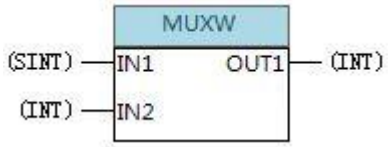
1.6.5 MUXW：16 位数据选择指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1 为 SINT 类型的变量；IN2，OUT1 为 INT 类型的变量。

功能描述： OUT1 = （起始地址 IN2 + 偏移量 IN1）所指的地址的值。

控件文本表示：OUT1 := MUXW (IN1, IN2);

控件图示：



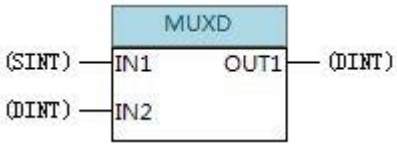
1.6.6 MUXD：32 位数据选择指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1 为 SINT 类型的变量；IN2，OUT1 为 DINT 类型的变量。

功能描述： OUT1 = （起始地址 IN2 + 偏移量 IN1）所指的地址的值。

控件文本表示：OUT1 := MUXD (IN1, IN2);

控件图示：



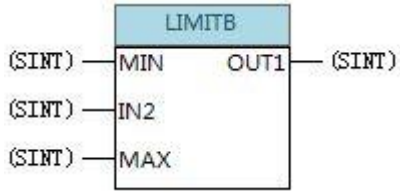
1.6.7 LIMITB：8 位上限/下限控制指令

接口设计：输入变量 MIN，IN2，MAX，输出变量 OUT1，均为 SINT 类型的变量。

功能描述： 若 IN2 > MAX, OUT1 = MAX; 若 IN2 < MIN, OUT1 = MIN; 否则, OUT1 = IN2。

控件文本表示：OUT1 := LIMITB (MIN, IN2, MAX);

控件图示：



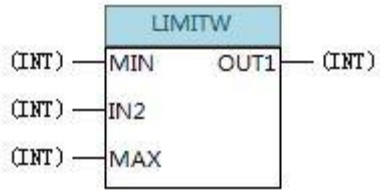
1.6.8 LIMITW：16 位上限/下限控制指令

接口设计：输入变量 MIN，IN2，MAX，输出变量 OUT1，均为 INT 类型的变量。

功能描述： 若 IN2 > MAX, OUT1 = MAX; 若 IN2 < MIN, OUT1 = MIN; 否则, OUT1 = IN2。

控件文本表示：OUT1 := LIMITW (MIN, IN2, MAX);

控件图示：



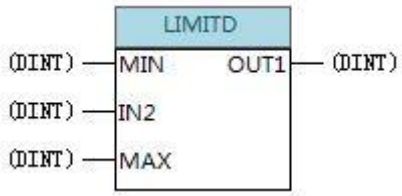
1.6.9 LIMITD：32 位上限/下限控制指令

接口设计：输入变量 MIN，IN2，MAX，输出变量 OUT1，均为 DINT 类型的变量。

功能描述： 若 IN2 > MAX, OUT1 = MAX; 若 IN2 < MIN, OUT1 = MIN; 否则, OUT1 = IN2。

控件文本表示：OUT1 := LIMITD (MIN, IN2, MAX);

控件图示：



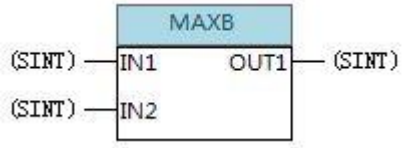
1.6.10 MAXB：8 位取大值指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述： OUT1 取 IN1，IN2 中的较大值。

控件文本表示：OUT1 := MAXB (IN1, IN2);

控件图示：



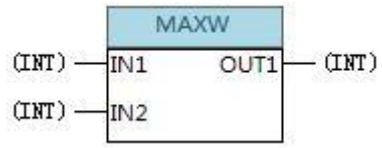
1.6.11 MAXW：16 位取大值指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述： OUT1 取 IN1，IN2 中的较大值。

控件文本表示：OUT1 := MAXW (IN1, IN2);

控件图示：



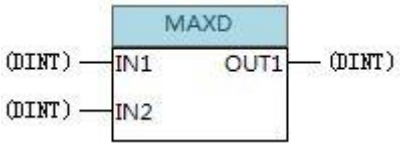
1.6.12 MAXD：32 位取大值指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述： OUT1 取 IN1，IN2 中的较大值。

控件文本表示：OUT1 := MAXD (IN1, IN2);

控件图示：



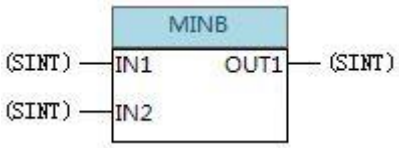
1.6.13 MINB：8 位取小值指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述： OUT1 取 IN1，IN2 中的较小值。

控件文本表示：OUT1 := MINB (IN1, IN2);

控件图示：



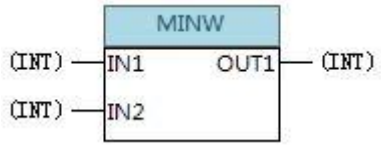
1.6.14 MINW：16 位取小值指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述： OUT1 取 IN1，IN2 中的较小值。

控件文本表示：OUT1 := MINW (IN1, IN2);

控件图示：



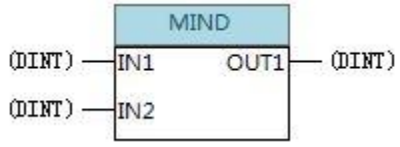
1.6.15 MIND：32 位取小值指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述： OUT1 取 IN1，IN2 中的较小值。

控件文本表示：OUT1 := MIND (IN1, IN2);

控件图示：



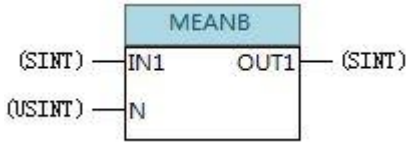
1.6.16 MEANB：字节型平均数指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，OUT1 为 SINT 类型的变量，N 为 USINT 类型的变量。

功能描述：计算从 IN1 起连续 N 个数的平均数，保存至 OUT1 中。每个数大小为 1 Byte，小数部分舍去。

控件文本表示：OUT1 := MEANB (IN1, N);

控件图示：



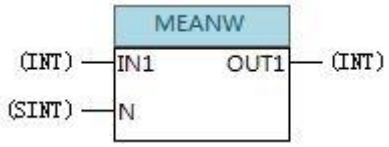
1.6.17 MEANW：字型平均数指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，OUT1 为 INT 类型的变量，N 为 USINT 类型的变量。

功能描述：计算从 IN1 起连续 N 个数的平均数，保存至 OUT1 中。每个数大小为 2 Byte，小数部分舍去。

控件文本表示：OUT1 := MEANW (IN1, N);

控件图示：



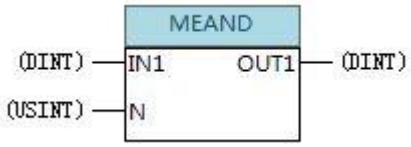
1.6.18 MEAND：双字型平均数指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，OUT1 为 DINT 类型的变量，N 为 USINT 类型的变量。

功能描述：计算从 IN1 起连续 N 个数的平均数，保存至 OUT1 中。每个数大小为 4 Byte，小数部分舍去。

控件文本表示：OUT1 := MEAND (IN1, N);

控件图示：



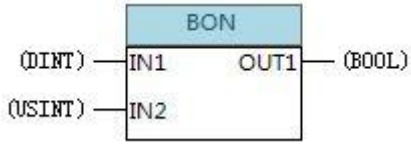
1.6.19 BON：ON 位判断指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1 为 DINT 类型的变量，IN2 为 USINT 类型的变量，OUT1 为 BOOL 类型的变量。

功能描述：若 IN1 二进制值的第 IN2 + 1 位为 1，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := BON (IN1, IN2);

控件图示：



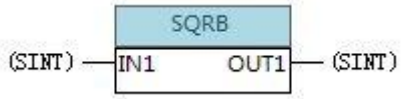
1.6.20 SQRB：字节型开方指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 的正数平方根，小数部分舍去。

控件文本表示：OUT1 := SQRB (IN1);

控件图示：



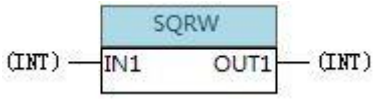
1.6.21 SQRW：字型开方指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 的正数平方根，小数部分舍去。

控件文本表示：OUT1 := SQRW (IN1);

控件图示：



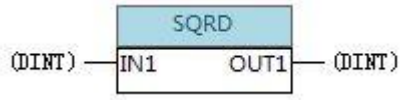
1.6.22 SQRD：双字型开方指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 的正数平方根，小数部分舍去。

控件文本表示：OUT1 := SQRD (IN1);

控件图示：



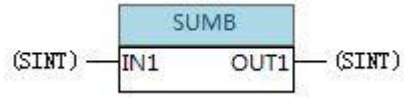
1.6.23 SUMB：字节型置 ON 位总和指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：将 IN1 二进制值中，值为 1 的位的个数，保存到 OUT1 中。

控件文本表示：OUT1 := SUMB (IN1);

控件图示：



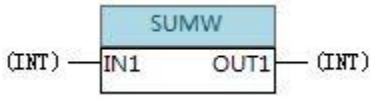
1.6.24 SUMW：字型置 ON 位总和指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：将 IN1 二进制值中，值为 1 的位的个数，保存到 OUT1 中。

控件文本表示：OUT1 := SUMW (IN1);

控件图示：



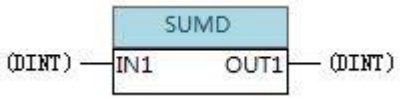
1.6.25 SUMD：双字型置 ON 位总和指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：将 IN1 二进制值中，值为 1 的位的个数，保存到 OUT1 中。

控件文本表示：OUT1 := SUMD (IN1);

控件图示：



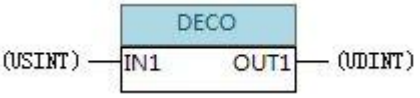
1.6.26 DECO：译码指令

接口设计：输入变量 IN1，输出变量 OUT1。IN1 为 USINT 类型的变量，OUT1 为 UDINT 类型的变量。

功能描述：当 IN1 的值在 0 至 31 之间时，将该值译为位号，将 OUT1 二进制值的该位号对应的位置 1，其他位置 0。

控件文本表示：OUT1 := DECO (IN1);

控件图示：



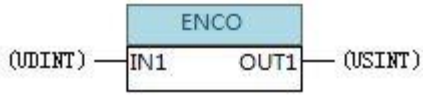
1.6.27 ENCO：编码指令

接口设计：输入变量 IN1，输出变量 OUT1。IN1 为 UDINT 类型的变量，OUT1 为 USINT 类型的变量。

功能描述：记录 IN1 的二进制值中值为 1 的最低位，将该位号保存至 OUT1 中。

控件文本表示：OUT1 := ENCO (IN1);

控件图示：



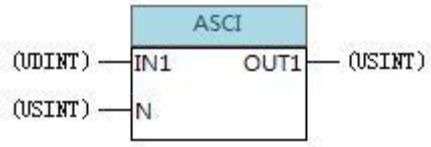
1.6.28 ASCI：ASCII 转换指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1 为 UDINT 类型的变量；N，OUT1 为 USINT 类型的变量。

功能描述：将十六进制树 IN1 转换成 ACSII 码，并保存至 OUT1，转换的字符数由 N（N <= 8）指定。

控件文本表示：OUT1 := ASCI (IN1，N);

控件图示：



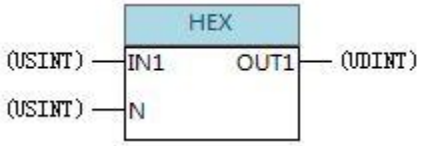
1.6.29 HEX：十六进制转换指令

接口设计：输入变量 IN1，N，输出变量 OUT1。IN1，N 为 USINT 类型的变量；OUT1 为 UDINT 类型的变量。

功能描述：将以 IN1 为起始地址开始的 N 个（N <= 8）ASCII 码值转换为十六进制数，并保存至 OUT1 中。

控件文本表示：OUT1 := HEX (IN1，N);

控件图示：



1.7 算术运算

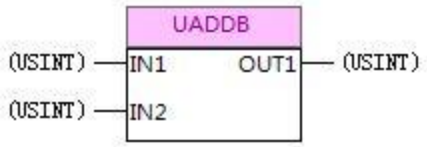
1.7.1 UADDB：无符号字节型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := UADDB (IN1，IN2);

控件图示：



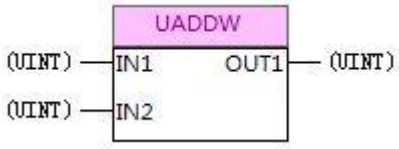
1.7.2 UADDW：无符号字型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := UADDW (IN1，IN2);

控件图示：



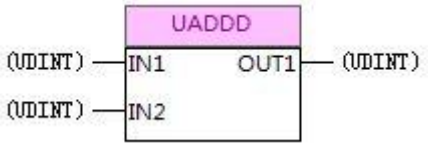
1.7.3 UADDD：无符号双字型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := UADDD (IN1，IN2);

控件图示：



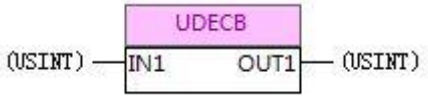
1.7.4 UDECB：无符号字节型自减指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 - 1。

控件文本表示：OUT1 := UDECB (IN1);

控件图示：



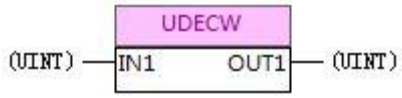
1.7.5 UDECW：无符号字型自减指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 - 1。

控件文本表示：OUT1 := UDECW (IN1);

控件图示：



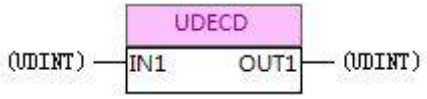
1.7.6 UDECD：无符号双字型自减指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 - 1。

控件文本表示：OUT1 := UDECD (IN1);

控件图示：



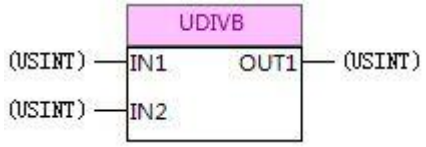
1.7.7 UDIVB：无符号字节型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := UDIVB (IN1, IN2);

控件图示：



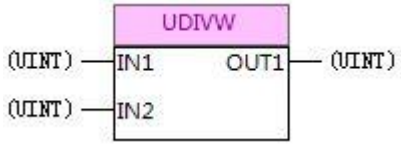
1.7.8 UDIVW：无符号字型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := UDIVW (IN1, IN2);

控件图示：



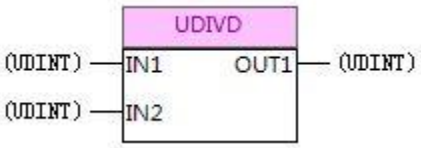
1.7.9 UDIVD：无符号双字型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := UDIVD (IN1, IN2);

控件图示：



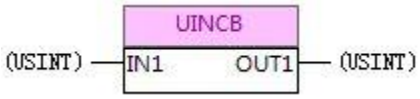
1.7.10 UINCB：无符号字节型自加指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 + 1。

控件文本表示：OUT1 := UINCB (IN1);

控件图示：



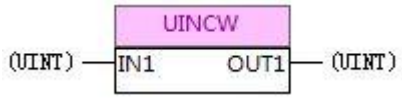
1.7.11 UINCW：无符号字型自加指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 + 1。

控件文本表示：OUT1 := UINCW (IN1);

控件图示：



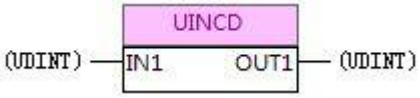
1.7.12 UINCD：无符号双字型自加指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 + 1。

控件文本表示：OUT1 := UINCD (IN1);

控件图示：



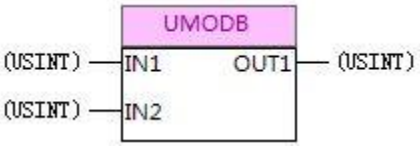
1.7.13 UMODB：无符号字节型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := UMODB (IN1, IN2);

控件图示：



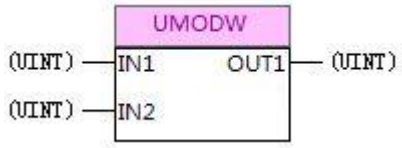
1.7.14 UMODW：无符号字型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := UMODW (IN1, IN2);

控件图示：



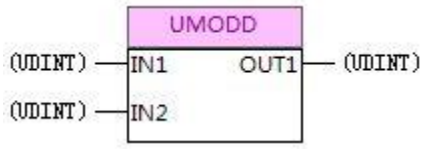
1.7.15 UMODD：无符号双字型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := UMODD (IN1, IN2);

控件图示：



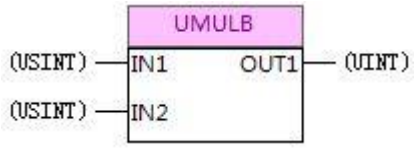
1.7.16 UMULB：无符号字节型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := UMULB (IN1, IN2);

控件图示：



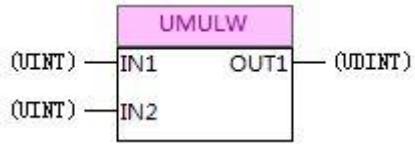
1.7.17 UMULW：无符号字型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := UMULW (IN1, IN2);

控件图示：



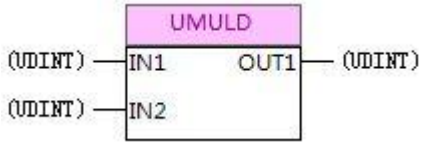
1.7.18 UMULD：无符号双字节型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := UMULD (IN1, IN2);

控件图示：



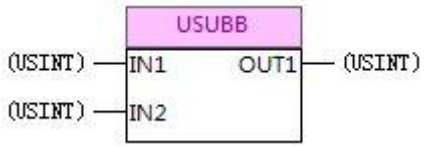
1.7.19 USUBB：无符号字节型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 USINT 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := USUBB (IN1, IN2);

控件图示：



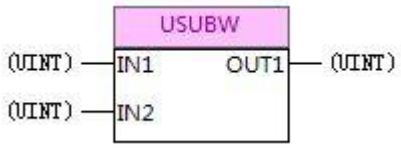
1.7.20 USUBW：无符号字型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := USUBW (IN1, IN2);

控件图示：



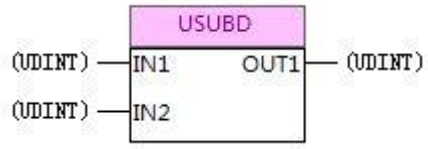
1.7.21 USUBD：无符号双字型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UDINT 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := USUBD (IN1, IN2);

控件图示：



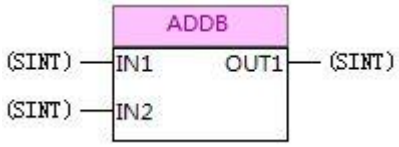
1.7.22 ADDB：有符号字节型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := ADDB (IN1, IN2);

控件图示：



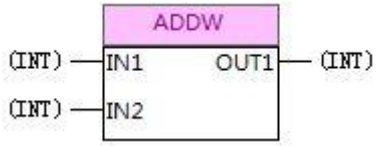
1.7.23 ADDW：有符号字型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := ADDW (IN1, IN2);

控件图示：



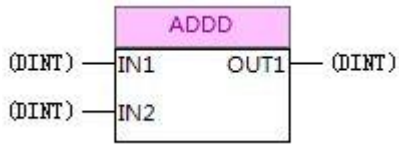
1.7.24 ADDD：有符号双字型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := ADDD (IN1, IN2);

控件图示：



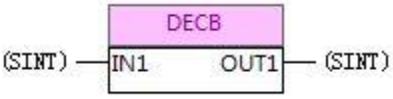
1.7.25 DECB：有符号字节型自减指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 - 1。

控件文本表示：OUT1 := DECB (IN1);

控件图示：



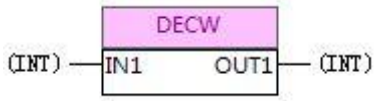
1.7.26 DECW：有符号字型自减指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 - 1。

控件文本表示：OUT1 := DECW (IN1);

控件图示：



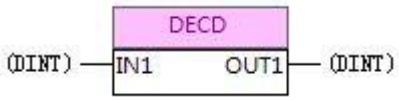
1.7.27 DECD：有符号双字型自减指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 - 1。

控件文本表示：OUT1 := DECD (IN1);

控件图示：



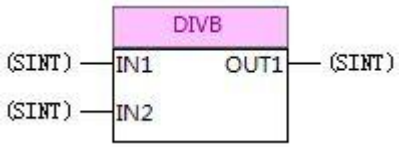
1.7.28 DIVB：有符号字节型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := DIVB (IN1, IN2);

控件图示：



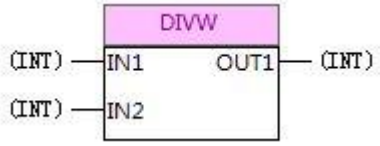
1.7.29 DIVW：有符号字型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := DIVW (IN1, IN2);

控件图示：



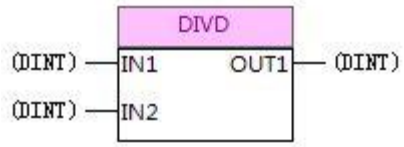
1.7.30 DIVD：有符号双字型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := DIVD (IN1, IN2);

控件图示：



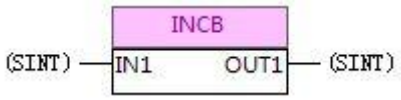
1.7.31 INCB：有符号字节型自加指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 + 1。

控件文本表示：OUT1 := INCB (IN1);

控件图示：



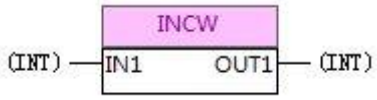
1.7.32 INCW：有符号字型自加指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 + 1。

控件文本表示：OUT1 := INCW (IN1);

控件图示：



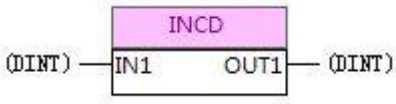
1.7.33 INCD：有符号双字型自加指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 + 1。

控件文本表示：OUT1 := INCD (IN1);

控件图示：



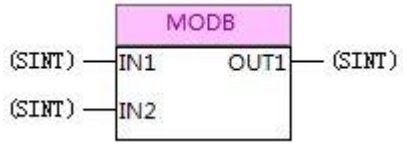
1.7.34 MODB：有符号字节型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := MODB (IN1, IN2);

控件图示：



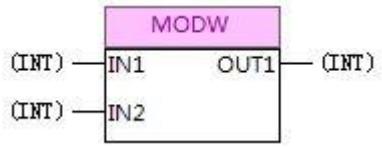
1.7.35 MODW：有符号字型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := MODW (IN1, IN2);

控件图示：



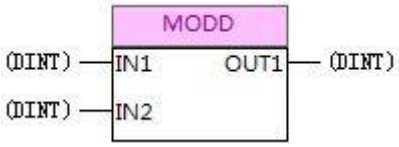
1.7.36 MODD：有符号双字型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := MODD (IN1, IN2);

控件图示：



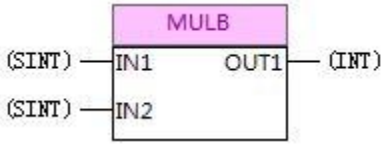
1.7.37 MULB：有符号字节型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := MULB (IN1, IN2);

控件图示：



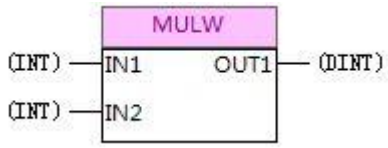
1.7.38 MULW：有符号字型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := MULW (IN1, IN2);

控件图示：



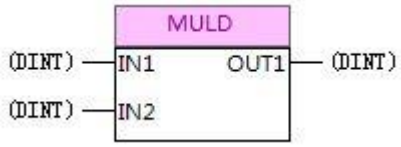
1.7.39 MULD：有符号双字型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := MULD (IN1, IN2);

控件图示：



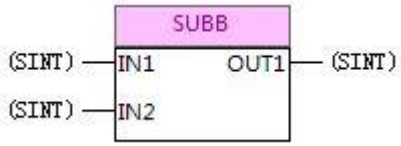
1.7.40 SUBB：有符号字节型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 SINT 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := SUBB (IN1, IN2);

控件图示：



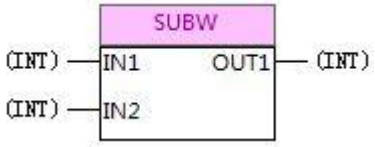
1.7.41 SUBW：有符号字型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 INT 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := SUBW (IN1, IN2);

控件图示：



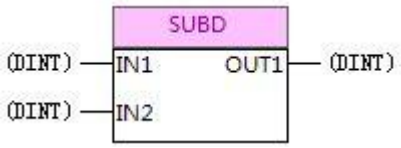
1.7.42 SUBD：有符号双字型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 DINT 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := SUBD (IN1, IN2);

控件图示：



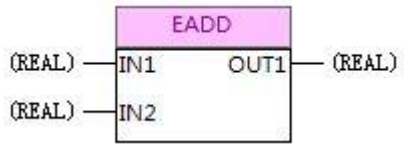
1.7.43 EADD：浮点数加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := EADD (IN1, IN2);

控件图示：



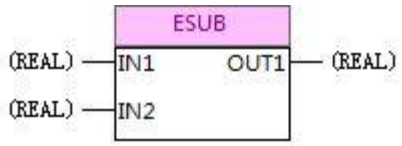
1.7.44 ESUB：浮点数减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := ESUB (IN1, IN2);

控件图示：



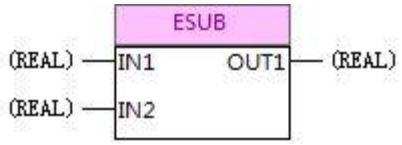
1.7.45 EMUL：浮点数乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := EMUL (IN1, IN2);

控件图示：



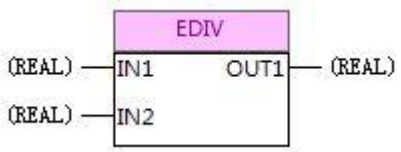
1.7.46 EDIV：浮点数除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := EDIV (IN1, IN2);

控件图示：



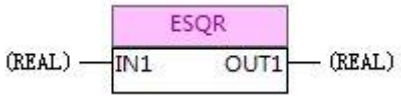
1.7.47 ESQR：浮点数开方指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = IN1 的正数平方根。

控件文本表示：OUT1 := ESQR (IN1);

控件图示：



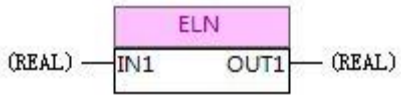
1.7.48 ELN：浮点数 e 为底对数指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = ln(IN1)。

控件文本表示：OUT1 := ELN (IN1);

控件图示：



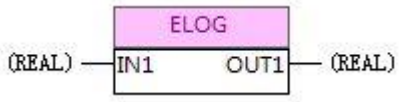
1.7.49 ELOG：浮点数 10 为底对数指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = $\log_{10}$(IN1)。

控件文本表示：OUT1 := ELOG (IN1);

控件图示：



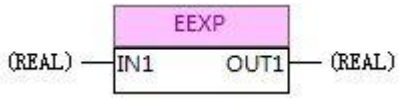
1.7.50 EEXP：浮点数 e 为底幂指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = exp(IN1)。

控件文本表示：OUT1 := EEXP (IN1);

控件图示：



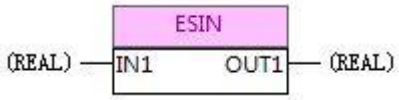
1.7.51 ESIN：浮点数正弦指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = sin(IN1)。

控件文本表示：OUT1 := ESIN (IN1);

控件图示：



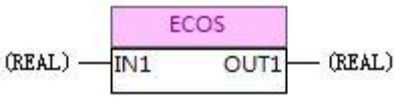
1.7.52 ECOS：浮点数余弦指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = cos(IN1)。

控件文本表示：OUT1 := ECOS (IN1);

控件图示：



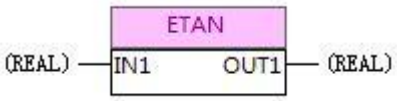
1.7.53 ETAN：浮点数正切指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = tan(IN1)。

控件文本表示：OUT1 := ETAN (IN1);

控件图示：



1.7.54 EASIN：浮点数反正弦指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = arcsin(IN1)。

控件文本表示：OUT1 := EASIN (IN1);

控件图示：



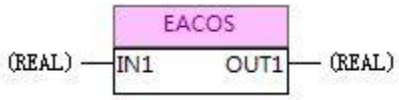
1.7.55 EACOS：浮点数反余弦指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = arccos(IN1)。

控件文本表示：OUT1 := EACOS (IN1);

控件图示：



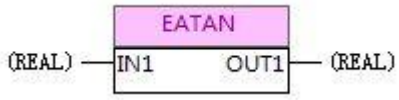
1.7.56 EATAN：浮点数反正切指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：OUT1 = arctan(IN1)。

控件文本表示：OUT1 := EATAN (IN1);

控件图示：



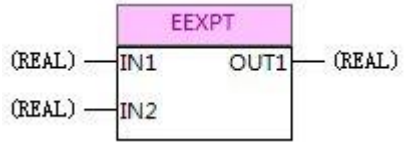
1.7.57 EEXPT：浮点数幂指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 REAL 类型的变量。

功能描述：$OUT1 = IN1^{IN2}$。

控件文本表示：OUT1 := EEXPT (IN1, IN2);

控件图示：



1.8 无类型运算

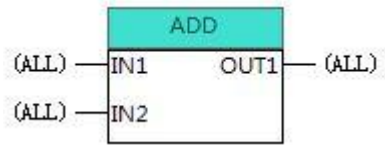
1.8.1 ADD：无类型加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，变量可为任意类型。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := ADD (IN1, IN2);

控件图示：



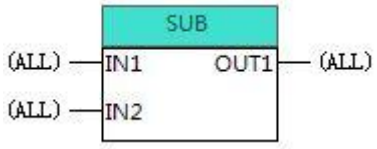
1.8.2 SUB：无类型减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，变量可为任意类型。

功能描述：OUT1 = IN1 - IN2。

控件文本表示：OUT1 := SUB (IN1, IN2);

控件图示：



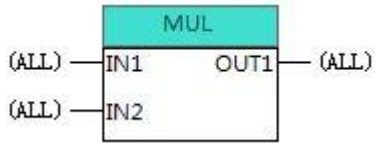
1.8.3 MUL：无类型乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，变量可为任意类型。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := MUL (IN1, IN2);

控件图示：



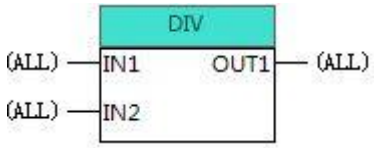
1.8.4 DIV：无类型除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，变量可为任意类型。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := DIV (IN1, IN2);

控件图示：



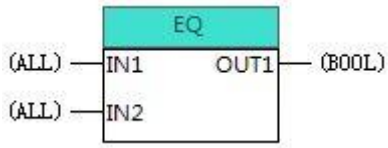
1.8.5 EQ：无类型比较等于指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 可为任意类型，OUT1 为 BOOL 类型的变量。

功能描述：如果 IN1 = IN2，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := EQ (IN1, IN2);

控件图示：



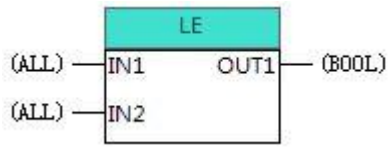
1.8.6 LE：无类型比较小于等于指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 可为任意类型，OUT1 为 BOOL 类型的变量。

功能描述：如果 IN1 <= IN2，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := LE (IN1, IN2);

控件图示：



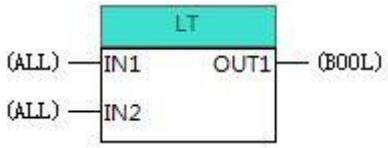
1.8.7 LT：无类型比较小于指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 可为任意类型，OUT1 为 BOOL 类型的变量。

功能描述：如果 IN1 < IN2，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := LT (IN1, IN2);

控件图示：



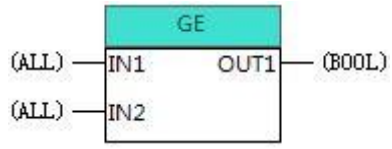
1.8.8 GE：无类型比较大于等于指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 可为任意类型，OUT1 为 BOOL 类型的变量。

功能描述：如果 IN1 >= IN2，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := GE (IN1, IN2);

控件图示：



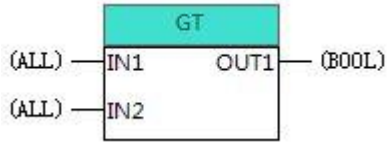
1.8.9 GT：无类型比较大于指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 可为任意类型，OUT1 为 BOOL 类型的变量。

功能描述：如果 IN1 > IN2，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := GT (IN1, IN2);

控件图示：



1.8.10 NE：无类型比较不等于指令

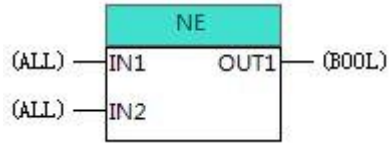
接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 可为任意类型，OUT1 为 BOOL

类型的变量。

功能描述：如果 IN1 ≠ IN2，则 OUT1 = TRUE；否则，OUT1 = FALSE。

控件文本表示：OUT1 := NE (IN1, IN2);

控件图示：



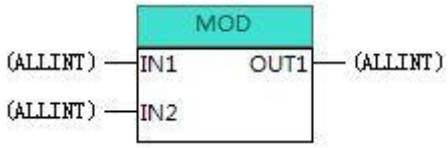
1.8.11 MOD：无类型取模指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，可为任意整数类型。

功能描述：OUT1 = IN1 % IN2。

控件文本表示：OUT1 := MOD (IN1, IN2);

控件图示：



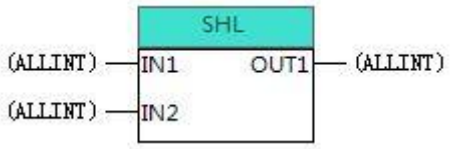
1.8.12 SHL：无类型左移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，可为任意整数类型。

功能描述：OUT1 = IN1 << IN2。

控件文本表示：OUT1 := SHL (IN1, IN2);

控件图示：



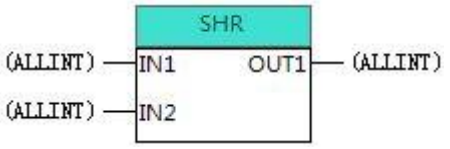
1.8.13 SHR：无类型右移指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，可为任意整数类型。

功能描述：OUT1 = IN1 >> IN2。

控件文本表示：OUT1 := SHR (IN1, IN2);

控件图示：



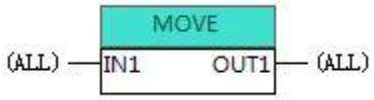
1.8.14 MOVE：无类型传送指令

接口设计：输入变量 IN1，输出变量 OUT1，可为任意类型。

功能描述：将寄存器 IN1 的值传送至寄存器 OUT1。

控件文本表示：OUT1 := MOVE (IN1);

控件图示：



1.9 外部设备 I/O

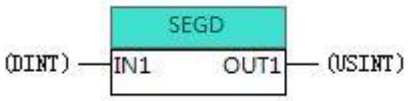
1.9.1 SEGD：7 段译码器指令

接口设计：输入变量 IN1，为 DINT 类型的变量；输出变量 OUT1，为 USINT 类型的变量。

功能描述：将 IN1 低 4 位指定的 0-F（16 进制）数据译为七段码显示的数据传送到 OUT1 指定的元件中。

控件文本表示：OUT1 := SEGD (IN1);

控件图示：



1.10 数据类型转换

1.10.1 SINT_TO_USINT：有符号字节型转无符号字节型指令

接口设计：输入变量 IN1，为 SINT 类型；输出变量 OUT1，为 USINT 类型。

功能描述：将 SINT 的输入变量转换为 USINT 的输出变量。

控件文本表示：OUT1 := SINT_TO_USINT(IN1);

控件图示：



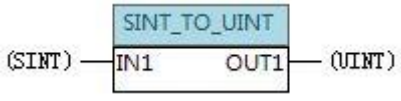
1.10.2 SINT_TO_UINT：有符号字节型转无符号字型指令

接口设计：输入变量 IN1，为 SINT 类型；输出变量 OUT1，为 UINT 类型。

功能描述：将 SINT 的输入变量转换为 UINT 的输出变量。

控件文本表示：OUT1 := SINT_TO_UINT (IN1);

控件图示：



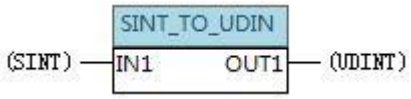
1.10.3 SINT_TO_UDINT：有符号字节型转无符号双字型指令

接口设计：输入变量 IN1，为 SINT 类型；输出变量 OUT1，为 UDINT 类型。

功能描述：将 SINT 的输入变量转换为 UDINT 的输出变量。

控件文本表示：OUT1 := SINT_TO_ UDINT (IN1);

控件图示：



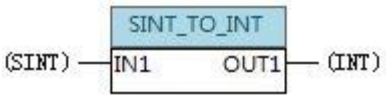
1.10.4 SINT_TO_INT：有符号字节型转有符号字型指令

接口设计：输入变量 IN1，为 SINT 类型；输出变量 OUT1，为 INT 类型。

功能描述：将 SINT 的输入变量转换为 INT 的输出变量。

控件文本表示：OUT1 := SINT_TO_ INT (IN1);

控件图示：



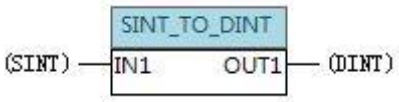
1.10.5 SINT_TO_DINT：有符号字节型转无符号双字型指令

接口设计：输入变量 IN1，为 SINT 类型；输出变量 OUT1，为 DINT 类型。

功能描述：将 SINT 的输入变量转换为 DINT 的输出变量。

控件文本表示：OUT1 := SINT_TO_ DINT (IN1);

控件图示：



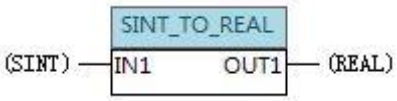
1.10.6 SINT_TO_REAL：有符号字节型转浮点型指令

接口设计：输入变量 IN1，为 SINT 类型；输出变量 OUT1，为 REAL 类型。

功能描述：将 SINT 的输入变量转换为 REAL 的输出变量。

控件文本表示：OUT1 := SINT_TO_ REAL (IN1);

控件图示：



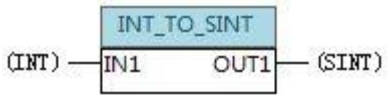
1.10.7 INT_TO_USINT：有符号字型转无符号字节型指令

接口设计：输入变量 IN1，为 INT 类型；输出变量 OUT1，为 USINT 类型。

功能描述：将 INT 的输入变量转换为 USINT 的输出变量。

控件文本表示：OUT1 := INT_TO_ USINT (IN1);

控件图示：



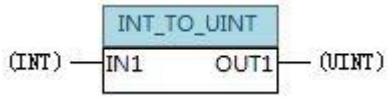
1.10.8 INT_TO_UINT：有符号字型转无符号字型指令

接口设计：输入变量 IN1，为 INT 类型；输出变量 OUT1，为 UINT 类型。

功能描述：将 INT 的输入变量转换为 UINT 的输出变量。

控件文本表示：OUT1 := INT_TO_ UINT (IN1);

控件图示：



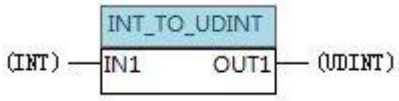
1.10.9 INT_TO_UDINT：有符号字型转无符号双字型指令

接口设计：输入变量 IN1，为 INT 类型；输出变量 OUT1，为 UDINT 类型。

功能描述：将 INT 的输入变量转换为 UDINT 的输出变量。

控件文本表示：OUT1 := INT_TO_ UDINT (IN1);

控件图示：



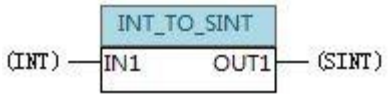
1.10.10 INT_TO_SINT：有符号字型转有符号字节型指令

接口设计：输入变量 IN1，为 INT 类型；输出变量 OUT1，为 SINT 类型。

功能描述：将 INT 的输入变量转换为 SINT 的输出变量。

控件文本表示：OUT1 := INT_TO_ SINT (IN1);

控件图示：



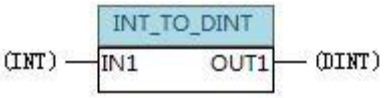
1.10.11 INT_TO_DINT：有符号字型转有符号双字型指令

接口设计：输入变量 IN1，为 INT 类型；输出变量 OUT1，为 DINT 类型。

功能描述：将 INT 的输入变量转换为 DINT 的输出变量。

控件文本表示：OUT1 := INT_TO_ DINT (IN1);

控件图示：



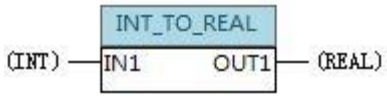
1.10.12 INT_TO_REAL：有符号字型转浮点型指令

接口设计：输入变量 IN1，为 INT 类型；输出变量 OUT1，为 REAL 类型。

功能描述：将 INT 的输入变量转换为 REAL 的输出变量。

控件文本表示：OUT1 := INT_TO_ REAL (IN1);

控件图示：



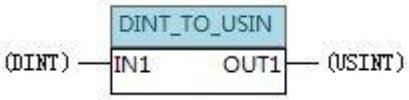
1.10.13 DINT_TO_USINT：有符号双字型转无符号字节型指令

接口设计：输入变量 IN1，为 DINT 类型；输出变量 OUT1，为 USINT 类型。

功能描述：将 DINT 的输入变量转换为 USINT 的输出变量。

控件文本表示：OUT1 := DINT _TO_ USINT (IN1);

控件图示：



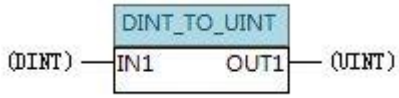
1.10.14 DINT _TO_UINT：有符号双字型转无符号字型指令

接口设计：输入变量 IN1，为 DINT 类型；输出变量 OUT1，为 UINT 类型。

功能描述：将 DINT 的输入变量转换为 UINT 的输出变量。

控件文本表示：OUT1 := DINT _TO_ UINT (IN1);

控件图示：



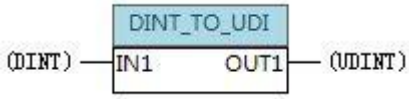
1.10.15 DINT _TO_UDINT：有符号双字型转无符号双字型指令

接口设计：输入变量 IN1，为 DINT 类型；输出变量 OUT1，为 UDINT 类型。

功能描述：将 DINT 的输入变量转换为 UDINT 的输出变量。

控件文本表示：OUT1 := DINT _TO_ UDINT (IN1);

控件图示：



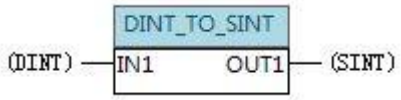
1.10.16 DINT _TO_SINT：有符号双字型转有符号字节型指令

接口设计：输入变量 IN1，为 DINT 类型；输出变量 OUT1，为 SINT 类型。

功能描述：将 DINT 的输入变量转换为 SINT 的输出变量。

控件文本表示：OUT1 := DINT _TO_ SINT (IN1);

控件图示：



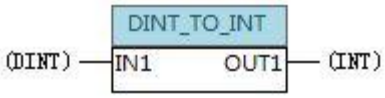
1.10.17 DINT _TO_INT：有符号双字型转有符号字型指令

接口设计：输入变量 IN1，为 DINT 类型；输出变量 OUT1，为 INT 类型。

功能描述：将 DINT 的输入变量转换为 INT 的输出变量。

控件文本表示：OUT1 := DINT _TO_ INT (IN1);

控件图示：



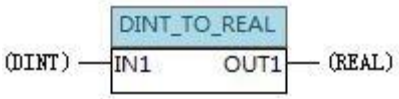
1.10.18 DINT _TO_REAL：有符号双字型转浮点型指令

接口设计：输入变量 IN1，为 DINT 类型；输出变量 OUT1，为 REAL 类型。

功能描述：将 DINT 的输入变量转换为 REAL 的输出变量。

控件文本表示：OUT1 := DINT _TO_ REAL (IN1);

控件图示：



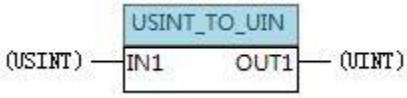
1.10.19 USINT _TO_UINT：无符号字节型转无符号字型指令

接口设计：输入变量 IN1，为 USINT 类型；输出变量 OUT1，为 UINT 类型。

功能描述：将 USINT 的输入变量转换为 UINT 的输出变量。

控件文本表示：OUT1 := USINT _TO_ UINT (IN1);

控件图示：



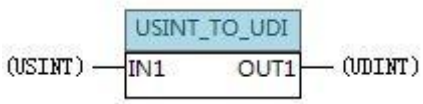
1.10.20 USINT _TO_UDINT：无符号字节型转无符号双字型指令

接口设计：输入变量 IN1，为 USINT 类型；输出变量 OUT1，为 UDINT 类型。

功能描述：将 USINT 的输入变量转换为 UDINT 的输出变量。

控件文本表示：OUT1 := USINT _TO_ UDINT (IN1);

控件图示：



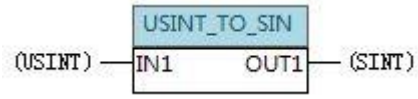
1.10.21 USINT _TO_SINT：无符号字节型转有符号字节型指令

接口设计：输入变量 IN1，为 USINT 类型；输出变量 OUT1，为 SINT 类型。

功能描述：将 USINT 的输入变量转换为 SINT 的输出变量。

控件文本表示：OUT1 := USINT _TO_ SINT (IN1);

控件图示：



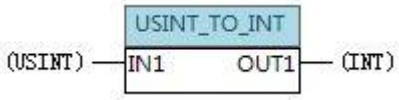
1.10.22 USINT _TO_INT：无符号字节型转有符号字型指令

接口设计：输入变量 IN1，为 USINT 类型；输出变量 OUT1，为 INT 类型。

功能描述：将 USINT 的输入变量转换为 INT 的输出变量。

控件文本表示：OUT1 := USINT _TO_ INT (IN1);

控件图示：



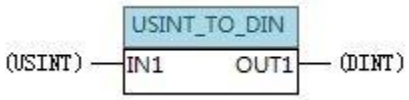
1.10.23 USINT _TO_DINT：无符号字节型转有符号双字型指令

接口设计：输入变量 IN1，为 USINT 类型；输出变量 OUT1，为 DINT 类型。

功能描述：将 USINT 的输入变量转换为 DINT 的输出变量。

控件文本表示：OUT1 := USINT _TO_ DINT (IN1);

控件图示：



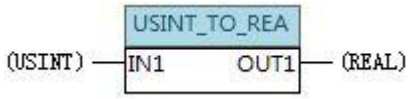
1.10.24 USINT _TO_REAL：无符号字节型转浮点型指令

接口设计：输入变量 IN1，为 USINT 类型；输出变量 OUT1，为 REAL 类型。

功能描述：将 USINT 的输入变量转换为 REAL 的输出变量。

控件文本表示：OUT1 := USINT _TO_ REAL (IN1);

控件图示：



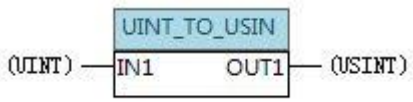
1.10.25 UINT_TO_USINT：无符号字型转无符号字节型指令

接口设计：输入变量 IN1，为 UINT 类型；输出变量 OUT1，为 USINT 类型。

功能描述：将 UINT 的输入变量转换为 USINT 的输出变量。

控件文本表示：OUT1 := UINT _TO_ USINT (IN1);

控件图示：



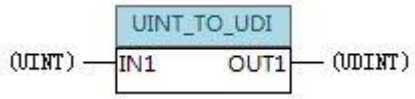
1.10.26 UINT_TO_UDINT：无符号字型转无符号双字型指令

接口设计：输入变量 IN1，为 UINT 类型；输出变量 OUT1，为 UDINT 类型。

功能描述：将 UINT 的输入变量转换为 UDINT 的输出变量。

控件文本表示：OUT1 := UINT _TO_ UDINT (IN1);

控件图示：



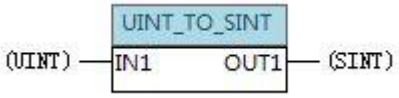
1.10.27 UINT_TO_SINT：无符号字型转有符号字节型指令

接口设计：输入变量 IN1，为 UINT 类型；输出变量 OUT1，为 SINT 类型。

功能描述：将 UINT 的输入变量转换为 SINT 的输出变量。

控件文本表示：OUT1 := UINT _TO_ SINT (IN1);

控件图示：



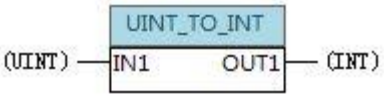
1.10.28 UINT_TO_INT：无符号字型转有符号字型指令

接口设计：输入变量 IN1，为 UINT 类型；输出变量 OUT1，为 INT 类型。

功能描述：将 UINT 的输入变量转换为 INT 的输出变量。

控件文本表示：OUT1 := UINT _TO_ INT (IN1);

控件图示：



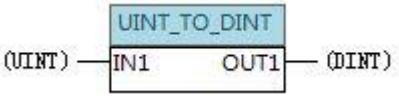
1.10.29 UINT_TO_DINT：无符号字型转有符号双字型指令

接口设计：输入变量 IN1，为 UINT 类型；输出变量 OUT1，为 DINT 类型。

功能描述：将 UINT 的输入变量转换为 DINT 的输出变量。

控件文本表示：OUT1 := UINT _TO_ DINT (IN1);

控件图示：



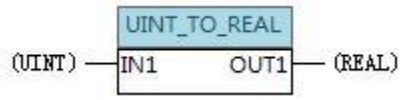
1.10.30 UINT_TO_REAL：无符号字型转浮点型指令

接口设计：输入变量 IN1，为 UINT 类型；输出变量 OUT1，为 REAL 类型。

功能描述：将 UINT 的输入变量转换为 REAL 的输出变量。

控件文本表示：OUT1 := UINT _TO_ REAL (IN1);

控件图示：



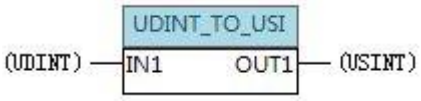
1.10.31 UDINT_TO_USINT：无符号双字型转无符号字节型指令

接口设计：输入变量 IN1，为 UDINT 类型；输出变量 OUT1，为 USINT 类型。

功能描述：将 UDINT 的输入变量转换为 USINT 的输出变量。

控件文本表示：OUT1 := UDINT _TO_ USINT (IN1);

控件图示：



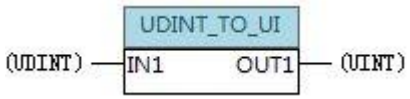
1.10.32 UDINT _TO_UINT：无符号双字型转无符号字型指令

接口设计：输入变量 IN1，为 UDINT 类型；输出变量 OUT1，为 UINT 类型。

功能描述：将 UDINT 的输入变量转换为 UINT 的输出变量。

控件文本表示：OUT1 := UDINT _TO_ UINT (IN1);

控件图示：



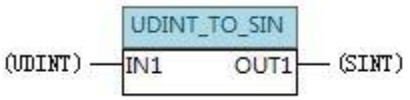
1.10.33 UDINT _TO_SINT：无符号双字型转有符号字节型指令

接口设计：输入变量 IN1，为 UDINT 类型；输出变量 OUT1，为 SINT 类型。

功能描述：将 UDINT 的输入变量转换为 SINT 的输出变量。

控件文本表示：OUT1 := UDINT _TO_ SINT (IN1);

控件图示：



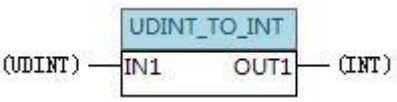
1.10.34 UDINT _TO_INT：无符号双字型转有符号字型指令

接口设计：输入变量 IN1，为 UDINT 类型；输出变量 OUT1，为 INT 类型。

功能描述：将 UDINT 的输入变量转换为 INT 的输出变量。

控件文本表示：OUT1 := UDINT _TO_ INT (IN1);

控件图示：



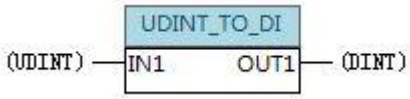
1.10.35 UDINT _TO_DINT：无符号双字型转有符号双字型指令

接口设计：输入变量 IN1，为 UDINT 类型；输出变量 OUT1，为 DINT 类型。

功能描述：将 UDINT 的输入变量转换为 DINT 的输出变量。

控件文本表示：OUT1 := UDINT _TO_ DINT (IN1);

控件图示：



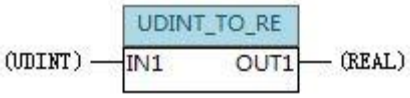
1.10.36 UDINT _TO_REAL：无符号双字型转浮点型指令

接口设计：输入变量 IN1，为 UDINT 类型；输出变量 OUT1，为 REAL 类型。

功能描述：将 UDINT 的输入变量转换为 REAL 的输出变量。

控件文本表示：OUT1 := UDINT _TO_ REAL (IN1);

控件图示：



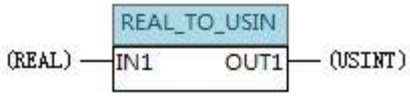
1.10.37 REAL_TO_USINT：浮点型转无符号字节型指令

接口设计：输入变量 IN1，为 REAL 类型；输出变量 OUT1，为 USINT 类型。

功能描述：将 REAL 的输入变量转换为 USINT 的输出变量。

控件文本表示：OUT1 := REAL _TO_ USINT (IN1);

控件图示：



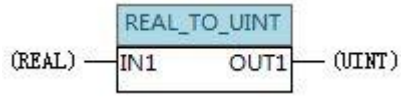
1.10.38 REAL _TO_UINT：浮点型转无符号字型指令

接口设计：输入变量 IN1，为 REAL 类型；输出变量 OUT1，为 UINT 类型。

功能描述：将 REAL 的输入变量转换为 UINT 的输出变量。

控件文本表示：OUT1 := REAL _TO_ UINT (IN1);

控件图示：



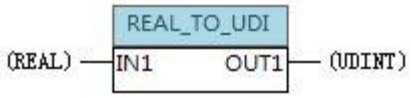
1.10.39 REAL _TO_UDINT：浮点型转无符号双字型指令

接口设计：输入变量 IN1，为 REAL 类型；输出变量 OUT1，为 UDINT 类型。

功能描述：将 REAL 的输入变量转换为 UDINT 的输出变量。

控件文本表示：OUT1 := REAL _TO_ UDINT (IN1);

控件图示：



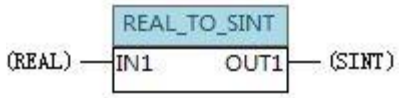
1.10.40 REAL _TO_SINT：浮点型转有符号字节型指令

接口设计：输入变量 IN1，为 REAL 类型；输出变量 OUT1，为 SINT 类型。

功能描述：将 REAL 的输入变量转换为 SINT 的输出变量。

控件文本表示：OUT1 := REAL _TO_ SINT (IN1);

控件图示：



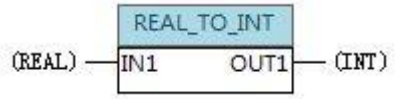
1.10.41 REAL _TO_INT：浮点型转有符号字型指令

接口设计：输入变量 IN1，为 REAL 类型；输出变量 OUT1，为 INT 类型。

功能描述：将 REAL 的输入变量转换为 INT 的输出变量。

控件文本表示：OUT1 := REAL _TO_ INT (IN1);

控件图示：



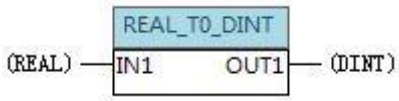
1.10.42 REAL _TO_DINT：浮点型转有符号双字型指令

接口设计：输入变量 IN1，为 REAL 类型；输出变量 OUT1，为 DINT 类型。

功能描述：将 REAL 的输入变量转换为 DINT 的输出变量。

控件文本表示：OUT1 := REAL _TO_ DINT (IN1);

控件图示：



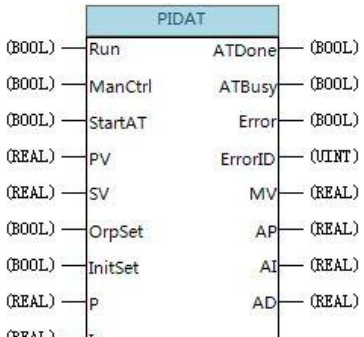
1.11 模拟量控制

1.11.1 PIDAT：无符号字节转 BCD 指令

接口设计：输入变量 Run，ManCtrl，StartAT，PV，SV，OrpSet，InitSet，P，I，D，ManMV；输出变量 ATDone，ATBusy，Error，ErrorID，MV，AP，AI，AD。Run，ManCtrl，StartAT，OrpSet，InitSet，ATDone，ATBusy，Error 为 BOOL 类型的变量；PV，SV，P，I，D，ManMV，MV，AP，AI，AD 为 REAL 类型的变量，ErrorID 为 UINT 类型的变量。

控件文本表示：(ATDone, ATBusy, Error, ErrorID, MV, AP, AI, AD) := PIDAT (Run, ManCtrl, StartAT, PV, SV, OrpSet, InitSet, P, I, D, ManMV);

控件图示：



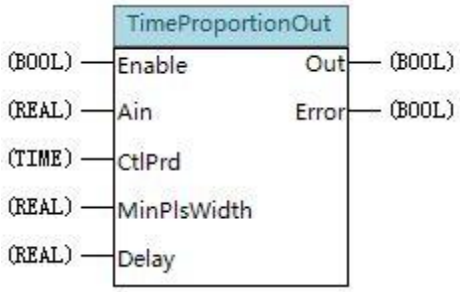
1.11.2 TimeproportionOut：时间比例输出指令

接口设计：输入变量 Enable，Ain，CtlPrd，MinPlsWidth，Delay；输出变量 OUT，Error。Enable，Out，Error 为 BOOL 类型的变量，Ain，MinPlsWidth，Delay 为 REAL 类型的变量，CtlPrd 为 TIME 类型的变量。

功能描述：时间比例输出，根据模拟量的值计算出位开关状态。Ain：模拟量输入（0-100%）；CtrPrd：控制周期（ms）；MinPlsWidth：最小脉冲宽度（0-50%）；Delay 延时启动；OUT：输出（0 或 1）。

控件文本表示：(OUT, Error) := TimeproportionOut (Enable, Ain, CtlPrd, MinPlsWidth, Delay);

控件图示：



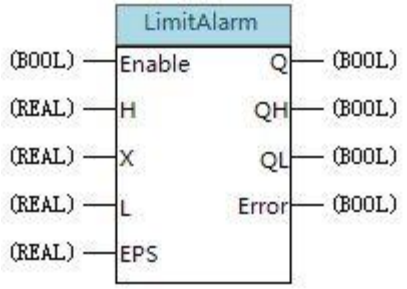
1.11.3 LimitAlarm：超限报警指令

接口设计：输入变量 Enable，H，X，L，EPS；输出变量 Q，QH，QL，Error。Enable，Q，QH，QL，Error 为 BOOL 类型的变量，H，X，L，EPS 为 REAL 类型的变量。

功能描述：超限报警功能块，Enable 为 0 时复位报警输出，为 1 时使能。当输入 X > H 或 X < L 时产生报警输出 Q，同时根据报警超 H 或超 L，置位 QH 或 QL。EPS 为回差。

控件文本表示：(Q, QH, QL, Error) := LimitAlarm (Enable, H, X, L, EPS);

控件图示：



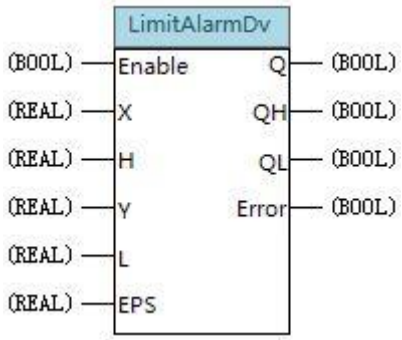
1.11.4 LimitAlarmDv：超偏差报警指令

接口设计：输入变量 Enable，H，X，Y，L，EPS；输出变量 Q，QH，QL，Error。Enable，Q，QH，QL，Error 为 BOOL 类型的变量，X，H，Y，L，EPS 为 REAL 类型的变量。

功能描述：超偏差报警功能块，Enable 为 0 时复位报警输出，为 1 时使能。当输入 X – Y > H 或 Y - X > L 时产生报警输出 Q，同时根据报警超 H 或超 L，置位 QH 或 QL。EPS 为回差，用于报警消除。

控件文本表示：(Q, QH, QL, Error) := LimitAlarmDv (Enable, X, H, Y, L, EPS);

控件图示：



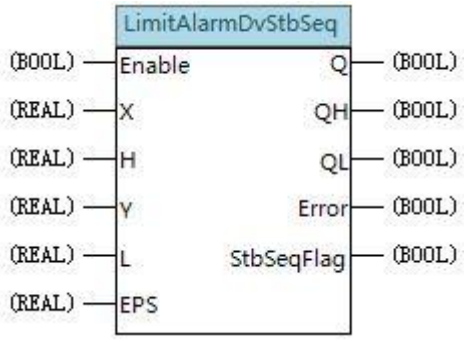
1.11.5 LimitAlarmDvStbSeq：稳态超偏差报警指令

接口设计：输入变量 Enable，H，X，Y，L，EPS；输出变量 Q，QH，QL，Error，StbSeqFlag。Enable，Q，QH，QL，Error，StbSeqFlag 为 BOOL 类型的变量，X，H，Y，L，EPS 为 REAL 类型的变量。

功能描述：稳态超偏差报警功能块，Enable 为 0 时复位报警输出，为 1 时使能。如果 X > Y，当 X – Y < H 时，清除 StbSeqFlag，进入报警状态。如果 X < Y，当 Y – X < L 时，清除 StbSeqFlag，进入报警状态。当输入 X – Y > H 或 Y - X > L 时产生报警输出 Q，同时根据报警超 H 或超 L，置位 QH 或 QL，但是在 StbSeqFlag 标志消除前，不报警。EPS 为回差，用于报警消除。

控件文本表示：(Q, QH, QL, Error) := LimitAlarmDvStbSeq (Enable, X, H, Y, L, EPS);

控件图示：



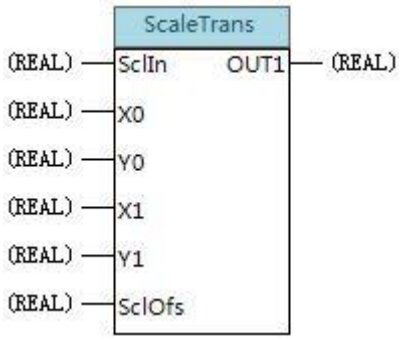
1.11.6 ScaleTrans：范围转换指令

接口设计：输入变量 SclIn，X0，Y0，X1，Y1，SclOfs；输出变量 OUT1，均为 REAL 类型的变量。

功能描述：ScaleTrans 指令将输入值从输入范围缩放到输出范围。输入范围由输入范围下限 X0 和输入范围上限 X1 指定。输出范围由输出范围下限 Y0 和输出范围上限 Y1 指定。偏移量 SclOfs 的值会添加到已缩放到输出范围的值中，最后作为结果输出。

控件文本表示：OUT1 := ScaleTrans (SclIn, X0, Y0, X1, Y1, SclOfs);

控件图示：



1.12 BCD 码转换

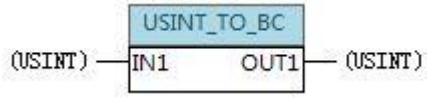
1.12.1 USINT_TO_BCD：无符号字节转 BCD 指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 USINT 类型。

功能描述：将输入的十进制数用 BCD 码表示（有效输入范围：0-99，越界输出 0）。

控件文本表示：OUT1 := USINT_TO_BCD (IN1);

控件图示：



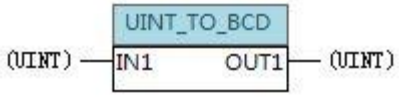
1.12.2 UINT_TO_BCD：无符号字转 BCD 指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UINT 类型。

功能描述：将输入的十进制数用 BCD 码表示（有效输入范围：0-9999，越界输出 0）。

控件文本表示：OUT1 := UINT _TO_BCD (IN1);

控件图示：



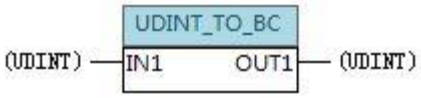
1.12.3 UDINT_TO_BCD：无符号双字转 BCD 指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UDINT 类型。

功能描述：将输入的十进制数用 BCD 码表示（有效输入范围：0-99999999，越界输出 0）。

控件文本表示：OUT1 := UDINT _TO_BCD (IN1);

控件图示：



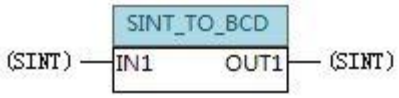
1.12.4 SINT_TO_BCD：有符号字节转 BCD 指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型。

功能描述：将输入的十进制数用 BCD 码表示（有效输入范围：-79-79，越界输出 0）。

控件文本表示：OUT1 := SINT _TO_BCD (IN1);

控件图示：



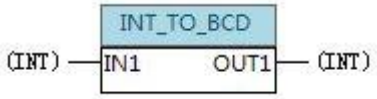
1.12.5 INT_TO_BCD：有符号字转 BCD 指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型。

功能描述：将输入的十进制数用 BCD 码表示（有效输入范围：-7999-7999，越界输出 0）。

控件文本表示：OUT1 := INT _TO_BCD (IN1);

控件图示：



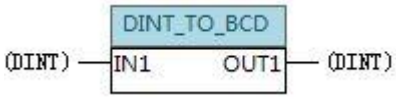
1.12.6 DINT_TO_BCD：有符号双字转 BCD 指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型。

功能描述：将输入的十进制数用 BCD 码表示（有效输入范围：-79999999-79999999，越界输出 0）。

控件文本表示：OUT1 := DINT _TO_BCD (IN1);

控件图示：



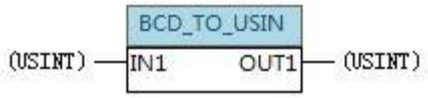
1.12.7 BCD_TO_USINT：BCD 转无符号字节指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 USINT 类型。

功能描述：将 BCD 码表示的数据转换成对应的十进制数据（输入必须为 BCD 格式数据，否则输出 0）。

控件文本表示：OUT1 := BCD_TO_USINT (IN1);

控件图示：



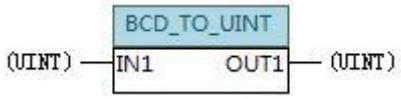
1.12.8 BCD_TO_UINT：BCD 转无符号字指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UINT 类型。

功能描述：将 BCD 码表示的数据转换成对应的十进制数据（输入必须为 BCD 格式数据，否则输出 0）。

控件文本表示：OUT1 := BCD_TO_ UINT (IN1);

控件图示：



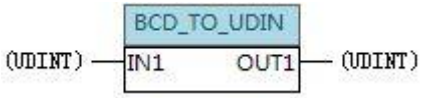
1.12.9 BCD_TO_UDINT：BCD 转无符号双字指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UDINT 类型。

功能描述：将 BCD 码表示的数据转换成对应的十进制数据（输入必须为 BCD 格式数据，否则输出 0）。

控件文本表示：OUT1 := BCD_TO_ UDINT (IN1);

控件图示：



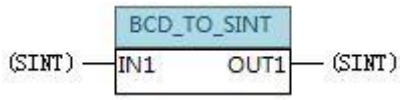
1.12.10 BCD_TO_SINT：BCD 转有符号字节指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 SINT 类型。

功能描述：将 BCD 码表示的数据转换成对应的十进制数据（输入必须为 BCD 格式数据，否则输出 0）。

控件文本表示：OUT1 := BCD_TO_ SINT (IN1);

控件图示：



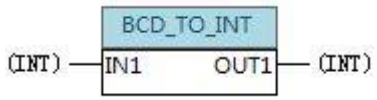
1.12.11 BCD_TO_INT：BCD 转有符号字指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 INT 类型。

功能描述：将 BCD 码表示的数据转换成对应的十进制数据（输入必须为 BCD 格式数据，否则输出 0）。

控件文本表示：OUT1 := BCD_TO_ INT (IN1);

控件图示：



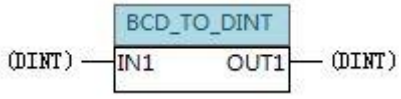
1.12.12 BCD_TO_DINT：BCD 转有符号双字指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 DINT 类型。

功能描述：将 BCD 码表示的数据转换成对应的十进制数据（输入必须为 BCD 格式数据，否则输出 0）。

控件文本表示：OUT1 := BCD_TO_ DINT (IN1);

控件图示：



1.13 时间和日期

对于日期时间变量（DT），定义为 8 个 UINT 类型的数组，顺序分别为秒、分、时、日、月、年、星期几、一年中的第几天。作为输入变量传入功能块时，传递数组的首地址。

日期变量（DATE），表示日期年月日，定义为 8 个 UINT 类型的数组，顺序同 DT 变量，只有年月日部分有意义，其他部分为 0。

一天中的时间变量（TOD），表示一天中的某个时刻，定义为 8 个 UINT 类型的数组，顺序同 DT 变量，只有时分秒部分有意义，其他部分为 0。

TIME 类型变量表示一段时间，单位是毫秒。毫秒单位向秒单位的转换，采用四舍五入的方式。

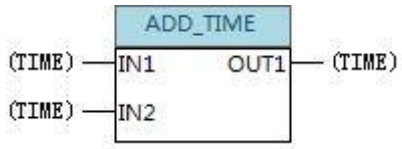
1.13.1 ADD_TIME：TIME 变量的加法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 TIME 类型。

功能描述：OUT1 = IN1 + IN2。

控件文本表示：OUT1 := ADD_TIME (IN1, IN2);

控件图示：



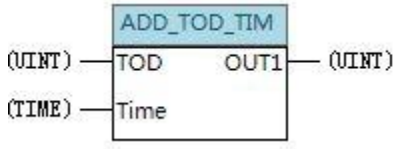
1.13.2 ADD_TOD_TIME：TOD 变量和 TIME 变量的加法指令

接口设计：输入变量 TOD，Time，输出变量 OUT1。TOD，OUT1 为 UINT 类型的变量，Time 为 TIME 类型的变量。

功能描述： TOD 变量加上 TIME 变量，OUT1 = TOD + Time。

控件文本表示：OUT1 := ADD_TOD_TIME (TOD, Time);

控件图示：



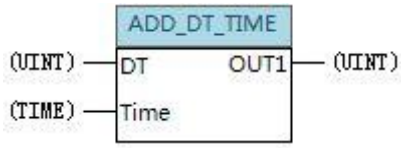
1.13.3 ADD_DT_TIME：DT 变量和 TIME 变量的加法指令

接口设计：输入变量 DT，Time，输出变量 OUT1。TOD，OUT1 为 UINT 类型的变量，Time 为 TIME 类型的变量。

功能描述： DT 变量加上 TIME 变量，OUT1 = DT + Time。

控件文本表示：OUT1 := ADD_DT_TIME (DT, Time);

控件图示：



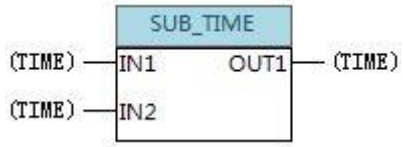
1.13.4 SUB_TIME：TIME 减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 TIME 类型。

功能描述：OUT1 = IN1 – IN2。

控件文本表示：OUT1 := SUB _TIME (IN1, IN2);

控件图示：



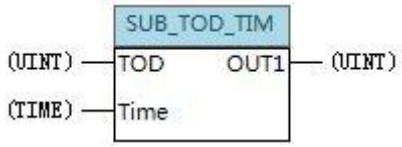
1.13.5 SUB_TOD_TIME：TOD 变量和 TIME 变量的减法指令

接口设计：输入变量 TOD，Time，输出变量 OUT1。TOD，OUT1 为 UINT 类型的变量，Time 为 TIME 类型的变量。

功能描述：TOD 变量减去 TIME 变量，OUT1 = TOD – Time。

控件文本表示：OUT1 := SUB _ TOD _TIME (IN1, IN2);

控件图示：



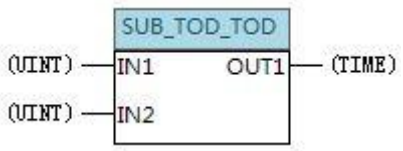
1.13.6 SUB_TOD_ TOD：TOD 变量的减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，IN2 为 UINT 类型的变量，OUT1 为 TIME 类型的变量。

功能描述：计算一天中的两个时间差， 输出两个时间相差的毫秒数，OUT1 = IN1 – IN2。

控件文本表示：OUT1 := SUB _ TOD _ TOD (IN1, IN2);

控件图示：



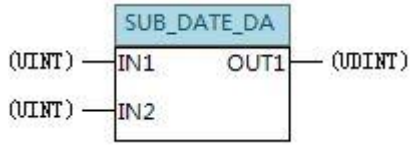
1.13.7 SUB_DATE_DATE：DATE 变量的减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：输出两个日期之间相差的天数，OUT1 = ABS(IN1 – IN2)。

控件文本表示：OUT1 := SUB_DATE_DATE (IN1, IN2);

控件图示：



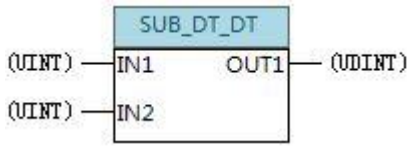
1.13.8 SUB_DT_DT：DT 变量的减法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：输出两个日期时间之间相差的秒数，OUT1 =ABS(IN1 – IN2)。

控件文本表示：OUT1 := SUB_DT_DT (IN1, IN2);

控件图示：



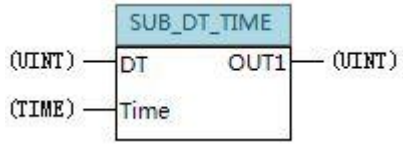
1.13.9 SUB _DT_TIME：DT 变量和 TIME 变量的减法指令

接口设计：输入变量 DT，Time，输出变量 OUT1。TOD，OUT1 为 UINT 类型的变量，Time 为 TIME 类型的变量。

功能描述： DT 变量减去 TIME 变量，OUT1 = DT - Time。

控件文本表示：OUT1 := SUB_DT_TIME (DT, Time);

控件图示：



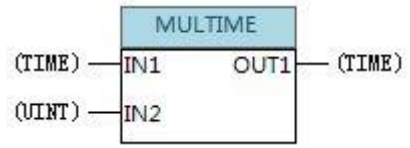
1.13.10 MULTIME：TIME 变量和数字的乘法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 Time 类型的变量，IN2 为 UINT 类型的变量。

功能描述：OUT1 = IN1 * IN2。

控件文本表示：OUT1 := MULTIME (IN1, IN2);

控件图示：



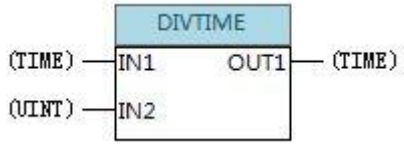
1.13.11 DIVTIME：TIME 变量和数字的除法指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1。IN1，OUT1 为 Time 类型的变量，IN2 为 UINT 类型的变量。

功能描述：OUT1 = IN1 / IN2。

控件文本表示：OUT1 := DIVTIME (IN1, IN2);

控件图示：



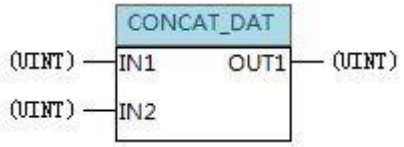
1.13.12 CONCAT_DATE_TOD：DATE 变量和 TOD 变量的连接指令

接口设计：输入变量 IN1，IN2，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：连接 DATE 的年月日和 TOD 的时分秒。

控件文本表示：OUT1 := CONCAT_DATE_TOD (IN1, IN2);

控件图示：



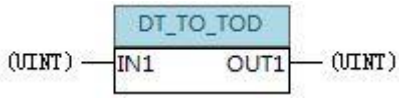
1.13.13 DT_TO_TOD：DT 变量转 TOD 变量指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：从 DT（年月日时分秒）的中提取 TOD（时分秒）。

控件文本表示：OUT1 := DT_TO_TOD (IN1);

控件图示：



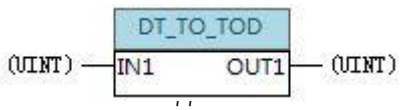
1.13.14 DT_TO_DATE：DT 变量转 DATE 变量指令

接口设计：输入变量 IN1，输出变量 OUT1，均为 UINT 类型的变量。

功能描述：从 DT（年月日时分秒）的中提取 DATE（年月日）。

控件文本表示：OUT1 := DT_TO_ DATE (IN1);

控件图示：



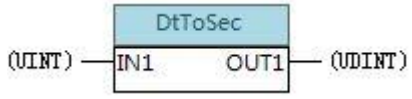
1.13.15 DtToSec：DT 变量转秒数指令

接口设计：输入变量 IN1，为 UINT 类型的变量；输出变量 OUT1，为 UDINT 类型的变量。

功能描述：将日期时间 DT（年月日时分秒）转化为从 1970 年 1 月 1 日 00:00:00 开始的秒数。

控件文本表示：OUT1 := DtToSec (IN1);

控件图示：



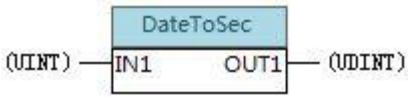
1.13.16 DateToSec：Date 变量转秒数指令

接口设计：输入变量 IN1，为 UINT 类型的变量；输出变量 OUT1，为 UDINT 类型的变量。

功能描述：将日期 DATE（年月日）转化为从 1970 年 1 月 1 日 00:00:00 开始的秒数。

控件文本表示：OUT1 := DateToSec (IN1);

控件图示：



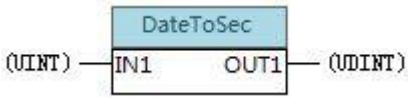
1.13.17 TodToSec：Tod 变量转秒数指令

接口设计：输入变量 IN1，为 UINT 类型的变量；输出变量 OUT1，为 UDINT 类型的变量。

功能描述：将时间 TOD（时间秒）转化为从 00:00:00 开始的秒数。

控件文本表示：OUT1 := TodToSec (IN1);

控件图示：



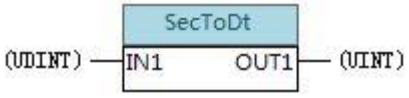
1.13.18 SecToDt：秒数转 DT 变量指令

接口设计：输入变量 IN1，为 UDINT 类型的变量；输出变量 OUT1，为 UINT 类型的变量。

功能描述：将秒数转化为从 1970 年 1 月 1 日 00:00:00 开始的日期时间。

控件文本表示：OUT1 := SecToDt (IN1);

控件图示：



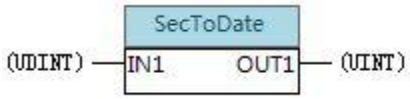
1.13.19 SecToDate：秒数转 DATE 变量指令

接口设计：输入变量 IN1，为 UDINT 类型的变量；输出变量 OUT1，为 UINT 类型的变量。

功能描述：将秒数转化为从 1970 年 1 月 1 日 00:00:00 开始的日期。

控件文本表示：OUT1 := SecToDate (IN1);

控件图示：



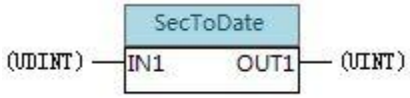
1.13.20 SecToTod：秒数转 TOD 变量指令

接口设计：输入变量 IN1，为 UDINT 类型的变量；输出变量 OUT1，为 UINT 类型的变量。

功能描述：将秒数转化为从 00:00:00 开始的某个时间。

控件文本表示：OUT1 := SecToTod (IN1);

控件图示：



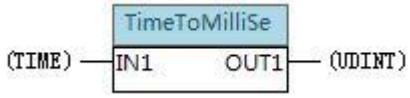
1.13.21 TimeToMilliSec：TIME 变量转毫秒数指令

接口设计：输入变量 IN1，为 TIME 类型的变量；输出变量 OUT1，为 UDINT 类型的变量。

功能描述：将 TIME 变量转化为毫秒数。

控件文本表示：OUT1 := TimeToMilliSec (IN1);

控件图示：



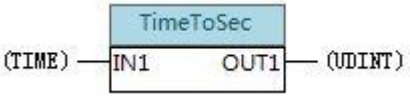
1.13.22 TimeToSec：TIME 变量转秒数指令

接口设计：输入变量 IN1，为 TIME 类型的变量；输出变量 OUT1，为 UDINT 类型的变量。

功能描述：将 TIME 变量转化为秒数。

控件文本表示：OUT1 := TimeToSec (IN1);

控件图示：



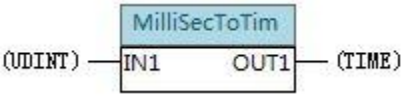
1.13.23 MilliSecToTime：毫秒数转 TIME 变量指令

接口设计：输入变量 IN1，为 UDINT 类型的变量；输出变量 OUT1，为 TIME 类型的变量。

功能描述：将毫秒数转化为 TIME 变量。

控件文本表示：OUT1 := MilliSecToTime (IN1);

控件图示：



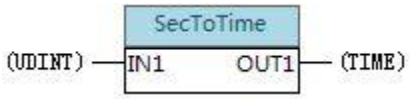
1.13.24 SecToTime：秒数转 TIME 变量指令

接口设计：输入变量 IN1，为 UDINT 类型的变量；输出变量 OUT1，为 TIME 类型的变量。

功能描述：将秒数转化为 TIME 变量。

控件文本表示：OUT1 := SecToTime (IN1);

控件图示：



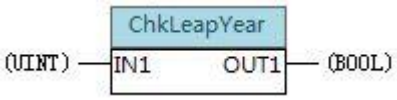
1.13.25 ChkLeapYear：判断闰年指令

接口设计：输入变量 IN1，为 UINT 类型的变量；输出变量 OUT1，为 BOOL 类型的变量。

功能描述：判断输入日期是否为闰年。

控件文本表示：OUT1 := ChkLeapYear (IN1);

控件图示：



1.13.26 GetDaysOfMonth：获取天数指令

接口设计：输入变量 Year，Month，输出变量 OUT1。Year 为 UINT 类型的变量；Month，OUT1 为 USINT 类型的变量。

功能描述：根据输入的年份和月份，输出当月的总天数。

控件文本表示：OUT1 := GetDaysOfMonth (IN1, IN2);

控件图示：



1.13.27 DaysToMonth：计算月份指令

接口设计：输入变量 Year，Month，为 UINT 类型的变量；输出变量 OUT1，为 USINT 类型的变量。

功能描述：根据输入的年份和从 1 月 1 日开始的天数，计算月份。控件文本表示：OUT1 := DaysToMonth (IN1, IN2);

控件图示：



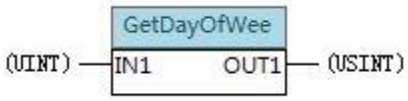
1.13.28 GetDayOfWeek：获取星期几指令

接口设计：输入变量 IN1，为 UINT 类型的变量；输出变量 OUT1，为 USINT 类型的变量。

功能描述：根据输入的日期（年月日），当天是星期几，输出 0-6，0 表示星期日。

控件文本表示：OUT1 := GetDayOfWeek (IN1, IN2);

控件图示：



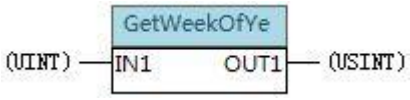
1.13.29 GetWeekOfYear：获取第几周指令

接口设计：输入变量 IN1，为 UINT 类型的变量；输出变量 OUT1，为 USINT 类型的变量。

功能描述：根据输入的日期（年月日），输出该日期是概念的第几周，1 月 1 日总在第一周。

控件文本表示：OUT1 := GetWeekOfYear (IN1, IN2);

控件图示：



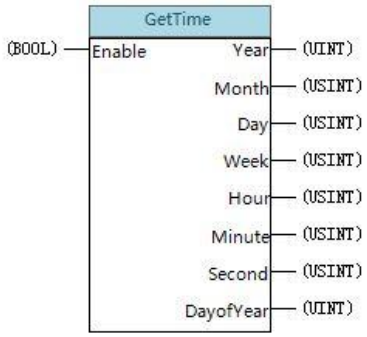
1.13.30 GetTime：获取系统时间指令

接口设计：输入变量 Enable，为 BOOL 类型的变量；输出变量 Year，Month，Day，Week，Hour，Minute，Second，DayofYear，其中 Year，DaysofYear 为 UINT 类型的变量，其他位 USINT 类型的变量。

功能描述：获取系统时间，分别为年、月、日、周几（0-6），时、分、秒、日期在年中的序号（0-365）。

控件文本表示：OUT1 := GetTime (IN1, IN2);

控件图示：



1.13.31 SetTime：设置系统时间指令

接口设计：输入变量 Enable，为 BOOL 类型的变量；输出变量 Year，Month，Day，Week，Hour，Minute，Second，DayofYear，其中 Year，DaysofYear 为 UINT 类型的变量，其他位

USINT 类型的变量。

功能描述：设置系统时间，分别为年、月、日、周几（0-6），时、分、秒、日期在年中的序号（0-365）。

控件文本表示：OUT1 := SetTime (IN1, IN2);

控件图示：



2 运动控制指令

本部分所有运动控制指令均参考 PLCopen 运动控制规范的 Part1、Part2 部分的规定设计，功能块设计符合规范要求，分为单轴运动控制和多轴运动控制两个部分。

单轴运动控制的功能块包括:

1.1 MC_Power（控制轴使能）

1.2 MC_Stop（停止轴）

1.3 MC_Halt（暂停轴）

1.4 MC_MoveAbsolute（控制轴进行绝对运动）

1.5 MC_MoveRelative（控制轴进行相对运动）

1.6 MC_MoveAdditive（控制轴进行附加运动）

1.7 MC_MoveSuperimposed（控制轴进行叠加运动）

1.8 MC_HaltSuperimposed（停止轴的叠加运动）

1.9 MC_MoveVelocity（控制轴达到特定速度）

1.10 MC_MoveContinuousAbsolute（控制轴以特定末速度运动到绝对位置）

1.11 MC_MoveContinuousRelative（控制轴以特定末速度运动到运动位置）

1.12 MC_TorqueControl（控制轴力矩）

1.13 MC_PositionProfile（使轴按位置表运动）

1.14 MC_VelocityProfile（使轴按速度表运动）

1.15 MC_AccelerationProfile（使轴按加速度表运动）

1.16 MC_SetPosition（将轴当前位置设为特定值）

1.17 MC_ReadParameter（读取参数）

1.18 MC_ReadBoolParameter（读取布尔参数）

1.19 MC_WriteParameter（写参数）

1.20 MC_WriteBoolParameter（写布尔参数）

1.21 MC_ReadDigitalInput（读取数字输入）

1.22 MC_ReadDigitalOutput（读取数字输出）

1.23 MC_WriteDigitalOutput（写数字输出）

1.24 MC_ReadActualPosition（读取轴当前位置）

1.25 MC_ReadActualVelocity（读取轴当前速度）

1.26 MC_ReadActualTorque（读取轴当前力矩）

1.27 MC_ReadStatus（读取轴当前状态）

1.28 MC_ReadAxisInfo（读取轴信息）

1.29 MC_ReadAxisError（读取轴错误）

1.30 MC_Reset（复位轴）

1.31 MC_TouchProbe（监控信号源，信号源上跳沿时记录轴位置）

1.32 MC_AbortTrigger（中断信号源监控）

多轴运动控制的功能块包括:

2.1 MC_CamTableSelect（创建并选择凸轮表）

2.2 MC_CamIn（使从轴与主轴以凸轮形式耦合）

2.3 MC_Camout（将从轴凸轮中解耦）

2.4 MC_GearIn（使从轴与主轴以齿轮形式耦合）

2.5 MC_GearOut（将从轴齿轮中解耦）

2.6 MC_PhasingAbsolute（在从轴所参考的主轴位置中创造一个绝对相移）

2.7 MC_PhasingRelative（在从轴所参考的主轴位置中创造一个相对相移）

2.1 单轴模块

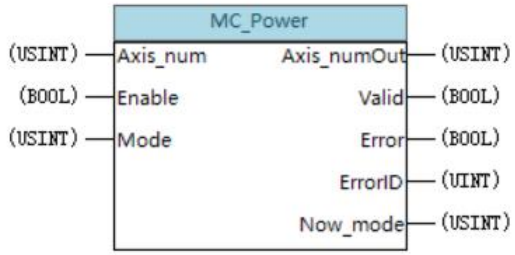
该部分功能块是控制单个轴的状态及运动。

2.1.1 MC_Power

这个模块功能是控制单轴使能状态（使能或失能）。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
B	Mode	USINT	驱动器控制模式
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	该轴当前使能情况
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	Now_mode	USINT	驱动器当前控制模式

功能块图形形式



功能块文本形式

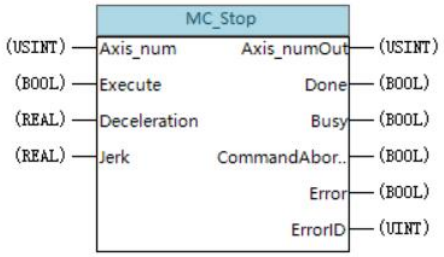
```
MC_Power(Axis_num := ,Enable := ,Mode := ,Axis_numOut => ,Valid => ,
          Error => ,ErrorID => ,Now_mode => );
```

2.1.2 MC_Stop

这个模块功能是控制单轴停止，可以打断任何其他运动模块。轴速度减为 0 后，DONE 置位。除非失能，否则只要 Execute 为 TRUE，轴状态始终为 Stopping，其余运动模块均不可操作，只有在轴速度为 0 且 Execute 置为 FALSE 时，轴状态才置为 Standstill。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_stop(Axis_num := ,Execute := ,Deceleration := ,Jerk := ,
        Axis_numOut => ,Done => ,Busy => ,CommandAborted => ,
        Error => ,ErrorID => );
```

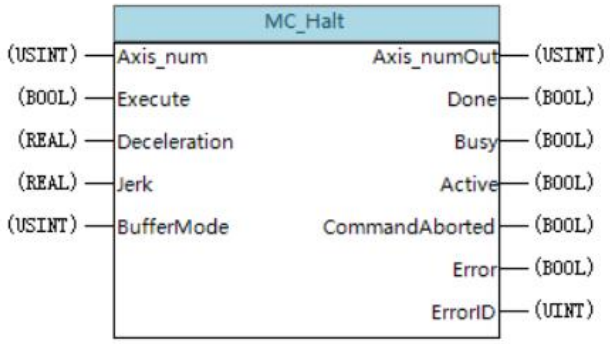
2.1.3 MC_Halt

这个模块功能将轴速度减速到 0，在运行过程中状态为 DiscrectMotion，直到速度为 0，

DONE 置位，状态变为 Standstill。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_halt(Axis_num := ,Execute := ,Deceleration := ,Jerk := ,
        BufferMode := ,Axis_numOut => ,Done => ,Busy => ,
Active => ,CommandAborted => ,Error => ,ErrorID => );
```

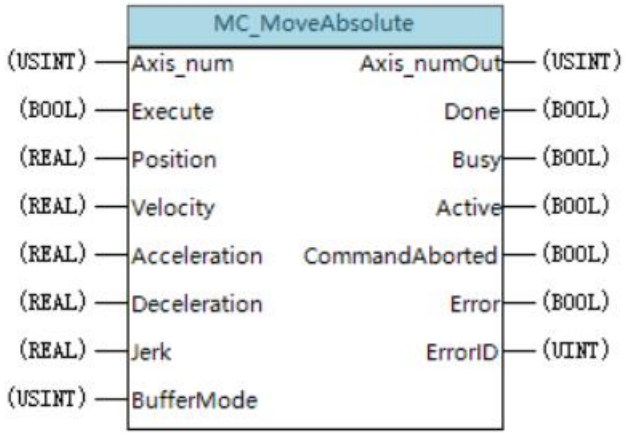
2.1.4 MC_MoveAbsolute

这个模块功能是将轴运行到指定的绝对位置。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能

B	Position	REAL	目标绝对位置
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveAbsolute(Axis_num := ,Execute := ,Position := ,Velocity := ,
                Acceleration := ,Deceleration := ,Jerk := ,
                BufferMode := ,Axis_numOut => ,Done => ,Busy => ,
                Active => ,CommandAborted => ,Error => ,ErrorID => );
```

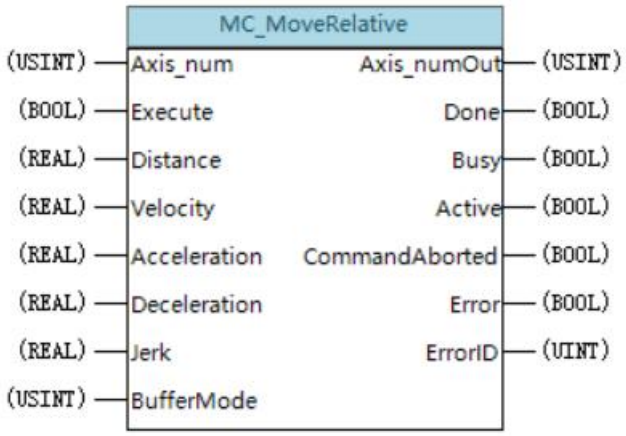
2.1.5 MC_MoveRelative

这个模块功能将轴运行到相对于当前位置的相对位置。

VAR_INPUT

B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Distance	REAL	目标相对位置
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

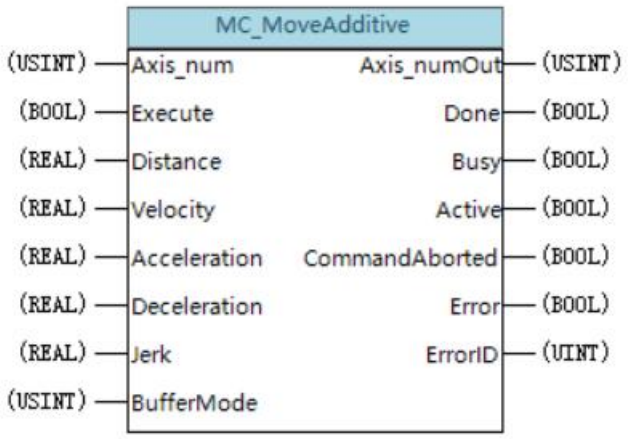
```
MC_MoveRelative(Axis_num := ,Execute := ,Distance := ,Velocity := ,
                Acceleration := ,Deceleration := ,Jerk := ,
                BufferMode := ,Axis_numOut => ,Done => ,Busy => ,
                Active => ,CommandAborted => ,Error => ,ErrorID => );
```

2.1.6 MC_MoveAdditive

这个模块功能将轴在当前的行程上增加一段距离，若是打断模式，则该模块会把被打断模块剩余的距离加上该模块的距离作为总距离。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Distance	REAL	额外增加的距离
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

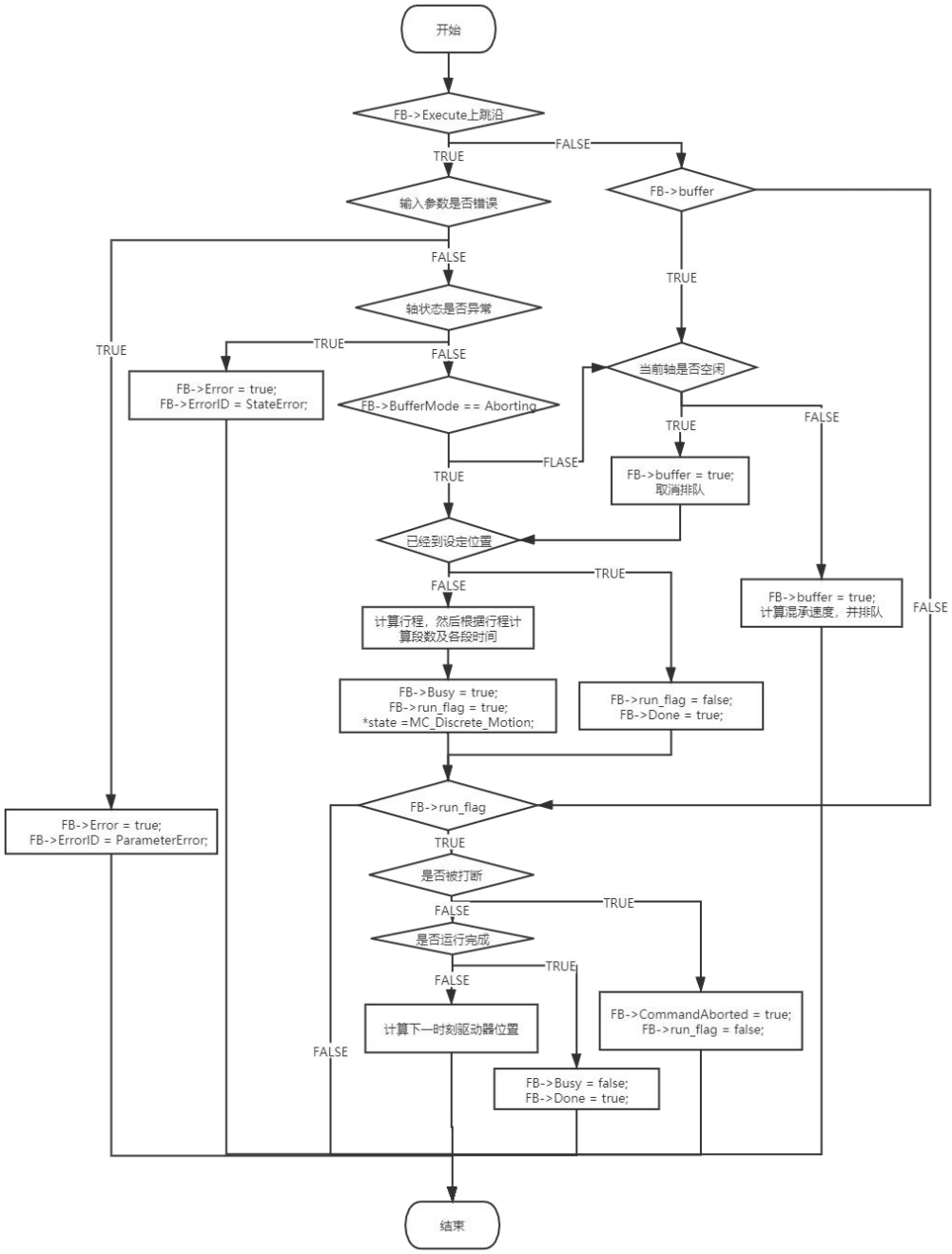
功能块图形形式



功能块文本形式

```
MC_MoveAdditive(Axis_num := ,Execute := ,Distance := ,Velocity := ,
                Acceleration := ,Deceleration := ,Jerk := ,
                BufferMode := ,Axis_numOut => ,Done => ,Busy => ,
                Active => ,CommandAborted => ,Error => ,ErrorID => );
```

功能块处理流程



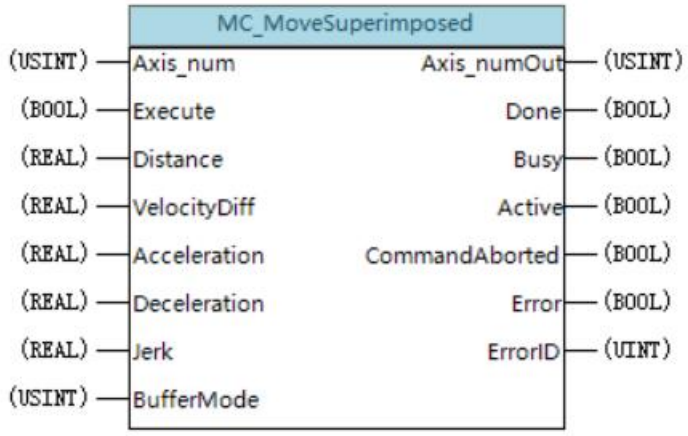
2.1.7 MC_MoveSuperimposed

这个模块功能是在轴原来的运动上叠加运动，不影响原来规划好的运动。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Distance	REAL	额外增加的距离
B	VelocityDiff	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度

E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveSuperimposed(Axis_num := ,Execute := ,Distance := ,
          VelocityDiff := ,Acceleration := ,Deceleration := ,Jerk
          := ,BufferMode := ,Axis_numOut => ,Done => ,Busy => ,
          Active => ,CommandAborted => ,Error => ,ErrorID => );
```

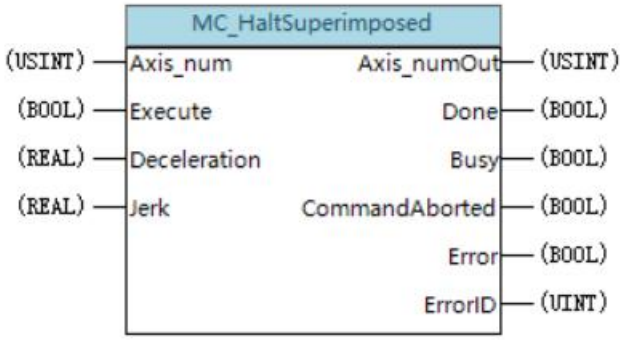
2.1.8 MC_HaltSuperimposed

这个模块功能将叠加部分的运动速度减到 0，该模块完成后轴整体的速度不一定为 0。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号

B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_HaltSuperimposed(Axis_num := ,Execute := ,Deceleration := ,
                    Jerk := ,Axis_numOut => ,Done => ,Busy => ,
                    CommandAborted => ,Error => ,ErrorID => );
```

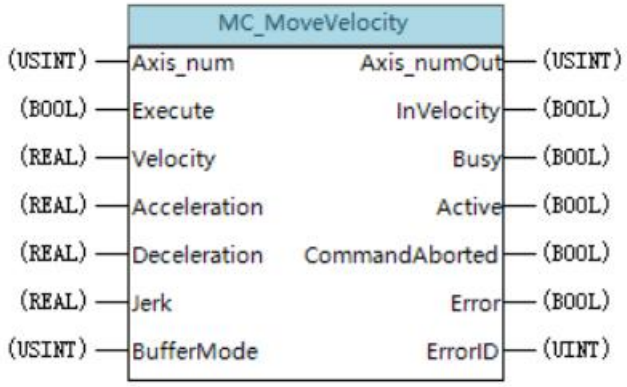
2.1.9 MC_MoveVelocity

这个模块功能是将轴运行到指定速度。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Velocity	REAL	目标速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	InVelocity	BOOL	轴是否到达目标速度
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断

B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveVelocity(Axis_num := ,Execute := ,Velocity := ,
                Acceleration := ,Deceleration := ,Jerk := ,
                BufferMode := ,Axis_numOut => ,InVelocity => ,
                Busy => ,Active => ,CommandAborted => ,
                Error => ,ErrorID => );
```

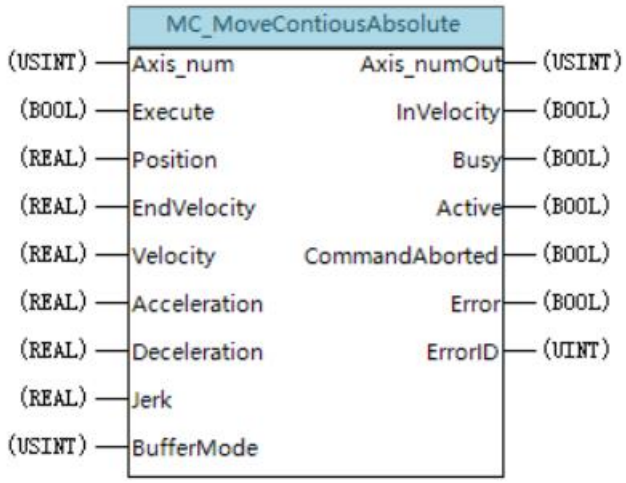
2.1.10 MC_MoveContinuousAbsolute

这个模块功能是将轴以指定末速度运行到指定绝对位置。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Position	REAL	目标绝对位置
B	EndVelocity	REAL	末速度
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	InVelocity	BOOL	轴是否到达目标速度
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴

E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveContiousAbsolute(Axis_num := ,Execute := ,Position := ,
          EndVelocity := ,Velocity := ,Acceleration := ,
          Deceleration := ,Jerk := ,BufferMode := ,
          Axis_numOut => ,InVelocity => ,Busy => ,
          Active => ,CommandAborted => ,Error => ,ErrorID => );
```

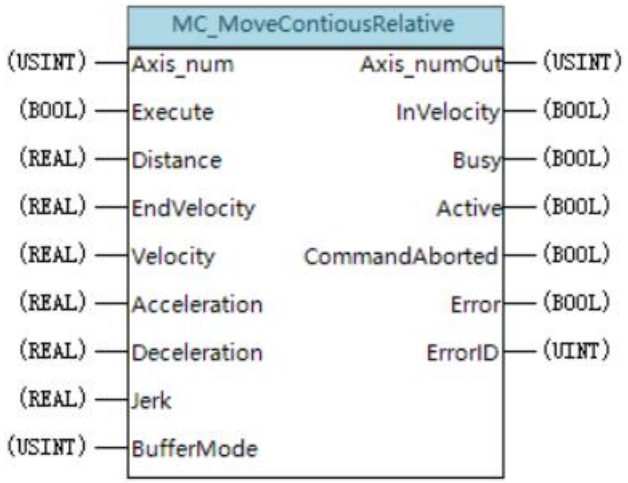
2.1.11 MC_MoveContinuousRelative

这个模块功能是将轴以指定末速度运行到指定相对位置。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Distance	REAL	目标绝对位置
B	EndVelocity	REAL	末速度
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号

B	InVelocity	BOOL	轴是否到达目标速度
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveContiousRelative(Axis_num := ,Execute := ,Distance := ,
          EndVelocity := ,Velocity := ,Acceleration := ,
          Deceleration := ,Jerk := ,BufferMode := ,
          Axis_numOut => ,InVelocity => ,Busy => ,
          Active => ,CommandAborted => ,Error => ,ErrorID => );
```

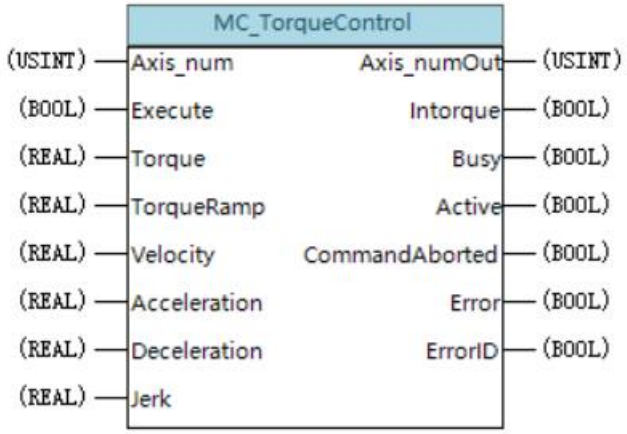
2.1.12 MC_TorqueControl

这个模块功能是以指定斜率控制力矩到指定值。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
B	Torque	REAL	目标力矩
B	TorqueRamp	REAL	力矩斜率
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Intorque	BOOL	是否达到目标力矩
E	Busy	BOOL	模块是否未完成

E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_TorqueControl(Axis_num := ,Execute := ,Torque := ,TorqueRamp := ,
                 Velocity := ,Acceleration := ,Deceleration := ,Jerk := ,
                 Axis_numOut => ,Intorque => ,Busy => ,Active => ,
                 CommandAborted => ,Error => ,ErrorID => );
```

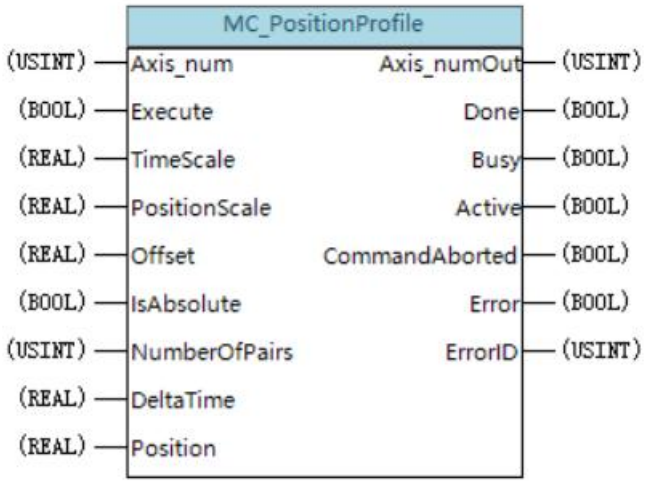
2.1.13 MC_PositionProfile

这个模块功能是使轴按照一定轨迹运动，运动轨迹由输入的间隔时间数组和位置数组来决定。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	TimeScale	REAL	时间缩放比例
E	PositionScale	REAL	位置缩放比例
E	Offset	REAL	位置偏移
B	IsAbsolute	REAL	TRUE 表示绝对位置，FALSE 表示相对位置
B	NumberOfPairs	REAL	数组长度
B	DeltaTime	REAL*	间隔时间数组指针
B	Position	REAL*	位置数组指针
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断

B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

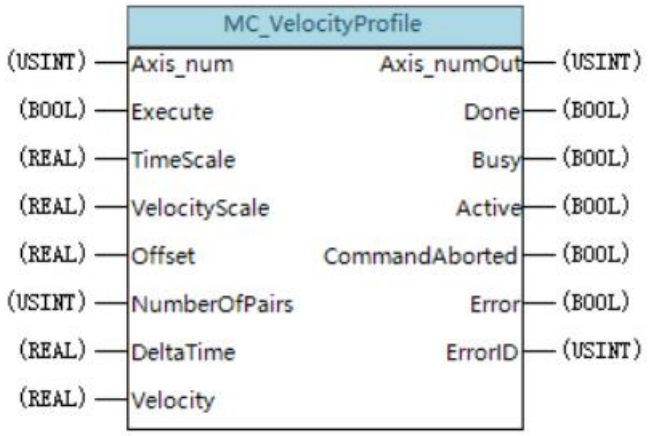
```
MC_PositionProfile(Axis_num := ,Execute := ,TimeScale := ,
      PositionScale := ,Offset := ,IsAbsolute := ,NumberOfPairs := ,
      DeltaTime := ,Position := ,Axis_numOut => ,Done => ,Busy => ,
      Active => ,CommandAborted => ,Error => ,ErrorID => );
```

2.1.14 MC_VelocityProfile

这个模块功能是使轴按照一定轨迹运动，运动轨迹由输入的间隔时间数组和速度数组来决定。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	TimeScale	REAL	时间缩放比例
E	VelocityScale	REAL	速度缩放比例
E	Offset	REAL	速度偏移
B	NumberOfPairs	REAL	数组长度
B	DeltaTime	REAL*	间隔时间数组指针
B	Velocity	REAL*	速度数组指针
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

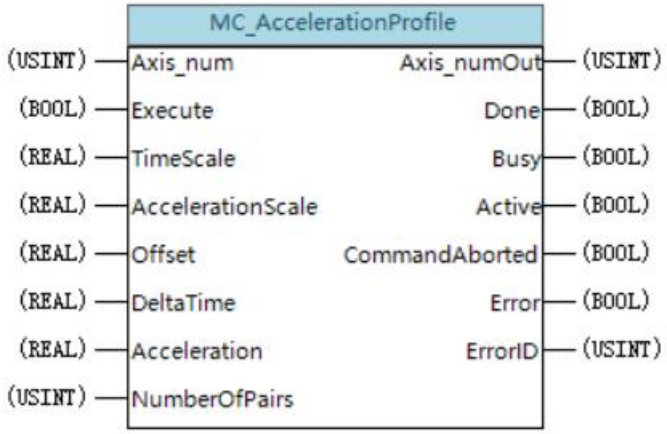
```
MC_VelocityProfile(Axis_num := ,Execute := ,TimeScale := ,
       VelocityScale := ,Offset := ,NumberOfPairs := ,DeltaTime := ,
       Velocity := ,Axis_numOut => ,Done => ,Busy => ,
       Active => ,CommandAborted => ,Error => ,ErrorID => );
```

2.1.15 MC_AccelerationProfile

这个模块功能是使轴按照一定轨迹运动，运动轨迹由输入的间隔时间数组和加速度数组来决定。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	TimeScale	REAL	时间缩放比例
E	AccelerationScale	REAL	加速度缩放比例
E	Offset	REAL	速度偏移
B	NumberOfPairs	REAL	数组长度
B	DeltaTime	REAL*	间隔时间数组指针
B	Acceleration	REAL*	加速度数组指针
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

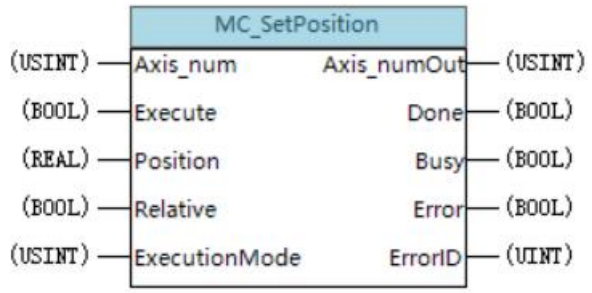
```
MC_AccelerationProfile(Axis_num := ,Execute := ,TimeScale := ,
      AccelerationScale := ,Offset := ,NumberOfPairs := ,
      DeltaTime := ,Acceleration := ,Axis_numOut => ,Done => ,
      Busy => ,Active => ,CommandAborted => ,Error => ,ErrorID => );
```

2.1.16 MC_SetPosition

这个模块功能是将当前轴的位置设定为指定值，即将零点进行偏移。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	Position	REAL	位置值
E	Relative	BOOL	TRUE 表示将现在位置加上 Position 作为当前位置， FLASE 表示将 Position 作为当前位置
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

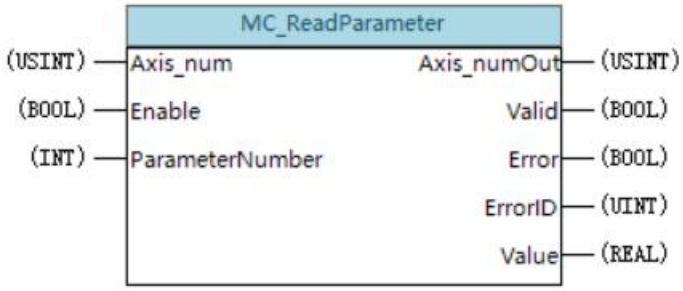
```
MC_SetPosition(Axis_num := ,Execute := ,Position := ,Relative := ,
          ExecutionMode := ,Axis_numOut => ,Done => ,Busy => ,
          Error => ,ErrorID => );
```

2.1.17 MC_ReadParameter

这个模块功能是读取轴指定的参数值。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
B	ParameterNumber	INT	所读取参数的编号
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	Value	REAL	所读取参数的值

功能块图形形式



功能块文本形式

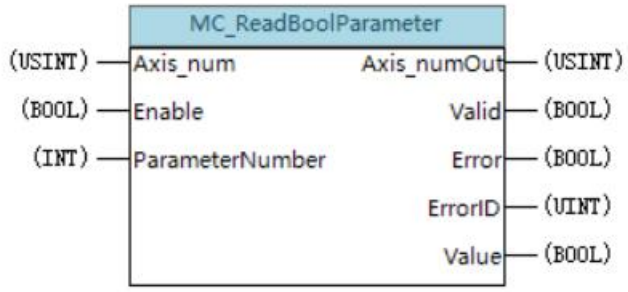
```
MC_ReadParameter(Axis_num := ,Enable := ,ParameterNumber := ,
          Axis_numOut => ,Valid => ,Error => ,ErrorID => ,Value => );
```

2.1.18 MC_ReadBoolParameter

这个模块功能是读取轴布尔参数中指定位的值。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
B	ParameterNumber	INT	布尔参数的位编号
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	Value	BOOL	读取对应位的值

功能块图形形式



功能块文本形式

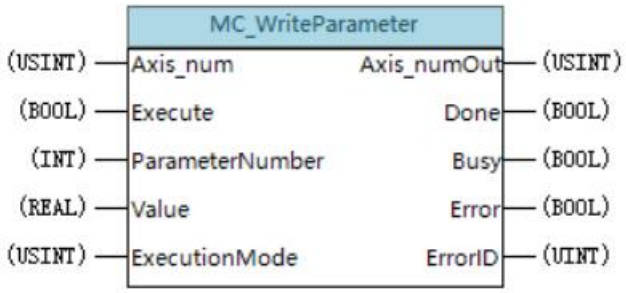
```
MC_ReadParameter(Axis_num := ,Enable := ,ParameterNumber := ,
          Axis_numOut => ,Valid => ,Error => ,ErrorID => ,Value => );
```

2.1.19 MC_WriteParameter

这个模块功能是将指定值写入轴的指定参数。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	ParameterNumber	INT	参数的编号
E	Value	REAL	写入的值
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

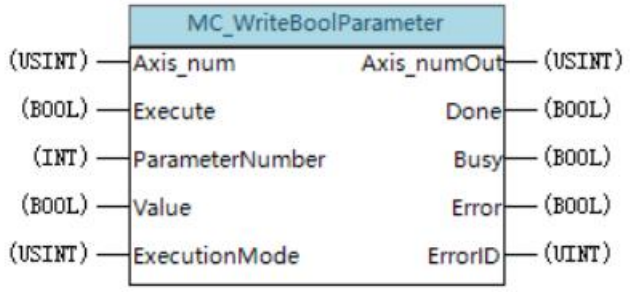
```
MC_WriteParameter(Axis_num := ,Execute := ,ParameterNumber := ,
                  Value := , ExecutionMode := ,Axis_numOut => ,
                  Done => ,Busy => ,Error => ,ErrorID => );
```

2.1.20 MC_WriteBoolParameter

这个模块功能是将指定值写入轴布尔参数的指定位。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	ParameterNumber	INT	布尔参数的位编号
E	Value	BOOL	写入的值
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_WriteBoolParameter(Axis_num := ,Execute := ,ParameterNumber := ,
          Value := ,ExecutionMode := ,Axis_numOut => ,Done => ,
```

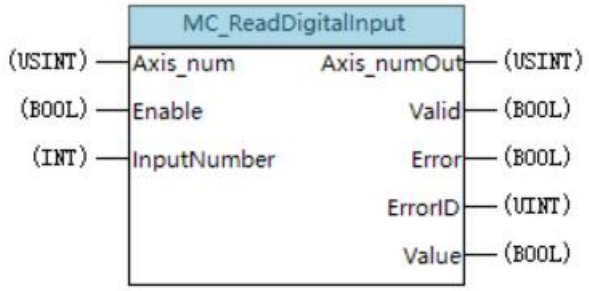
```
        Busy => ,Error => ,ErrorID => );
```

2.1.21 MC_ReadDigitalInput

这个模块功能是读取轴 DigitalInput 参数中指定位的值。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
B	InputNumber	INT	Input 中的位编号
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	Value	BOOL	所读取位的值

功能块图形形式



功能块文本形式

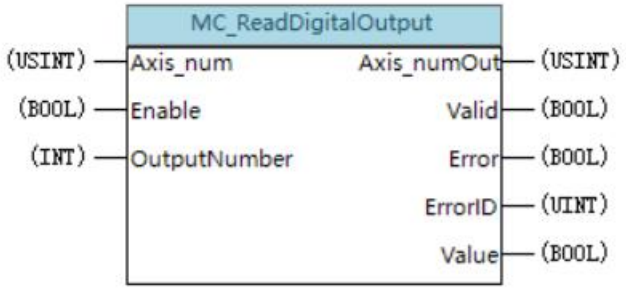
```
MC_ReadDigitalInput(Axis_num := ,Enable := ,InputNumber := ,
              Axis_numOut => ,Valid => ,Error => ,ErrorID => ,
              Value => );
```

2.1.22 MC_ReadDigitalOutput

这个模块功能是读取轴 DigitalOutput 参数中指定位的值。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
B	InputNumber	INT	Output 中的位编号
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	Value	BOOL	所读取位的值

功能块图形形式



功能块文本形式

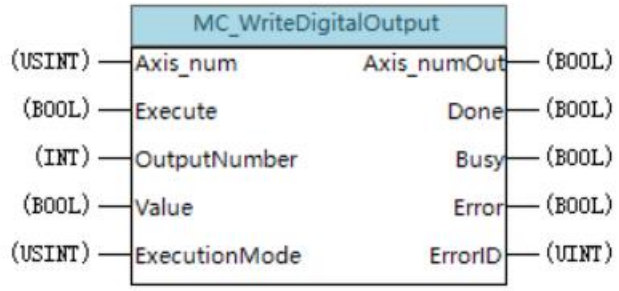
```
MC_ReadDigitalOutput(Axis_num := ,Enable := ,OutputNumber := ,
          Axis_numOut => ,Valid => ,Error => ,ErrorID => ,Value => );
```

2.1.23 MC_WriteDigitalOutput

这个模块功能是将指定值写入轴的指定参数。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
E	OutputNumber	INT	参数的编号
E	Value	BOOL	写入的值
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

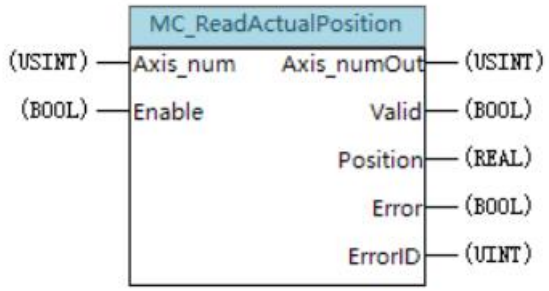
```
MC_WriteDigitalOutput(Axis_num := ,Execute := ,OutputNumber := ,
          Value := ,ExecutionMode := ,Axis_numOut => ,Done => ,
          Busy => ,Error => ,ErrorID => );
```

2.1.24 MC_ReadActualPosition

这个模块功能是读取轴的当前位置。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Position	REAL	当前位置
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

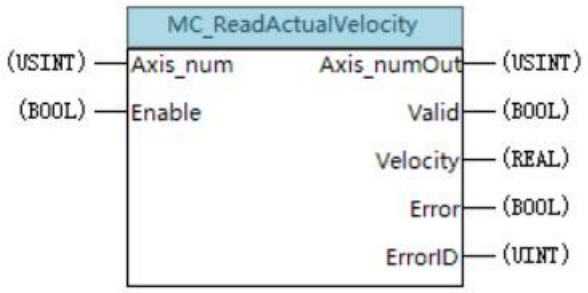
```
MC_ReadActualPosition(Axis_num := ,Enable := ,Axis_numOut => ,
                      Valid => ,Position => ,Error => ,ErrorID => );
```

2.1.25 MC_ReadActualVelocity

这个模块功能是读取轴的当前速度。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Velocity	REAL	当前速度
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

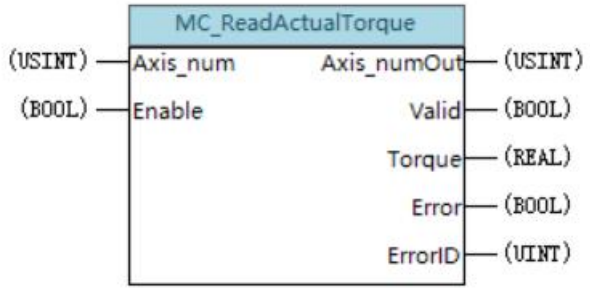
```
MC_ReadActualVelocity(Axis_num := ,Enable := ,Axis_numOut => ,
                      Valid => ,Velocity => ,Error => ,ErrorID => );
```

2.1.26 MC_ReadActualTorque

这个模块功能是读取轴的当前力矩。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效
B	Torque	REAL	当前力矩
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_ReadActualTorque(Axis_num := ,Enable := ,Axis_numOut => ,
                    Valid => ,Torque => ,Error => ,ErrorID => );
```

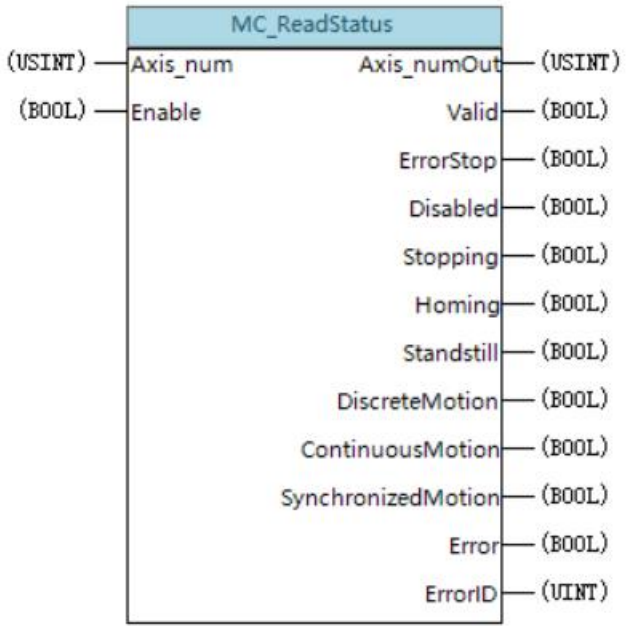
2.1.27 MC_ReadStatus

这个模块功能是读取轴的当前状态，相应的状态输出标志位会置位。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号

B	Valid	BOOL	当前读取的值是否有效
B	ErrorStop	BOOL	当前状态是否为 ErrorStop
B	Disabled	BOOL	当前状态是否为 Disabled
B	Stopping	BOOL	当前状态是否为 Stopping
B	Homing	BOOL	当前状态是否为 Homing
B	Standstill	BOOL	当前状态是否为 Standstill
B	DiscreteMotion	BOOL	当前状态是否为 DiscreteMotion
B	ContinuousMotion	BOOL	当前状态是否为 ContinuousMotion
B	SynchronizedMotion	BOOL	当前状态是否为 SynchronizedMotion
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_ReadStatus(Axis_num := ,Enable := ,Axis_numOut => ,Valid => ,
              ErrorStop => ,Disabled => ,Stopping => ,Homing => ,
              Standstill => ,DiscreteMotion => ,ContinuousMotion => ,
              SynchronizedMotion => ,Error => ,ErrorID => );
```

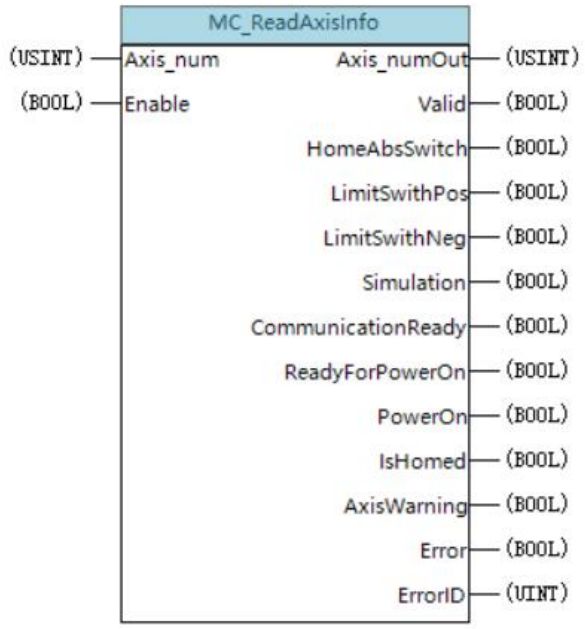
2.1.28 MC_ReadAxisInfo

这个模块功能是读取轴的当前信息，如限位开关、通讯状态、驱动器状态等。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效

B	HomeAbsSwitch	BOOL	原点开关是否置位
B	LimitSwithPos	BOOL	硬件正向限位开关是否置位
B	LimitSwithNeg	BOOL	硬件反向限位开关是否置位
B	Simulation	BOOL	当前是否处于仿真模式
B	CommunicationReady	BOOL	网络是否初始化完成可进行通信
B	ReadyForPowerOn	BOOL	驱动器是否处于可使能状态
B	PowerOn	BOOL	驱动器是否已使能
B	IsHomed	BOOL	轴是否已完成回零
B	AxisWarning	BOOL	驱动器是否报警
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_ReadAxisInfo(Axis_num := ,Enable := ,Axis_numOut => ,Valid => ,
          HomeAbsSwitch => ,LimitSwithPos => ,LimitSwithNeg => ,
          Simulation => ,CommunicationReady => ,ReadyForPowerOn => ,
          PowerOn => ,IsHomed => ,AxisWarning => ,Error => ,
          ErrorID => );
```

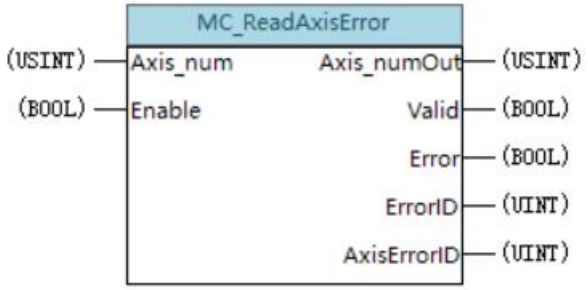
2.1.29 MC_ReadAxisError

这个模块功能是读取轴的错误。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Enable	BOOL	高电平使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Valid	BOOL	当前读取的值是否有效

B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	AxisErrorID	UINT	轴错误类型

功能块图形形式



功能块文本形式

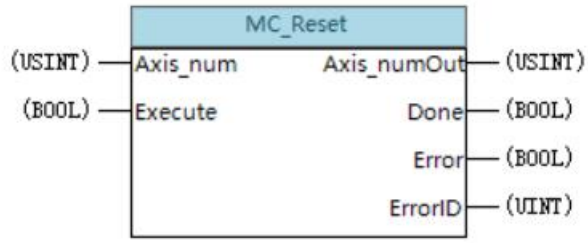
```
MC_ReadAxisError(Axis_num := ,Enable := ,Axis_numOut => ,Valid => ,
                 Error => ,ErrorID => ,AxisErrorID => );
```

2.1.30 MC_Reset

这个模块功能是将当前轴从状态 ErrorStop 转换成 Standstill 或 Disabled 并重置所有错误。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

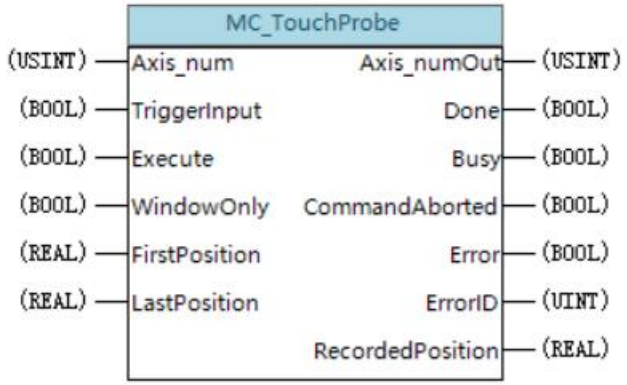
```
MC_Reset(Axis_num := ,Execute := ,Axis_numOut => ,Done => ,Error => ,
         ErrorID => );
```

2.1.31 MC_TouchProbe

模块名称	MC_TouchProbe
这个模块功能是检测信号源，当信号源上跳沿时记录下轴所在的位置。	

VAR_INPUT			
B	Axis_num	USINT	轴号
B	TriggerInput	BOOL	检测的信号源
B	Execute	BOOL	上跳沿使能
E	WindowOnly	BOOL	若为 TRUE,则只当轴位置在特定窗口范围内时信号源上跳沿才记录位置
E	FirstPosition	REAL	窗口位置起点
E	LastPosition	REAL	窗口位置终点
VAR_OUTPUT			
B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	CommandAborted	BOOL	模块是否被打断，只能被 MC_AbortTrigger 模块打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	RecordedPosition	REAL	记录的位置

功能块图形形式



功能块文本形式

```
MC_TouchProbe(Axis_num := ,TriggerInput := ,Execute := ,
              WindowOnly := , FirstPosition := ,LastPosition := ,
              Axis_numOut => ,Done => , Busy => ,
              CommandAborted => ,Error => ,ErrorID => ,
              RecordedPosition => );
```

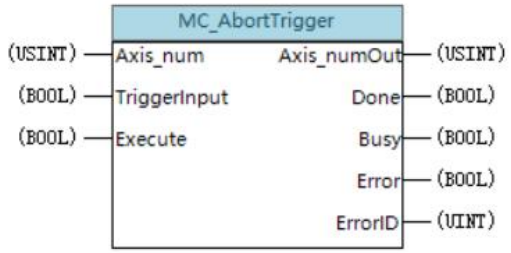
2.1.32 MC_AbortTrigger

这个模块功能是停止指定信号源的监控。

VAR_INPUT			
B	Axis_num	USINT	轴号
B	TriggerInput	BOOL	检测的信号源
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			

B	Axis_numOut	USINT	轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_AbortTrigger(Axis_num := ,TriggerInput := ,Execute := ,
                Axis_numOut => ,Done => ,Busy => ,Error => ,ErrorID => );
```

该部分功能块主要是控制多个轴的状态与运动，轴与轴之间是主从关系，一个主轴可以有多个从轴，但一个从轴最多只能有一个主轴。

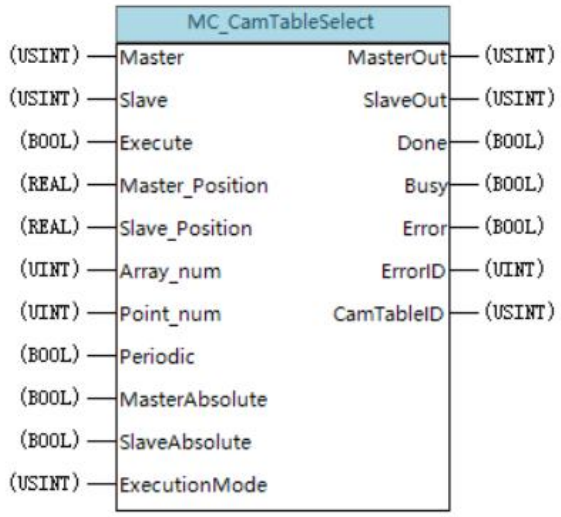
2.1.33 MC_CamTableSelect

该模块功能是为凸轮选择并创建凸轮表。

VAR_INPUT			
B	Master	USINT	主轴轴号
B	Slave	USINT	从轴轴号
B	Execute	BOOL	上跳沿使能
B	Master_Position	REAL*	凸轮表主轴位置
B	Slave_Position	REAL*	凸轮表从轴位置
B	Array_num	UINT	凸轮表数组长度
B	Point_num	UINT	凸轮表插补后的总点数
B	Periodic	BOOL	凸轮表是否具有周期性
E	MasterAbsolute	BOOL	若为 TRUE，则凸轮表中主轴的位置为绝对位置，否则凸轮表中主轴的位置为相对位置
E	SlaveAbsolute	BOOL	若为 TRUE，则凸轮表中从轴的位置为绝对位置，否则凸轮表中从轴的位置为相对位置
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
VAR_OUTPUT			
B	MasterOut	USINT	主轴轴号
B	SlaveOut	USINT	从轴轴号
B	Done	BOOL	模块是否完成

E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	CamTableID	USINT	创建的凸轮表编号，给 MC_CamIn 模块使用

功能块图形形式



功能块文本形式

```
MC_CamTableSelect(Master := ,Slave := ,Execute := ,
        Master_Position := ,Slave_Position := ,Array_num := ,
        Point_num := ,Periodic := ,MasterAbsolute := ,
        SlaveAbsolute := ,ExecutionMode := ,MasterOut => ,
        SlaveOut => ,Done => ,Busy => ,Error => ,
        ErrorID => ,CamTableID => );
```

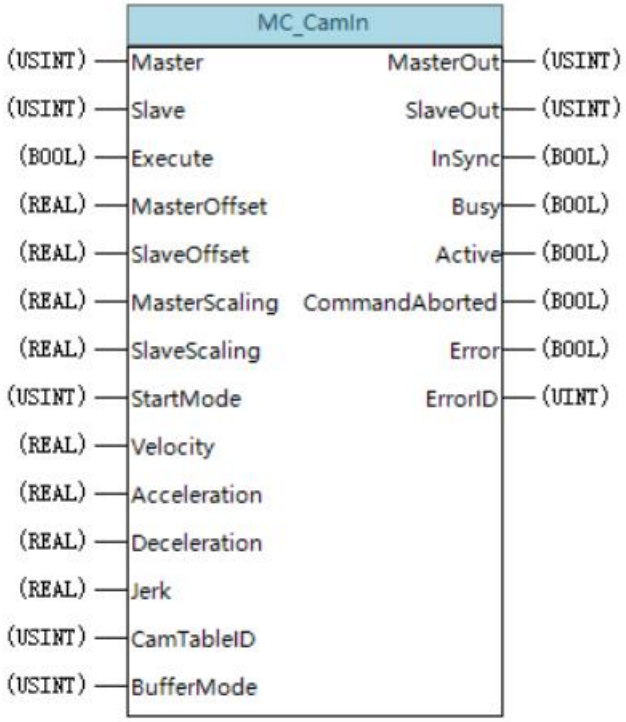
2.1.34 MC_CamIn

该模块功能是使凸轮耦合。

VAR_INPUT			
B	Master	USINT	主轴轴号
B	Slave	USINT	从轴轴号
B	Execute	BOOL	上跳沿使能
B	MasterOffset	REAL	凸轮表主轴偏移位置
B	SlaveOffset	REAL	凸轮表从轴偏移位置
B	MasterScaling	REAL	凸轮表主轴缩放比例
B	SlaveScaling	REAL	凸轮表从轴缩放比例
E	StartMode	USINT	启动模式 0.按照凸轮表的绝对性 1.主从轴均为绝对位置 2.主从轴均为相对位置 3.主轴为绝对位置，从轴为相对位置 4.主轴为相对位置，从轴为绝对位置
B	Velocity	REAL	从轴在同步过程中的最大速度

B	Acceleration	REAL	从轴在同步过程中的加速度
B	Deceleration	REAL	从轴在同步过程中的减速度
E	Jerk	REAL	从轴在同步过程中的加加速度
B	CamTableID	USINT	选择的凸轮表编号
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	MasterOut	USINT	主轴轴号
B	SlaveOut	USINT	从轴轴号
B	InSync	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	CamTableID	USINT	创建的凸轮表编号，给 MC_CamIn 模块使用

功能块图形形式



功能块文本形式

```
MC_CamIn(Master := ,Slave := ,Execute := ,MasterOffset := ,
        SlaveOffset := ,MasterScaling := ,SlaveScaling := ,
```

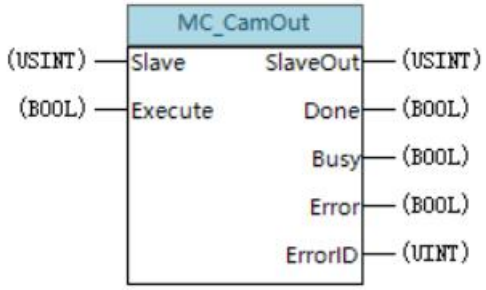
```
StartMode := ,Velocity := ,Acceleration := ,
Deceleration := ,Jerk := ,CamTableID := ,BufferMode := ,
MasterOut => ,SlaveOut => ,InSync => ,Busy => ,
Active => ,CommandAborted => ,Error => ,ErrorID => );
```

2.1.35 MC_CamOut

这个模块功能是立即将凸轮从轴与主轴解耦。

VAR_INPUT			
B	Slave	USINT	从轴号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	SlaveOut	USINT	轴号
B	Done	BOOL	当前模块是否完成
B	Busy	REAL	当前模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_CamOut(Slave := ,Execute := ,SlaveOut => ,Done => ,Busy => ,
          Error => ,ErrorID => );
```

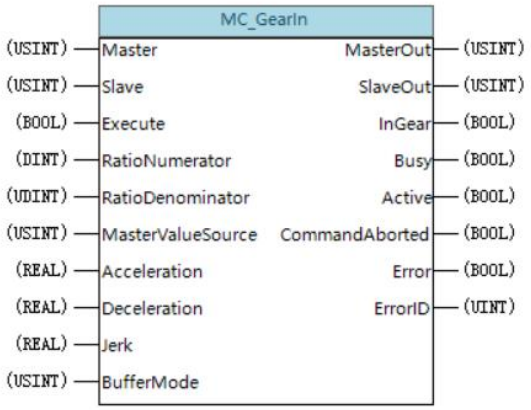
2.1.36 MC_GearIn

该模块功能是使齿轮耦合。

VAR_INPUT			
B	Master	USINT	主轴轴号
B	Slave	USINT	从轴轴号
B	Execute	BOOL	上跳沿使能
B	RatioNumerator	DINT	齿轮比分子
B	RatioDenominator	UDINT	齿轮比分母
E	MasterValueSource	USINT	参照的主轴值模式 0.根据主轴的设定值同步 1.根据主轴的实际值同步
B	Acceleration	REAL	从轴在同步过程中的加速度
B	Deceleration	REAL	从轴在同步过程中的减速度

E	Jerk	REAL	从轴在同步过程中的加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	MasterOut	USINT	主轴轴号
B	SlaveOut	USINT	从轴轴号
B	InGear	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	CamTableID	USINT	创建的凸轮表编号，给 MC_CamIn 模块使用

功能块图形形式



功能块文本形式

```
MC_GearIn(Master := ,Slave := ,Execute := ,RatioNumerator := ,
        RatioDenominator := ,MasterValueSource := ,Acceleration := ,
        Deceleration := ,Jerk := ,BufferMode := ,MasterOut => ,
        SlaveOut => ,InGear => ,Busy => ,Active => ,CommandAborted => ,
        Error => ,ErrorID => );
```

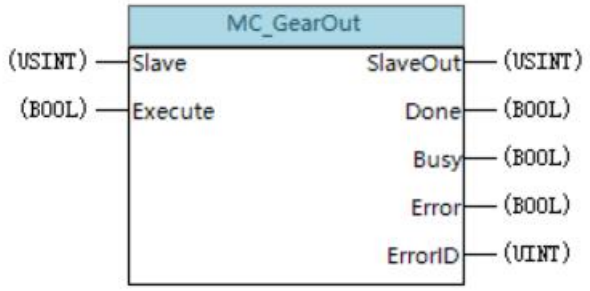
2.1.37 MC_GearOut

这个模块功能是立即将齿轮从轴与主轴解耦。

VAR_INPUT			
B	Slave	USINT	从轴号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	SlaveOut	USINT	轴号

B	Done	BOOL	当前模块是否完成
B	Busy	REAL	当前模块是否未完成
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_GearOut(Slave := ,Execute := ,SlaveOut => ,Done => ,Busy => ,
           Error => ,ErrorID => );
```

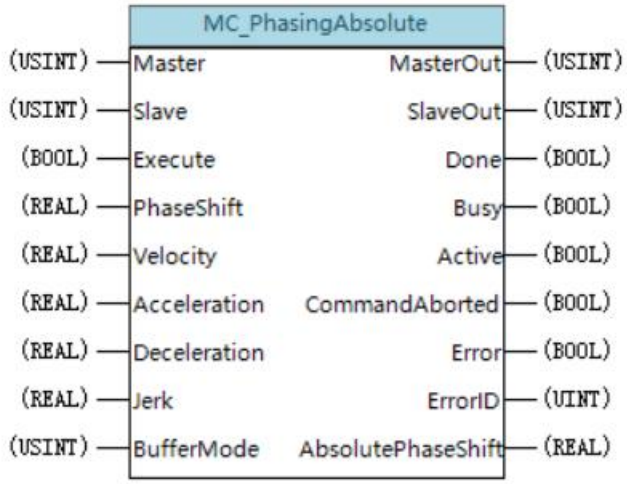
2.1.38 MC_PhasingAbsolute

这个模块功能是在从轴所参考的主轴位置上增加一个绝对相移。

VAR_INPUT			
B	Master	USINT	主轴轴号
B	Slave	USINT	从轴轴号
B	Execute	BOOL	上跳沿使能
B	PhaseShift	REAL	绝对相移
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	MasterOut	USINT	主轴轴号
B	SlaveOut	USINT	从轴轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错

B	ErrorID	UINT	出错类型
B	AbsolutePhaseShift	REAL	当前绝对相移

功能块图形形式



功能块文本形式

```
MC_PhasingAbsolute(Master := ,Slave := ,Execute := ,PhaseShift := ,
          Velocity := ,Acceleration := ,Deceleration := ,Jerk := ,
          BufferMode := ,MasterOut => ,SlaveOut => ,Done => ,Busy => ,
          Active => ,CommandAborted => ,Error => ,ErrorID => ,
          AbsolutePhaseShift => );
```

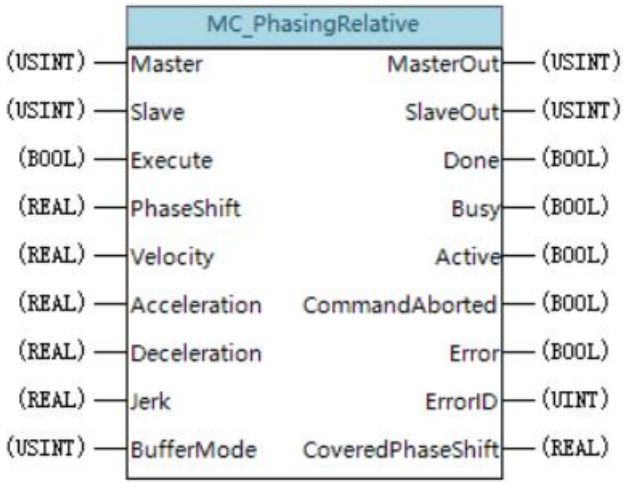
2.1.39 MC_PhasingRelative

这个模块功能是在从轴所参考的主轴位置上增加一个相对相移。

VAR_INPUT			
B	Master	USINT	主轴轴号
B	Slave	USINT	从轴轴号
B	Execute	BOOL	上跳沿使能
B	PhaseShift	REAL	相对相移
B	Velocity	REAL	最大速度
B	Acceleration	REAL	加速度
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	MasterOut	USINT	主轴轴号

B	SlaveOut	USINT	从轴轴号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被失能所打断
B	Error	BOOL	模块运行是否出错
B	ErrorID	UINT	出错类型
B	CoveredPhaseShift	REAL	当前相对相移

功能块图形形式



功能块文本形式

```
MC_PhasingRelative(Master := ,Slave := ,Execute := ,PhaseShift := ,
          Velocity := ,Acceleration := ,Deceleration := ,Jerk := ,
          BufferMode := ,MasterOut => ,SlaveOut => ,Done => ,Busy => ,
          Active => ,CommandAborted => ,Error => ,ErrorID => ,
          CoveredPhaseShift => );
```

2.2 轴组模块

该部分功能块是控制轴组的状态及运动。

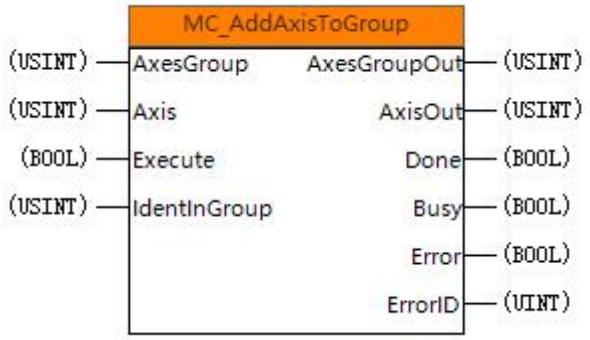
2.2.1 MC_AddAxisToGrououp

这个模块功能是把单轴加入到轴组

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	AxisOut	USINT	单轴轴号
B	Execute	BOOL	上跳沿使能
E	IdentInGroup	USINT	轴组内单轴编号
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Axis Out	USINT	单轴轴号
B	Done	BOOL	模块是否完成

E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

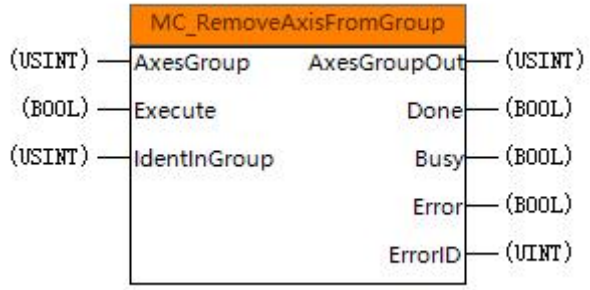
```
MC_AddAxisToGroup(AxesGroup := ,Axis := ,Execute := ,IdentInGroup:=
                  ,AxesGroupOut => ,AxisOut => ,Done => ,Busy
                 => ,Error => ,ErrorID => );
```

2.2.2 MC_RemoveAxisFromGroup

这个模块功能是把单轴移除轴组

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
E	IdentInGroup	USINT	轴组内单轴编号
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_RemoveAxisFromGroup(AxesGroup := ,Execute := ,IdentInGroup := ,Axe
                       sGroupOut => ,Done => ,Busy => ,Error
```

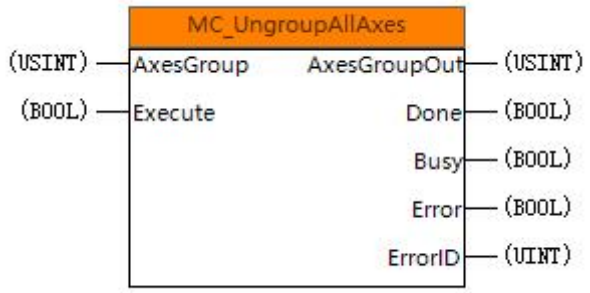
```
=> ,ErrorID => );
```

2.2.3 MC_UngroupAllAxes

这个模块功能是轴组内的所有轴

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_UngroupAllAxes(AxesGroup := ,Execute := ,AxesGroupOut => ,Done
                  => ,Busy => ,Error => ,ErrorID => );
```

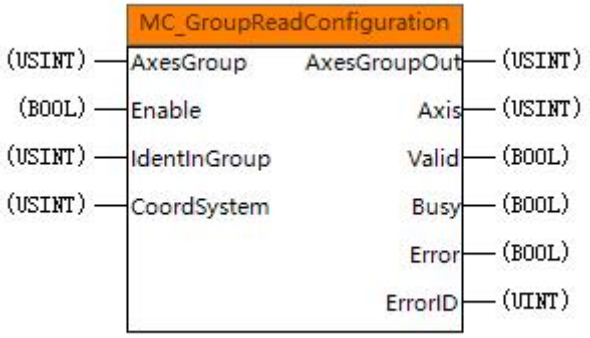
2.2.4 MC_GroupReadConfiguration

这个模块功能是读取轴组内指定单轴的配置信息

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
B	IdentInGroup	USINT	轴组内单轴编号
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Axis	USINT	单轴编号
B	Valid	BOOL	当前读取的值是否有效

E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

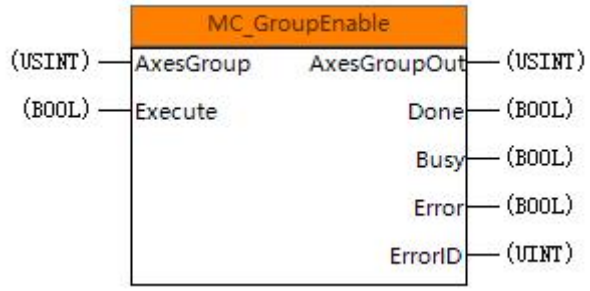
```
MC_GroupReadActualPosition(AxesGroup := ,Enable := ,CoordSystem :=
                           ,AxesGroupOut => ,Valid => ,Busy
                          => ,Error=> ,ErrorID => ,Position => );
```

2.2.5 MC_GroupEnable

这个模块功能是轴组使能，使轴组状态从 GroupDisabled 切换至 GroupStandby

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

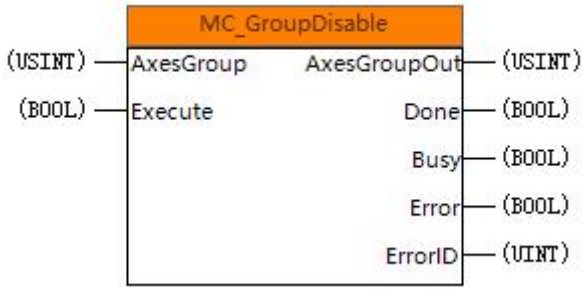
```
MC_GroupDisable(AxesGroup := ,Execute := ,AxesGroupOut => ,Done
                => ,Busy => ,Error => ,ErrorID => );
```

2.2.6 MC_GroupDisable

这个模块功能是轴组使能，使轴组状态从切换至 GroupDisabled

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_GroupDisable(AxesGroup := ,Execute := ,AxesGroupOut => ,Done
                => ,Busy => ,Error => ,ErrorID => );
```

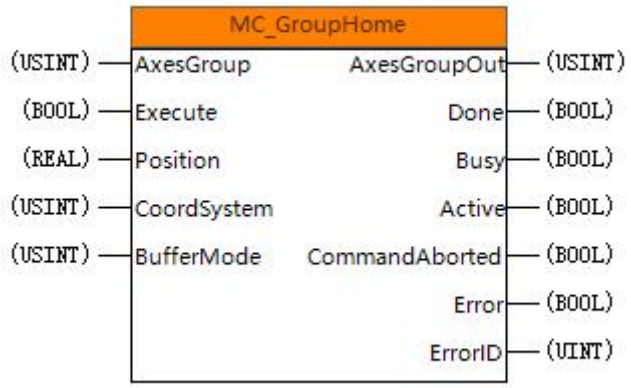
2.2.7 MC_GroupHome

这个模块功能是控制轴组运动回原点

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Position	REAL*	目标位置数组指针
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承

VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_GroupHome(AxesGroup := ,Execute := ,Position := ,CoordSystem :=
             ,BufferMode := ,AxesGroupOut => ,Done => ,Busy
            => ,Active => ,CommandAborted => ,Error => ,ErrorID => );
```

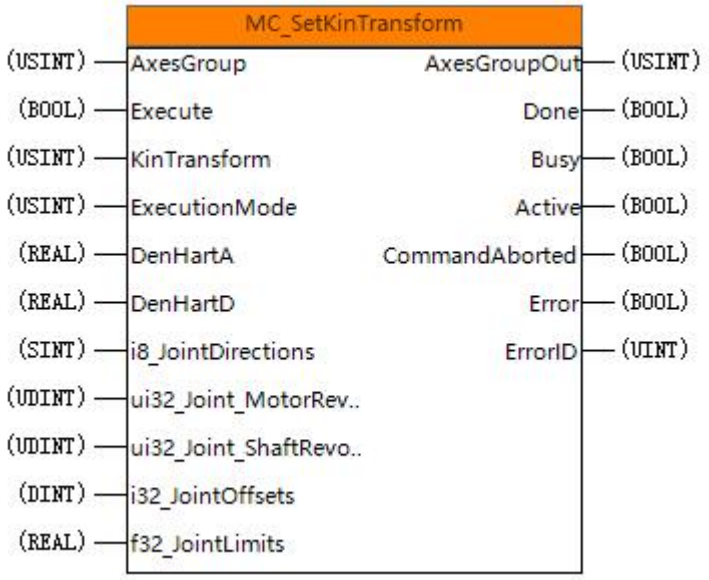
2.2.8 MC_SetKinTransform

这个模块功能是设置轴组 ACS 到 MCS 的运动变换

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	KinTransform	USINT	运动学类型
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
B	DenHartA	REAL*	DHA 参数指针
B	DenHartD	REAL*	DHA 参数指针
B	i8_JointDirections	SINT*	轴组内各单轴运动方向指针
B	ui32_Joint_MotorRevolution	UDINT*	轴组内各单轴齿轮比分母指针
B	ui32_Joint_ShaftRevolution	UDINT*	轴组内各单轴齿轮比分子指针
B	i32_JointOffsets	INT*	轴组内各单轴偏移量指针
B	f32_JointLimits	REAL*	轴组内各单轴限位指针

VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_SetKinTransform(AxesGroup := ,Execute := ,KinTransform := ,
                   ExecutionMode := ,DenHartA := ,DenHartD := ,i8_Jo
                   intDirections := ,ui32_Joint_MotorRevolution :=
                    ,ui32_Joint_ShaftRevolution := ,
                   i32_JointOffsets := ,f32_JointLimits := ,AxesGro
                   upOut => ,Done => ,Busy => ,Active
                   => ,CommandAborted  => ,Error => ,ErrorID => );
```

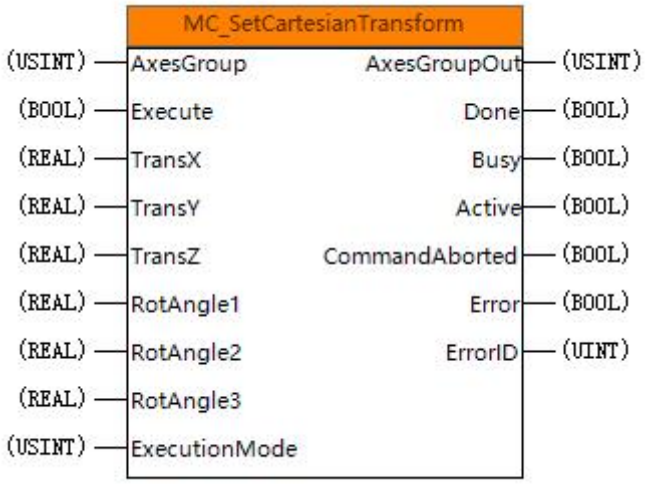
2.2.9 MC_SetCartesianTransform

这个模块功能是实现 MCS 到 PCS 的直角坐标变换

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	TranX	REAL	平移向量的 X 分量
B	TranY	REAL	平移向量的 Y 分量
B	TranZ	REAL	平移向量的 Z 分量
B	RotAngle1	REAL	旋转角的分量

B	RotAngle2	REAL	旋转角的分量
B	RotAngle3	REAL	旋转角的分量
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_SetCartesianTransform(AxesGroup := ,Execute := ,TransX := ,
                         TransY := ,TransZ := ,RotAngle1 := ,RotAng
                         le2 := ,RotAngle3 := ,ExecutionMode := ,Ax
                         esGroupOut => ,Done => ,Busy => ,Active
                         => ,CommandAborted => ,Error => ,ErrorID
                         => );
```

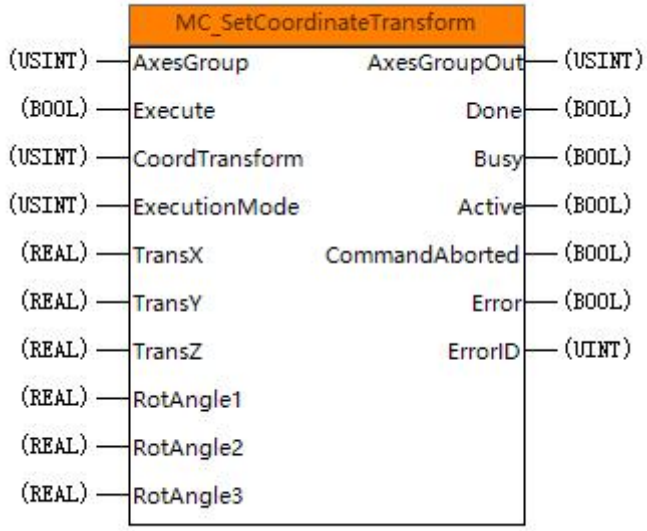
2.2.10 MC_SetCoordinateTransform

这个模块功能是实现 MCS 到 PCS 的坐标变换

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系

			2.MCS 世界坐标系 3.PCS 工件坐标系
E	ExecutionMode	USINT	指令缓冲类型 0.mcImmediately 立即执行 1.mcQueued 缓冲
B	TranX	REAL	平移向量的 X 分量
B	TranY	REAL	平移向量的 Y 分量
B	TranZ	REAL	平移向量的 Z 分量
B	RotAngle1	REAL	旋转角的分量
B	RotAngle2	REAL	旋转角的分量
B	RotAngle3	REAL	旋转角的分量
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

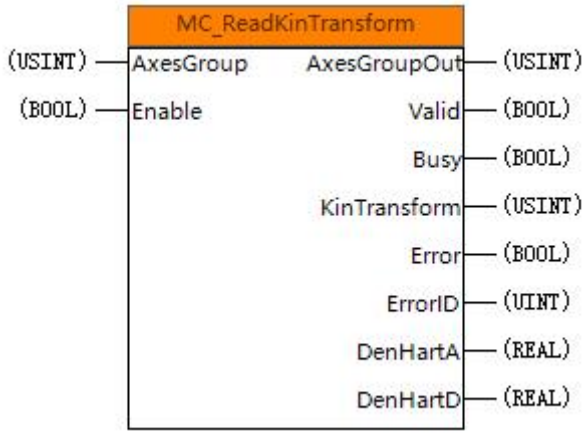
```
MC_SetCoordinateTransform(AxesGroup := ,Execute := ,
                          CoordTransform := ,ExecutionMode := ,Tran
                          sX := ,TransY := ,TransZ := ,RotAngle1 :=
                           ,RotAngle2 := ,RotAngle3 := ,AxesGroupOu
                          t => ,Done => ,Busy => ,Active
                          => ,CommandAborted => ,Error => ,ErrorID
                          => );
```

2.2.11 MC_ReadKinTransform

这个模块功能是读取轴组 ACS 到 MCS 运动变换参数

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	KinTransform	USINT	运动学类型
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
B	DenHartA	REAL*	DHA 参数指针
B	DenHartD	REAL*	DHA 参数指针

功能块图形形式



功能块文本形式

```
MC_ReadKinTransform(AxesGroup:=,Enable:=,AxesGroupOut=>,Valid=>,
                    Busy=>,KinTransform=>,Error=>,ErrorID=>,DenHa
                    rtA=>,DenHartD=>);
```

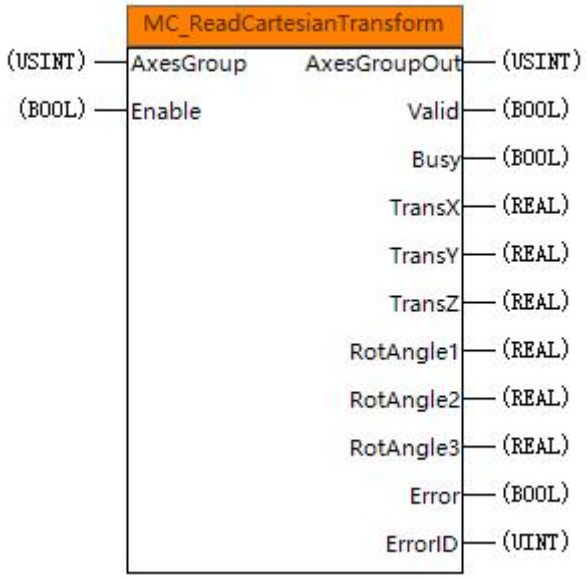
2.2.12 MC_ReadCartesianTransform

这个模块功能是读取轴组 ACS 到 MCS 直角坐标变换参数

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效

E	Busy	BOOL	模块是否未完成
B	TranX	REAL	平移向量的 X 分量
B	TranY	REAL	平移向量的 Y 分量
B	TranZ	REAL	平移向量的 Z 分量
B	RotAngle1	REAL	旋转角的分量
B	RotAngle2	REAL	旋转角的分量
B	RotAngle3	REAL	旋转角的分量
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_ReadCartesianTransform(AxesGroup := ,Enable := ,AxesGroupOut
                          => ,Valid => ,Busy => ,TransX => ,TransY
                          => ,TransZ => ,RotAngle1 => ,RotAngle2
                          => ,RotAngle3 => ,Error => ,ErrorID => );
```

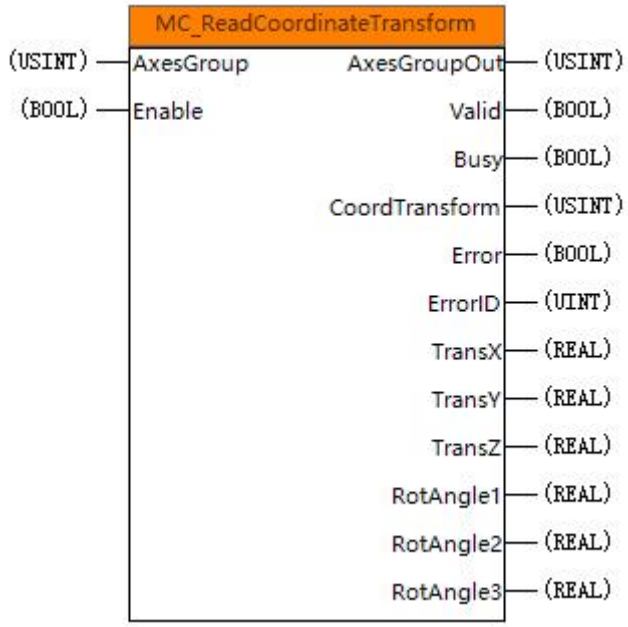
2.2.13 MC_ReadCoordinateTransform

这个模块功能是读取轴组 ACS 到 MCS 坐标变换参数

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	CoordTransform	USINT	坐标变换类型

B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
B	TranX	REAL	平移向量的 X 分量
B	TranY	REAL	平移向量的 Y 分量
B	TranZ	REAL	平移向量的 Z 分量
B	RotAngle1	REAL	旋转角的分量
B	RotAngle2	REAL	旋转角的分量
B	RotAngle3	REAL	旋转角的分量

功能块图形形式



功能块文本形式

```
MC_ReadCoordinateTransform(AxesGroup:=,Enable:=,AxesGroupOut=>,
                           Valid=>,Busy=>,CoordTransform=>,Error=>,
                           ErrorID=>,TransX=>,TransY=>,TransZ=>,Ro
                           tAngle1=>,RotAngle2=>,RotAngle3=>);
```

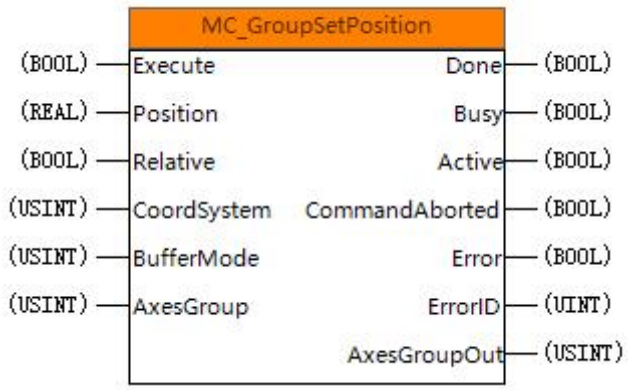
2.2.14 MC_GroupSetPosition

这个模块功能是将轴组设置为输入的位置

VAR_INPUT			
B	Execute	BOOL	上跳沿使能
B	Position	REAL*	目标位置数组指针
E	Relative	BOOL	为 TRUE 设置相对位置，为 FALSE 设置绝对位置
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断

			1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
B	AxesGroup	USINT	轴组编号
VAR_OUTPUT			
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
B	AexsGroupOut	USINT	轴组编号

功能块图形形式



功能块文本形式

```
MC_GroupSetPosition(Execute := ,Position := ,Relative := ,
                    CoordSystem := ,BufferMode := ,AxesGroup := ,Done
                    => ,Busy => ,Active => ,CommandAborted => ,Error
                    => ,ErrorID => ,AxesGroupOut => );
```

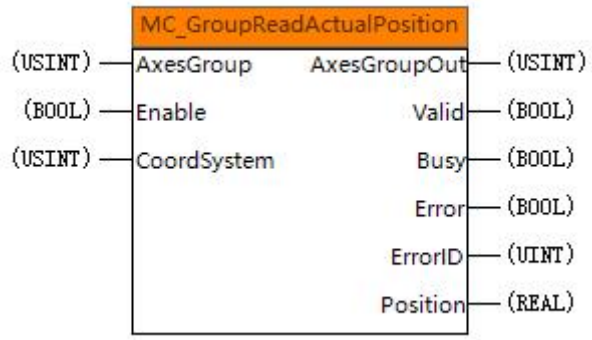
2.2.15 MC_GroupReadActualPosition

这个模块功能是读取轴组实际位置

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号

B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
B	Position	REAL*	实际位置数组指针

功能块图形形式



功能块文本形式

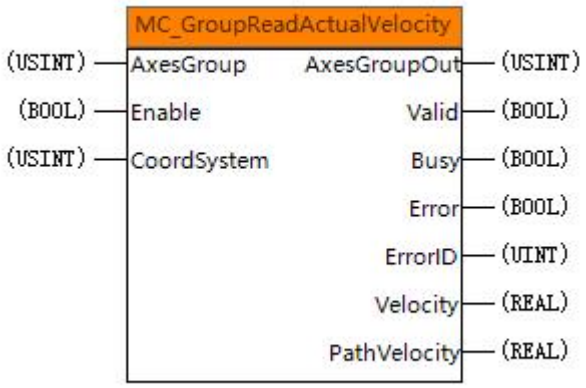
```
MC_GroupReadActualPosition(AxesGroup := ,Enable := ,CoordSystem :=
                           ,AxesGroupOut => ,Valid=> ,Busy => ,Error
                           => ,ErrorID => ,Position => );
```

2.2.16 MC_GroupReadActualVelocity

这个模块功能是读取轴组实际速度

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
B	Velocity	REAL*	各单轴实际速度数组指针
E	PathVelocity	REAL	TCP 实际速度

功能块图形形式



功能块文本形式

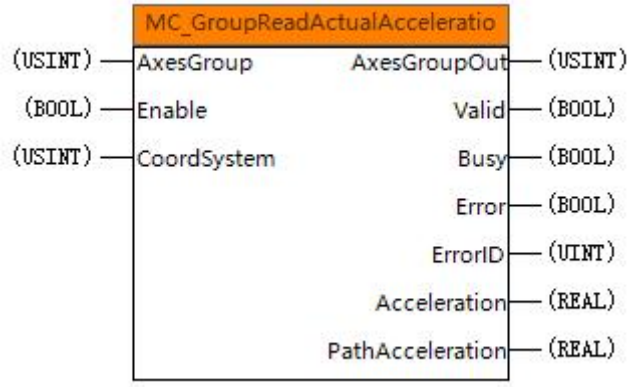
```
MC_GroupReadActualVelocity(AxesGroup := ,Enable := ,CoordSystem :=
                           ,AxesGroupOut => ,Valid => ,Busy
                          => ,Error => ,ErrorID => ,Velocity
                          => ,PathVelocity => );
```

2.2.17 MC_GroupReadActualAcceleration

这个模块功能是读取轴组实际加速度

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
B	Acceleration	REAL*	各单轴实际加速度数组指针
E	PathAcceleration	REAL	TCP 实际加速度

功能块图形形式



功能块文本形式

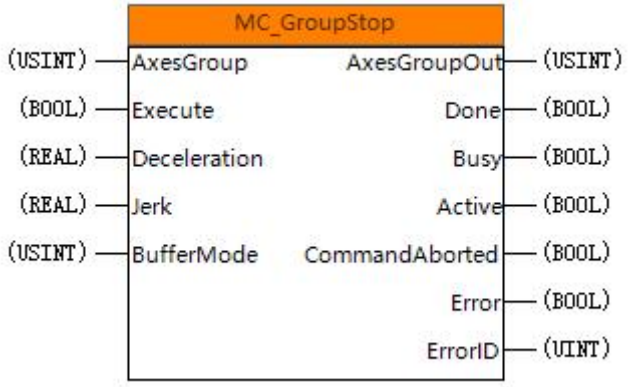
```
MC_GroupReadActualAcceleration(AxesGroup := ,Enable := ,CoordSystem
                               := ,AxesGroupOut => ,Valid => ,Busy
                              => ,Error => ,ErrorID => ,Acceleration
                              => ,PathAcceleration => );
```

2.2.18 MC_GroupStop

这个模块功能是使轴组减速停止，轴组状态切换为 GroupStopping

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

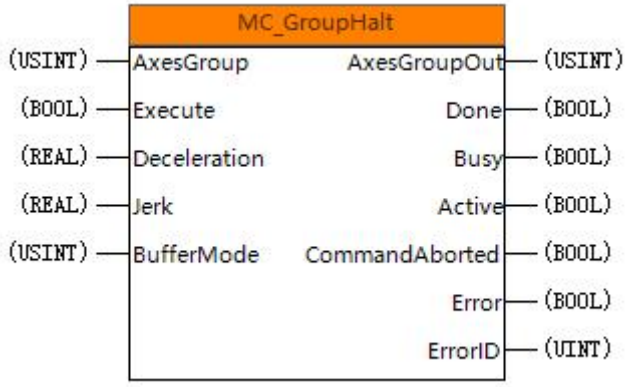
```
MC_GroupStop(AxesGroup := ,Execute := ,Deceleration := ,Jerk := ,Buffe
             rMode := ,AxesGroupOut => ,Done => ,Busy => ,Active
             => ,CommandAborted => ,Error => ,ErrorID => );
```

2.2.19 MC_GroupHalt

这个模块功能是使轴组减速停止，完成后轴组状态切换为 GroupStandby

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

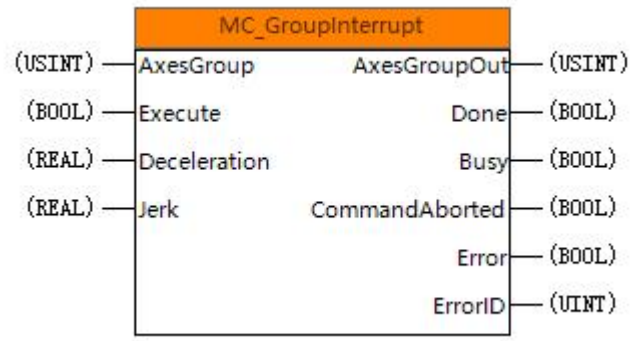
```
MC_GroupHalt(AxesGroup := ,Execute := ,Deceleration := ,Jerk := ,Buffe
             rMode := ,AxesGroupOut => ,Done => ,Busy => ,Active
             => ,CommandAborted => ,Error => ,ErrorID => );
```

2.2.20 MC_GroupInterrupt

这个模块功能是中断当前轴组运动，但可以恢复

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_GroupInterrupt(AxesGroup := ,Execute := ,Deceleration := ,Jerk
```

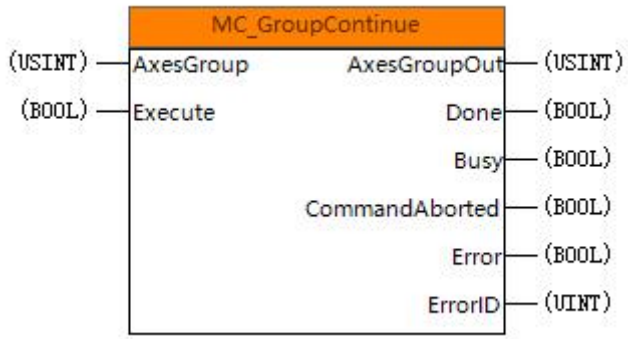
```
:= ,AxesGroupOut => ,Done => ,Busy
=> ,CommandAborted => ,Error => ,ErrorID => );
```

2.2.21 MC_GroupContinue

这个模块功能是返回并继续中断前的轴组运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

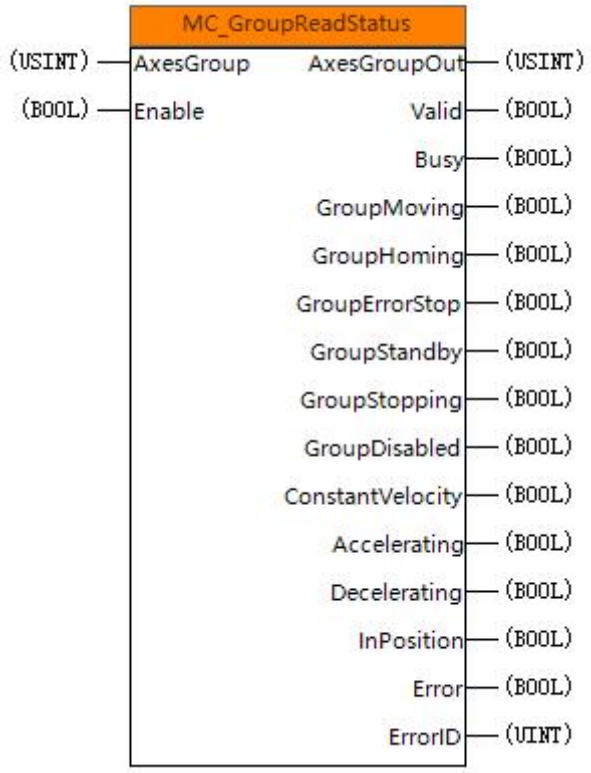
```
MC_GroupContinue(AxesGroup := ,Execute := ,AxesGroupOut => ,Done
=> ,Busy => ,CommandAborted => ,Error => ,ErrorID
=> );
```

2.2.22 MC_GroupReadStatus

这个模块功能是读取轴组状态参数

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	GroupMoving	BOOL	运动状态

B	GroupHoming	BOOL	回零状态
B	GroupErrorStop	BOOL	故障状态
B	GroupStrandby	BOOL	静止状态
B	GroupStopping	BOOL	停止状态
B	GroupDisabled	BOOL	失能状态
E	ConstantVelocity	BOOL	匀速状态
E	Accelerating	BOOL	加速状态
E	Deceletating	BOOL	减速状态
E	InPosition	BOOL	是否到达目标位置
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型



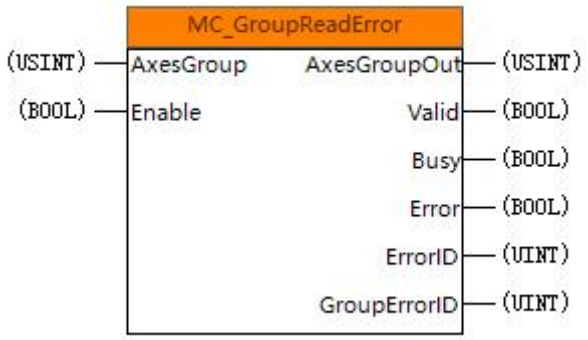
功能块文本形式

```
MC_GroupReadStatus(AxesGroup := ,Enable := ,AxesGroupOut => ,Valid
                   => ,Busy => ,GroupMoving => ,GroupHoming
                   => ,GroupErrorStop => ,GroupStandby
                   => ,GroupStopping => ,GroupDisabled
                   => ,ConstantVelocity => ,Accelerating
                   => ,Decelerating => ,InPosition => ,Error
                   => ,ErrorID => );
```

2.2.23 MC_GroupReadError

这个模块功能是读取轴组错误参数

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	为 TRUE 则使能，为 FALSE 则失能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Valid	BOOL	当前读取的值是否有效
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型
E	GroupErrorID	UINT	轴组出错类型



功能块文本形式

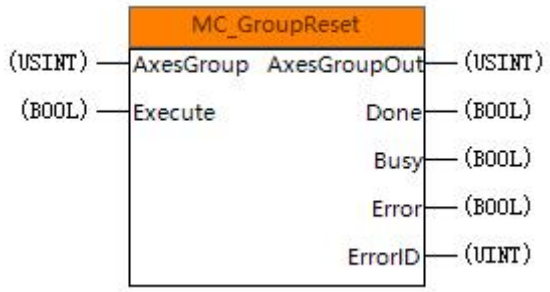
```
MC_GroupReadError(AxesGroup := ,Enable := ,AxesGroupOut => ,Valid
                  => ,Busy => ,Error => ,ErrorID => ,GroupErrorID => );
```

2.2.24 MC_GroupReset

这个模块功能是复位轴组状态，使轴组状态从 GroupErrorStop 切换至 GroupStandby

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
VAR_OUTPUT			
B	AexsGroupOut	USINT	轴组编号
B	Done	USINT	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_GroupReset(AxesGroup := ,Execute := ,AxesGroupOut => ,Done => ,Busy
              => ,Error => ,ErrorID => );
```

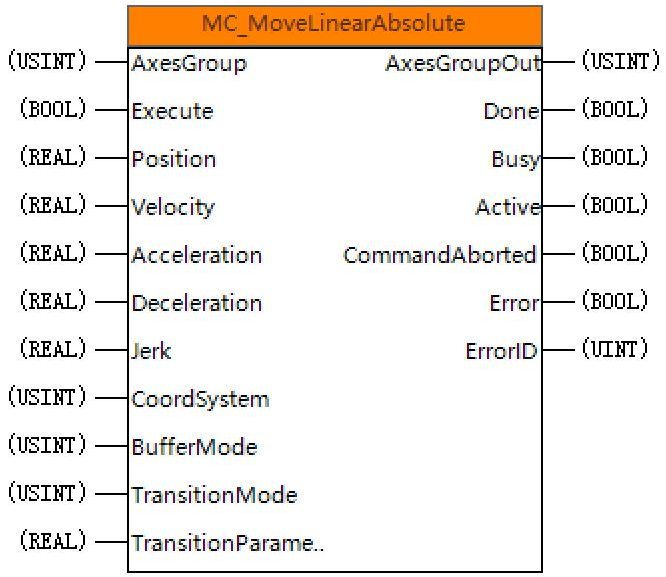
2.2.25 MC_MoveLinearAbsolute

这个模块功能是控制轴组直线绝对运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Position	REAL*	绝对目标位置数组指针
E	Velocity	REAL	速度
E	Acceleration	REAL	加速度
E	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
E	TransitionMode	USINT	过度模式
E	TransitionParameter	REAL	过度模式参数
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断

B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveLinearAbsolute(AxesGroup := ,Execute := ,Position := ,
                      Velocity := ,Acceleration := ,Deceleration
                      := ,Jerk := ,CoordSystem := ,BufferMode :=
                      ,TransitionMode := ,
                      TransitionParameter := ,AxesGroupOut
                      => ,Done => ,Busy => ,Active
                      => ,CommandAborted => ,Error => ,ErrorID => );
```

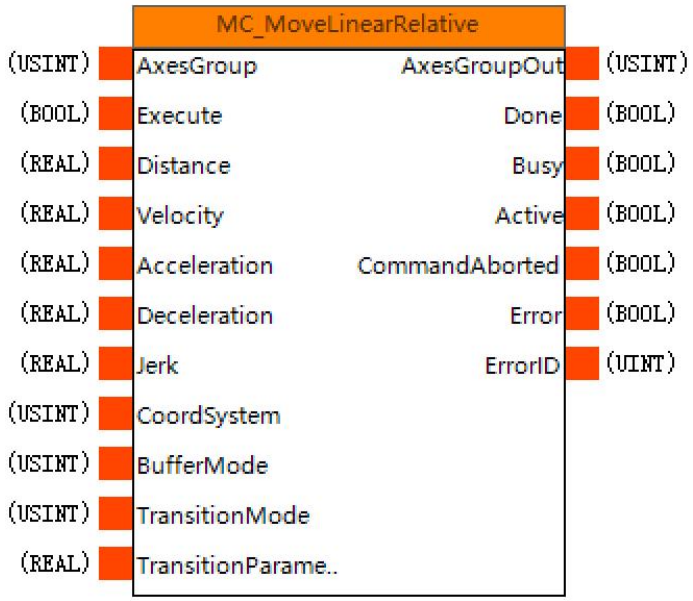
2.2.26 MC_MoveLinearRelative

这个模块功能是控制轴组直接相对运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Distance	REAL*	相对距离数组指针
E	Velocity	REAL	速度
E	Acceleration	REAL	加速度
E	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断

			1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
E	TransitionMode	USINT	过度模式
E	TransitionParameter	REAL	过度模式参数
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

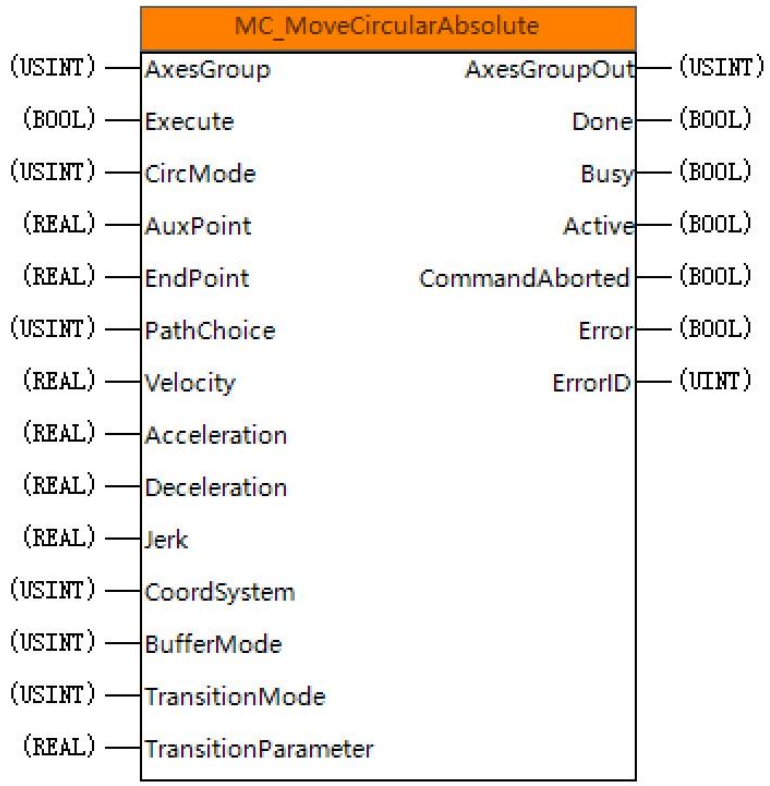
```
MC_MoveLinearRelative(AxesGroup := ,Execute := ,Distance := ,
                      Velocity := ,Acceleration := ,Deceleration
                      := ,Jerk := ,CoordSystem := ,BufferMode :=
                      ,TransitionMode := ,
                      TransitionParameter := ,AxesGroupOut
                      => ,Done => ,Busy => ,Active
                      => ,CommandAborted => ,Error => ,ErrorID => );
```

2.2.27 MC_MoveCircularAbsolute

这个模块功能是控制轴组圆弧绝对运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	CircMode	USINT	圆弧插补模式类型 1. BORDER 边界模式 2. CENTER 圆心模式
B	AuxPoint	REAL*	辅助点位置指针
B	EndPoint	REAL*	终止点位置指针
E	PathChoice	USINT	轨迹方向类型 1. CLOCKWISE 顺时针 2. COUNTERCLOCKWISE 逆时针
E	Velocity	REAL	速度
E	Acceleration	REAL	加速度
E	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
E	TransitionMode	USINT	过度模式
E	TransitionParameter	REAL	过度模式参数
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveCircularAbsolute(AxesGroup := ,Execute := ,CircMode := ,
                        AuxPoint := ,EndPoint := ,PathChoice := ,Veloc
                        ity := ,Acceleration := ,Deceleration := ,Jerk
                        := ,CoordSystem := ,BufferMode := ,Transition
                        Mode := ,TransitionParameter := ,
                        AxesGroupOut => ,Done => ,Busy => ,Active
                        => ,CommandAborted => ,Error => ,ErrorID => );
```

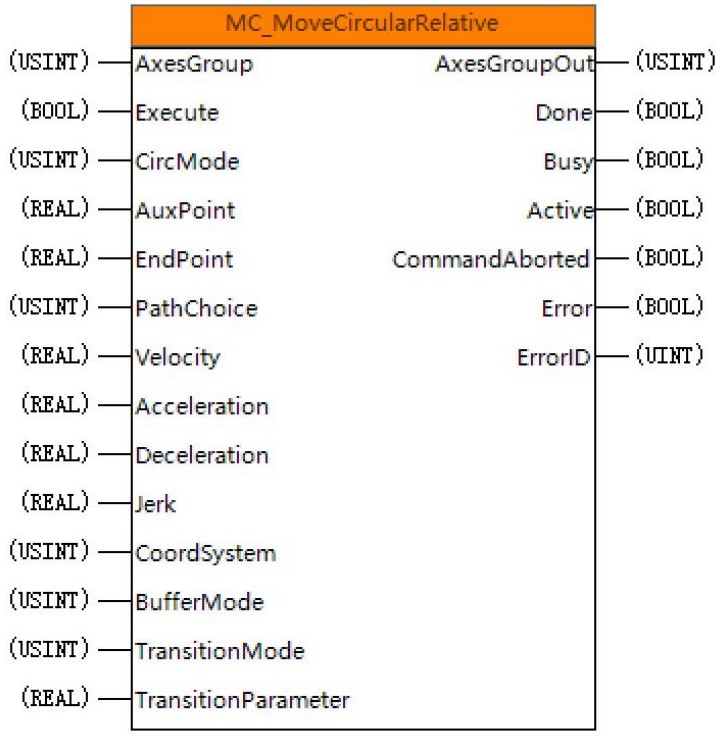
2.2.28 MC_MoveCircularRelative

这个模块功能是控制轴组圆弧相对运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	CircMode	USINT	圆弧插补模式类型 1. BORDER 边界模式 2. CENTER 圆心模式
B	AuxPoint	REAL*	辅助点位置指针
B	EndPoint	REAL*	终止点位置指针
E	PathChoice	USINT	轨迹方向类型 1. CLOCKWISE 顺时针 2. COUNTERCLOCKWISE 逆时针
E	Velocity	REAL	速度

E	Acceleration	REAL	加速度
E	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
E	TransitionMode	USINT	过度模式
E	TransitionParameter	REAL	过度模式参数
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveCircularRelative(AxesGroup := ,Execute := ,CircMode := ,
                        AuxPoint := ,EndPoint := ,PathChoice := ,Veloc
                        ity := ,Acceleration := ,Deceleration := ,Jerk
                        := ,CoordSystem := ,BufferMode := ,
                        TransitionMode := ,TransitionParameter := ,A
                        xesGroupOut => ,Done => ,Busy => ,Active
                        => ,CommandAborted => ,Error => ,ErrorID => );
```

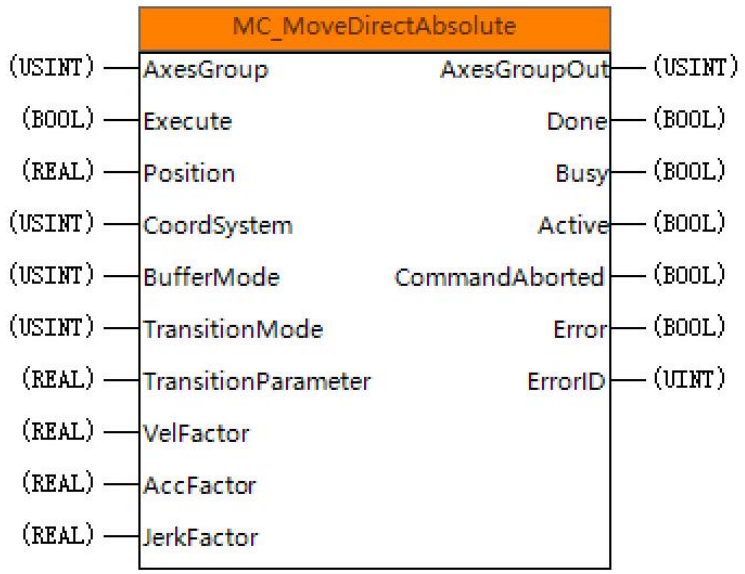
2.2.29 MC_MoveDirectAbsolute

这个模块功能是控制轴组点到点绝对运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
	Execute	BOOL	上跳沿使能
B	Position	REAL*	绝对目标位置数组指针
	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承

			4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
	TransitionMode	USINT	过度模式
	TransitionParamete r	REAL	过度模式参数
B	VelFactor	REAL	矢量速度的新覆盖因子
E	AccFactor	REAL	矢量加速/减速的新覆盖因子
E	JerkFactor	REAL	矢量加加速度的新覆盖因子
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
	Active	BOOL	当前模块是否正在控制轴
	CommandAborted	BOOL	模块是否被打断
	Error	BOOL	模块运行是否出错
	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_MoveDirectAbsolute(AxesGroup := ,Execute := ,Position := ,
                      CoordSystem := ,BufferMode := ,TransitionMod
                      e := ,TransitionParameter := ,VelFactor :=
                      ,AccFactor := ,JerkFactor := ,AxesGroupOut
                      => ,Done => ,Busy => ,Active
                      => ,CommandAborted => ,Error => ,ErrorID => );
```

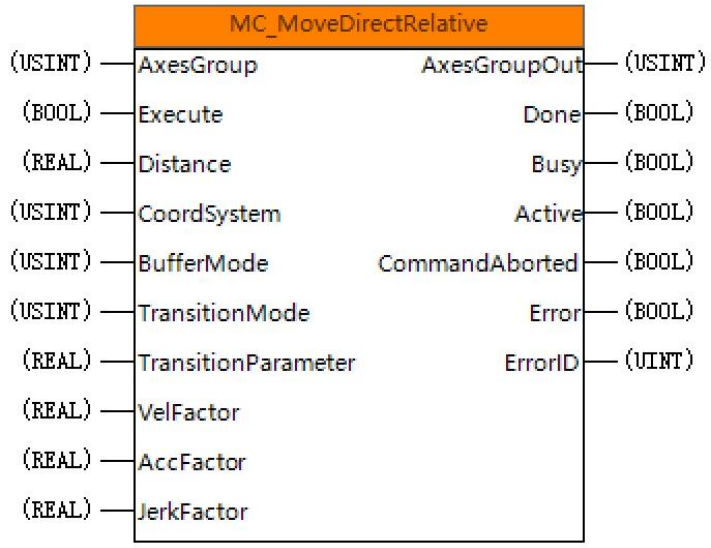
2.2.30 MC_MoveDirectRelative

这个模块功能是控制轴组点到点相对运动

VAR_INPUT

B	AxesGroup	USINT	轴组编号
B	Execute	BOOL	上跳沿使能
B	Distance	REAL*	相对距离数组指针
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
E	TransitionMode	USINT	过度模式
E	TransitionParameter	REAL	过度模式参数
B	VelFactor	REAL	矢量速度的新覆盖因子
E	AccFactor	REAL	矢量加速/减速的新覆盖因子
E	JerkFactor	REAL	矢量加加速度的新覆盖因子
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

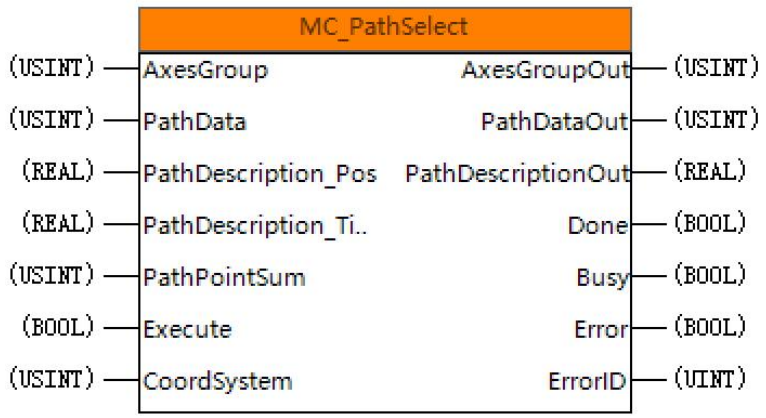
```
MC_MoveDirectRelative(AxesGroup := ,Execute := ,Distance := ,
                      CoordSystem := ,BufferMode := ,TransitionMod
                      e := ,TransitionParameter := ,VelFactor :=
                       ,AccFactor := ,JerkFactor := ,AxesGroupOut
                      => ,Done => ,Busy => ,Active
                      => ,CommandAborted => ,Error => ,ErrorID => );
```

2.2.31 MC_PathSelect

这个模块功能是设置一组运动路径点

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	PathData	USINT	路径编号
B	PathDescription_Pos	REAL*	路径描述的位置数组指针
B	PathDescription_Time	REAL*	路径描述的时间数组指针
B	PathPointSum	USINT	路径插补点总数
B	Execute	BOOL	上跳沿使能
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	PathDataOut	USINT	路径数据
B	PathDescriptionOut	REAL	路径描述的位置
B	Done	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

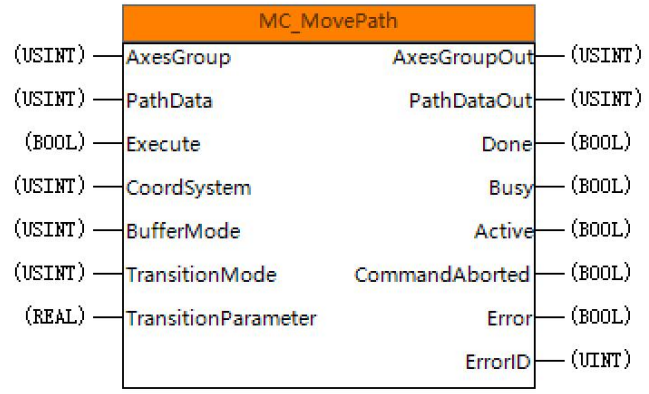
```
MC_PathSelect(AxesGroup := ,PathData := ,PathDescription_Pos := ,
              PathDescription_Time := ,PathPointSum := ,Execute :=
              ,CoordSystem := ,AxesGroupOut
              => ,PathDataOut=> ,PathDescriptionOut => ,Done => ,Busy
              => ,Error => ,ErrorID => );
```

2.2.32 MC_MovePath

这个模块功能是控制轴组按照运动路径点运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	PathData	USINT	路径编号
B	Execute	BOOL	上跳沿使能
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
E	TransitionMode	USINT	过度模式
E	TransitionParamete r	REAL	过度模式参数
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	PathDataOut	USINT	路径数据
E	Done	BOOL	模块是否完成
B	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

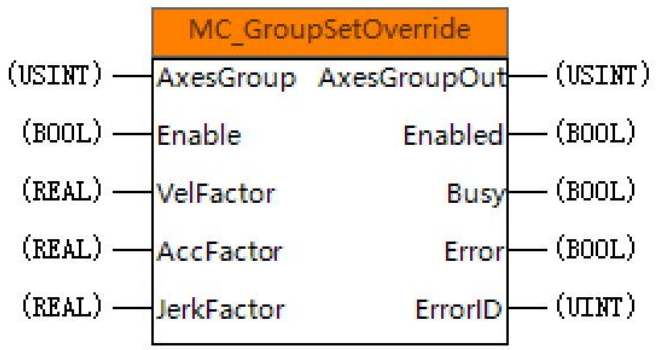
```
MC_MovePath(AxesGroup := ,PathData := ,Execute := ,CoordSystem
        := ,BufferMode := ,TransitionMode := ,TransitionParamet
        er := ,AxesGroupOut => ,PathDataOut => ,Done => ,Busy
        => ,Active => ,CommandAborted => ,Error => ,ErrorID => );
```

2.2.33 MC_GroupSetOverrid

这个模块功能是设置轴组的速度、加速度、加加速度比例

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Enable	BOOL	高电平使能
B	VelFactor	REAL	矢量速度的新覆盖因子
E	AccFactor	REAL	矢量加速/减速的新覆盖因子
E	JerkFactor	REAL	矢量加加速度的新覆盖因子
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	Enabled	BOOL	高电平使能
E	Busy	BOOL	模块是否未完成
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

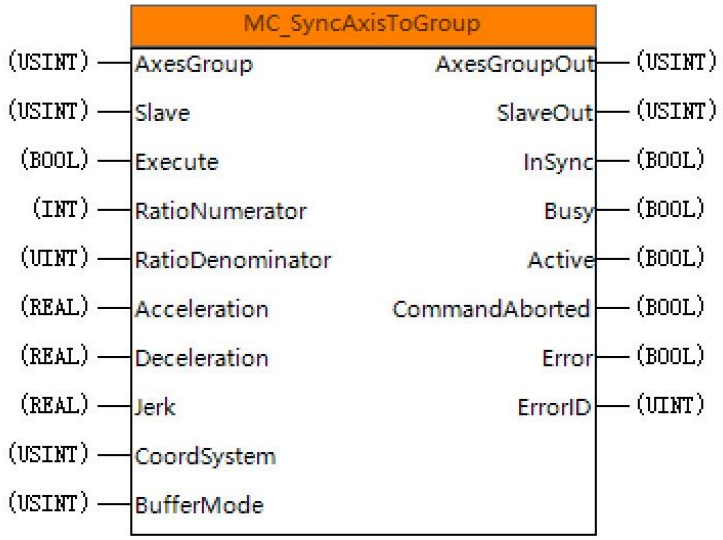
```
MC_GroupSetOverride(AxesGroup := ,Enable := ,VelFactor := ,
                    AccFactor := ,JerkFactor := ,AxesGroupOut
                    => ,Enabled => ,Busy => ,Error => ,ErrorID => );
```

2.2.34 MC_SyncAxisToGroup

这个模块功能是设置单轴相对轴组协调运动

VAR_INPUT			
B	AxesGroup	USINT	轴组编号
B	Slave	USINT	从轴编号
B	Execute	BOOL	上跳沿使能
E	RatioNumerator	INT	齿轮比分子
E	RatioDenominator	UINT	齿轮比分母
E	Acceleration	REAL	加速度
E	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承 3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	AxesGroupOut	USINT	轴组编号
B	SlaveOut	USINT	从轴编号
B	InSync	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_SyncAxisToGroup(AxesGroup := ,Slave := ,Execute := ,
                   RatioNumerator := ,RatioDenominator := ,Accelera
                   tion := ,Deceleration := ,Jerk := ,CoordSystem
                   := ,BufferMode := ,AxesGroupOut => ,SlaveOut
                   => ,InSync => ,Busy = >,Active => ,CommandAborted
                   => ,Error => ,ErrorID => );
```

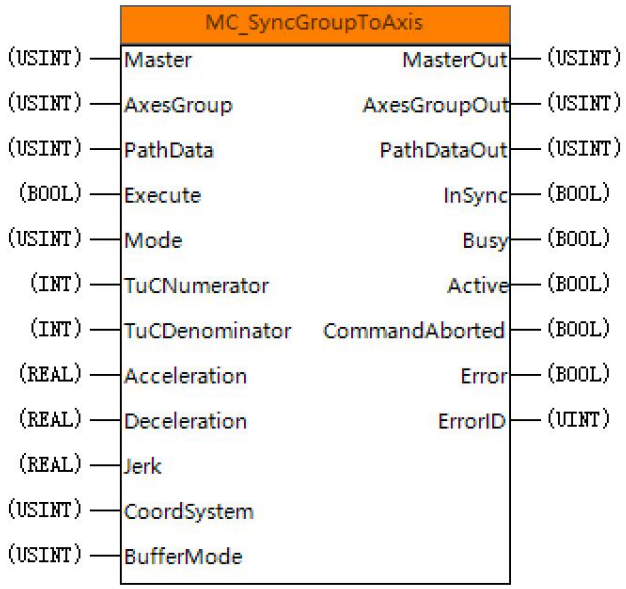
2.2.35 MC_SyncGroupToAxis

这个模块功能是设置轴组相对单轴协调运动

VAR_INPUT			
B	Master	USINT	主轴编号
B	AxesGroup	USINT	轴组编号
B	PathData	USINT	路径编号
B	Execute	BOOL	上跳沿使能
E	Mode	USINT	模式，周期或非周期
E	TuCNumberator	INT*	轴组中各轴的技术单元转换因子的分子数组指针
E	TuCDenominator	INT*	轴组中各轴的技术单元转换因子的分母数组指针
E	Acceleration	REAL	加速度
E	Deceleration	REAL	减速度
E	Jerk	REAL	加加速度
E	CoordSystem	USINT	坐标系类型 1.ACS 关节坐标系 2.MCS 世界坐标系 3.PCS 工件坐标系
E	BufferMode	USINT	运动混承类型 0.Aborting 立即打断 1.Buffered 缓冲 2.BlendingLow 按两个模块较低那个速度混承

			3.BlendingPrevious 按前一个模块速度混承 4.BlendingNext 按后一个模块速度混承 5.BlendingHigh 按两个模块较高那个速度混承
VAR_OUTPUT			
B	MasterOut	UINT	主轴编号
B	AxesGroupOut	UINT	轴组编号
B	PathDataOut	UINT	路径编号
B	InSync	BOOL	模块是否完成
E	Busy	BOOL	模块是否未完成
E	Active	BOOL	当前模块是否正在控制轴
E	CommandAborted	BOOL	模块是否被打断
B	Error	BOOL	模块运行是否出错
E	ErrorID	UINT	出错类型

功能块图形形式



功能块文本形式

```
MC_SyncGroupToAxis(Master := ,AxesGroup := ,PathData := ,
                   Execute := ,Mode := ,TuCNumerator := ,
                   TuCDenominator := ,Acceleration := ,Deceleration
                   := ,Jerk := ,CoordSystem := ,BufferMode := ,
                   MasterOut => ,AxesGroupOut => ,PathDataOut
                   => ,InSync => ,Busy => ,Active => ,CommandAborted
                   => ,Error => ,ErrorID => );
```

3 通信协议指令

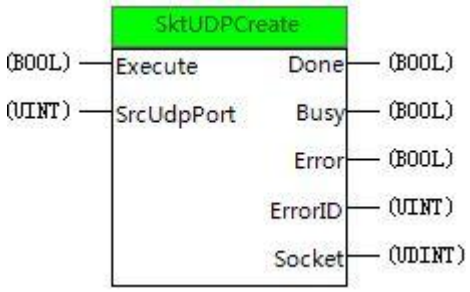
3.1 通讯基础指令

3.1.1 StkUDPCreate

这个功能块是根据输入的端口号创建 UDP 服务，并保存创建结果到输出变量中。

VAR_INPUT			
B	Execute	BOOL	非指针
B	SrcUdpPort	UINT	非指针，端口号
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，端口是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	Socket	UDINT	非指针，Socket 句柄

功能块图形形式



功能块文本形式

```
SktUDPCreate(Execute := ,SrcUdpPort := ,Done => ,Busy => ,Error
             => ,ErrorID => ,Socket => );
```

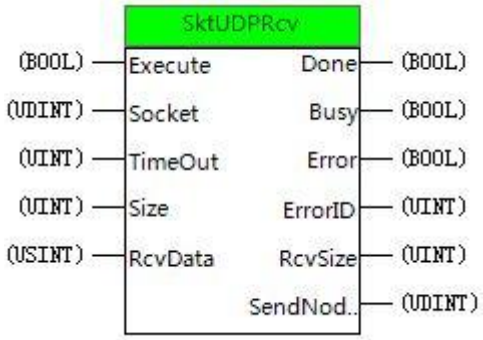
3.1.2 StkUDPRcv

这个功能块是从 Socket 输入缓冲区中读取数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Socket	UDINT	非指针，Socket 句柄
B	TimeOut	UINT	非指针，读取数据超时时间
B	Size	UINT	非指针，读取数据的大小
B	RcvData	USINT	指针，保存读到数据的缓冲区
B			
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用

B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	RcvSize	UINT	非指针，实际读到数据的大小
B	SendNodeAddr	UDINT	非指针，发送方的地址

功能块图形形式



功能块文本形式

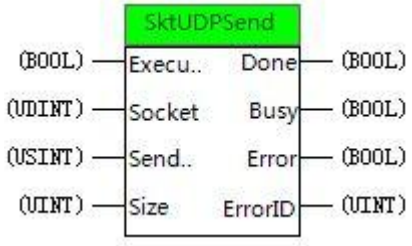
```
SktUDPRcv(Execute := ,Socket := , TimeOut := , Size := ,
          RcvData := ,Done => ,Busy => ,Error => ,ErrorID => ,RcvSize
          => ,SendNodeAddr =>);
```

3.1.3 StkUDPSend

这个功能块是向 Socket 发送数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Socket	UDINT	非指针，Socket 句柄
B	SendData	USINT	指针，待发送的数据
B	Size	UINT	非指针，待发送数据的大小
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

```
SktUDPSend(Execute := ,Socket := , SendData:= , Size := , Done
```

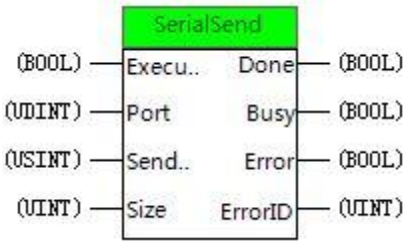
```
=> ,Busy => ,Error => ,ErrorID =>);
```

3.1.4 SerialSend

这个功能块是向串口号为 Port 的串口发送数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Port	UDINT	非指针，串口号
B	SendData	USINT	指针，待发送的数据
B	Size	UINT	非指针，待发送数据的大小
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

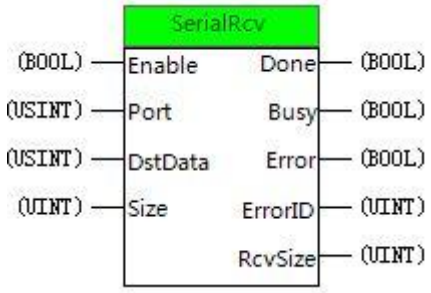
```
SerialSend(Execute := ,Port := , SendData := , Size := , Done => ,Busy
          => ,Error => ,ErrorID =>);
```

3.1.5 SerialRcv

这个功能块是从串口号为 Port 的串口输入缓冲区读取数据。

VAR_INPUT			
B	Enable	BOOL	非指针
B	Port	UDINT	非指针，串口号
B	DstData	USINT	指针，保存读到数据的缓冲区
B	Size	UINT	非指针，读取数据的大小
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	RcvSize	UINT	非指针，实际读到数据的大小

功能块图形形式



功能块文本形式

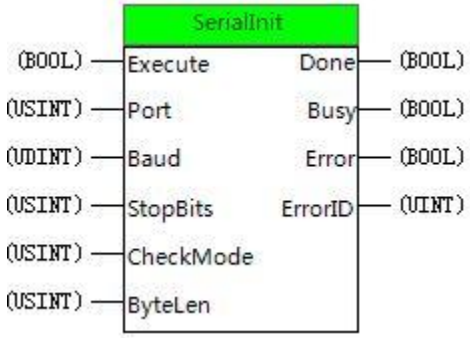
```
SerialRcv(Enable := ,Port := , DstData := , Size := , Done => ,Busy
          => ,Error => ,ErrorID =>, RcvSize =>);
```

3.1.6 SerialInit

这个功能块是初始化串口号为 Port 的串口。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Port	USINT	非指针，串口号
B	Baud	UDINT	非指针，波特率
B	StopBits	USINT	非指针，停止位
B	CheckMode	USINT	非指针，校验模式
B	ByteLen	USINT	非指针，字节长度
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

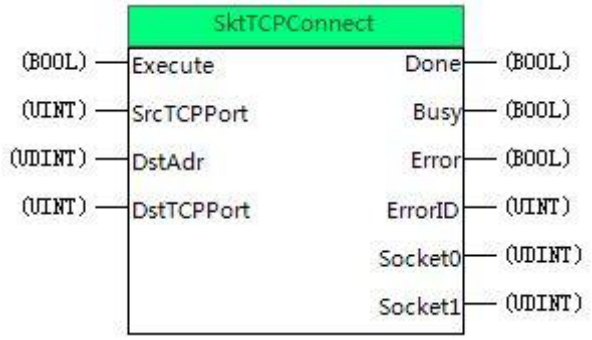
```
SerialInit(Execute := ,Port := , Baud := , StopBits := , CheckMode := ,
           ByteLen := , Done => ,Busy => ,Error => ,ErrorID =>);
```

3.1.7 StkTCPConnect

这个功能块是根据输入的源端口、目标地址、目标端口创建 TCP 连接，并把连接结果保存到输出变量中。

VAR_INPUT			
B	Execute	BOOL	非指针
B	SrcTCPPort	UINT	非指针，端口号
B	DstAdr	UDINT	非指针，目标地址
B	DstTCPPort	UINT	非指针，目标端口号
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	Socket0	UDINT	非指针，Socket 句柄
B	Socket1	UDINT	非指针，Socket 句柄

功能块图形形式



功能块文本形式

```
SktTCPConnect(Execute := ,SrcTCPPort := , DstAdr := ,
              DstTCPPort := ,Done => ,Busy => ,Error => ,ErrorID
              => ,Socket0 => ,Socket1 =>);
```

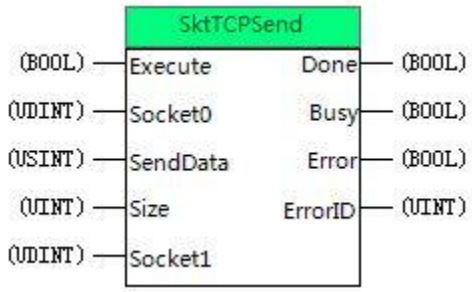
3.1.8 StkTCPSend

这个功能块是向 Socket 发送数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Socket0	UDINT	非指针，Socket 句柄
B	SendData	USINT	指针，待发送的数据
B	Size	UINT	非指针，待发送数据的大小
B	Socket1	UDINT	非指针，Socket 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针

B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

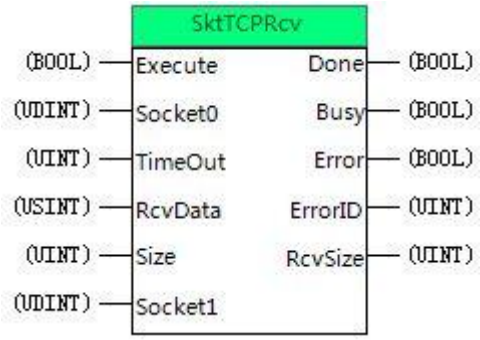
```
SktTCPSend(Execute := ,Socket0 := , SendData := ,
       Size := ,Socket1 := ,Done => ,Busy => ,Error => ,ErrorID
          =>);
```

3.1.9 StkTCPRcv

这个功能块是从 Socket 输入缓冲区中读取数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Socket0	UDINT	非指针，Socket 句柄
B	TimeOut	UINT	非指针，接收数据超时时间
B	RcvData	USINT	指针，保存接收数据的缓冲区
B	Size	UINT	非指针，要接收数据的大小
B	Socket1	UDINT	非指针，Socket 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	RcvSize	UINT	非指针，实际接收到数据的大小

功能块图形形式



功能块文本形式

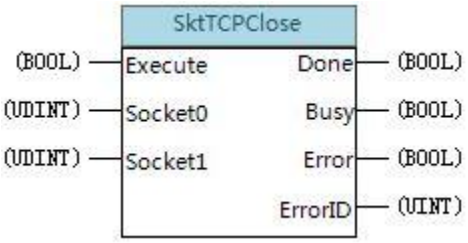
```
SktTCPRcv(Execute := ,Socket0 := , TimeOut := ,RcvData := ,
        Size := ,Socket1 := ,Done => ,Busy => ,Error => ,ErrorID
           => ,RcvSize =>);
```

3.1.10 StkTCPClose

这个功能块是关闭 TCP 连接。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Socket0	UDINT	非指针，Socket 句柄
B	Socket1	UDINT	非指针，Socket 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

```
SktTCPClose(Execute := ,Socket0 := ,Socket1 := ,Done => ,Busy => ,Error
              => ,ErrorID =>);
```

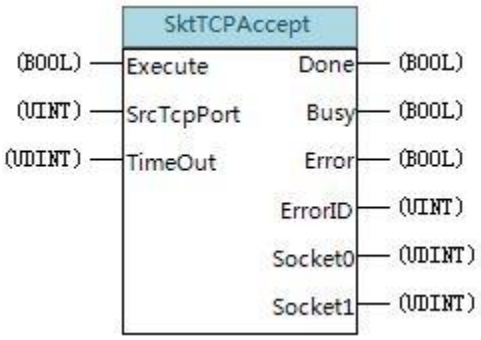
3.1.11 StkTCPAccept

这个功能块是接受 TCP 连接请求。

VAR_INPUT			
B	Execute	BOOL	非指针
B	SrcTcpPort	UINT	非指针，监听的端口号

B	TimeOut	UINT	非指针，监听超时时间
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	Socket0	UDINT	非指针，Socket 句柄
B	Socket1	UDINT	非指针，Socket 句柄

功能块图形形式



功能块文本形式

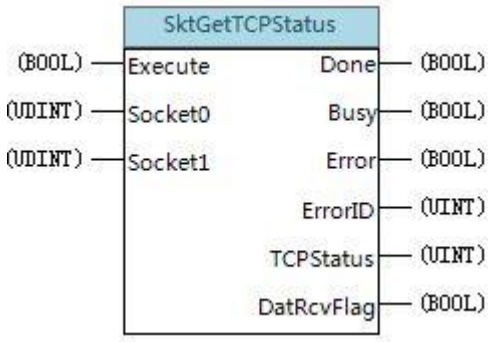
```
SktTCPAccept(Execute := ,SrcTcpPort := , TimeOut := ,Done => ,Busy
             => ,Error => ,ErrorID => ,Socket0 => ,Socket1 =>);
```

3.1.12 StkGetTCPStatus

这个功能块是获取 TCP 连接的状态。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Socket0	UDINT	非指针，Socket 句柄
B	Socket1	UDINT	非指针，Socket 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，执行是否出错
B	ErrorID	UINT	非指针，错误编号
B	TCPStatus	UINT	非指针，TCP 状态
B	DatRcvFlag	BOOL	非指针，数据接收标志

功能块图形形式



功能块文本形式

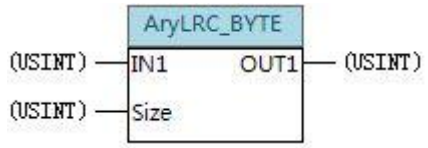
```
SktGetTCPStatus(Execute := ,Socket0 := ,Socket1 := ,Done => ,Busy
                => ,Error => ,ErrorID => ,TCPStatus => ,DatRcvFlag
                =>);
```

3.1.13 AryLRC_BYTE

这个功能块是计算字节数组的 LRC 值。

VAR_INPUT			
B	IN1	USINT	非指针，输入数据
B	Size	USINT	非指针，要处理的元素数量
VAR_OUTPUT			
B	OUT1	USINT	非指针，计算结果

功能块图形形式



功能块文本形式

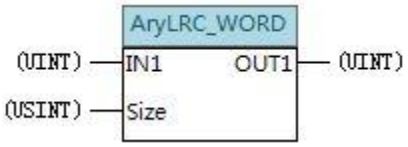
```
AryLRC_BYTE(IN1 := ,Size := ,OUT1 =>);
```

3.1.14 AryLRC_WORD

这个功能块是计算字数组的 LRC 值。

VAR_INPUT			
B	IN1	USINT	非指针，输入数据
B	Size	USINT	非指针，要处理的元素数量
VAR_OUTPUT			
B	OUT1	USINT	非指针，计算结果

功能块图形形式



功能块文本形式

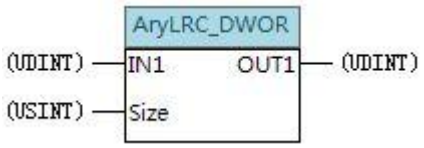
```
AryLRC_WORD(IN1 := ,Size := ,OUT1 =>);
```

3.1.15 AryLRC_DWORD

这个功能块是计算字数组的 LRC 值。

VAR_INPUT			
B	IN1	USINT	非指针，输入数据
B	Size	USINT	非指针，要处理的元素数量
VAR_OUTPUT			
B	OUT1	USINT	非指针，计算结果

功能块图形形式



功能块文本形式

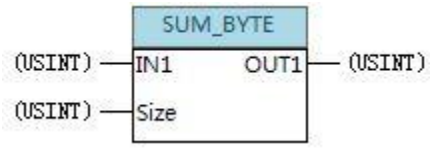
```
AryLRC_DWORD(IN1 := ,Size := ,OUT1 =>);
```

3.1.16 SUM_BYTE

这个功能块是计算字数组的校验和。

VAR_INPUT			
B	IN1	USINT	非指针，输入数据
B	Size	USINT	非指针，要处理的元素数量
VAR_OUTPUT			
B	OUT1	USINT	非指针，计算结果

功能块图形形式



功能块文本形式

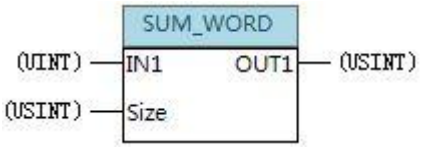
```
SUM_BYTE(IN1 := ,Size := ,OUT1 =>);
```

3.1.17 SUM_WORD

这个功能块是计算字数组的校验和。

VAR_INPUT			
B	IN1	UINT	非指针，输入数据
B	Size	USINT	非指针，要处理的元素数量
VAR_OUTPUT			
B	OUT1	USINT	非指针，计算结果

功能块图形形式



功能块文本形式

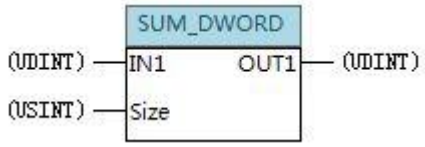
```
SUM_WORD(IN1 := ,Size := ,OUT1 =>);
```

3.1.18 SUM_DWORD

这个功能块是计算字数组的校验和。

VAR_INPUT			
B	IN1	UDINT	非指针，输入数据
B	Size	USINT	非指针，要处理的元素数量
VAR_OUTPUT			
B	OUT1	UDINT	非指针，计算结果

功能块图形形式



功能块文本形式

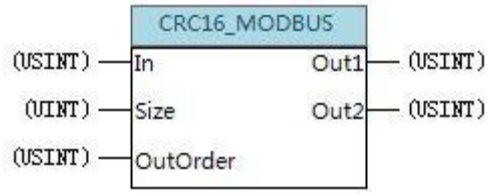
```
SUM_DWORD(IN1 := ,Size := ,OUT1 =>);
```

3.1.19 CRC16_MODBUS

这个功能块是 CRC-16 MODBUS 的形式校验。

VAR_INPUT			
B	In	USINT	非指针，输入数据
B	Size	UINT	非指针，要处理的元素数量
B	OutOrder	USINT	非指针，输出顺序
VAR_OUTPUT			
B	OUT1	USINT	非指针，计算结果
B	OUT2	USINT	非指针，计算结果

功能块图形形式



功能块文本形式

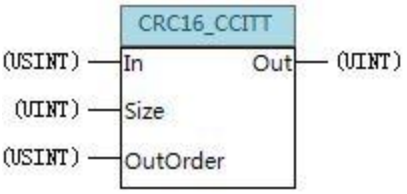
```
SUM_DWORD(IN1 := ,Size := ,OutOrder := ,OUT1 => , OUT2 =>);
```

3.1.20 CRC16_CCITT

这个功能块是 CRC 循环冗余码校验。

VAR_INPUT			
B	In	USINT	非指针，输入数据
B	Size	UINT	非指针，要处理的元素数量
B	OutOrder	USINT	非指针，输出顺序
VAR_OUTPUT			
B	OUT1	UINT	非指针，计算结果

功能块图形形式



功能块文本形式

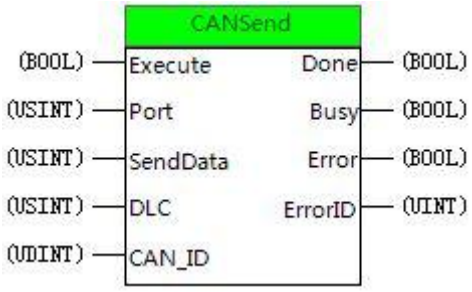
```
SUM_DWORD(IN1 := ,Size := ,OutOrder := ,OUT =>);
```

3.1.21 CANSend

这个功能块是向串口发送数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Port	USINT	非指针，串口号
B	SendData	USINT	指针，待发送的数据
B	DLC	USINT	非指针，数据长度代码
B	CAN_ID	UDINT	非指针，CAN ID
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

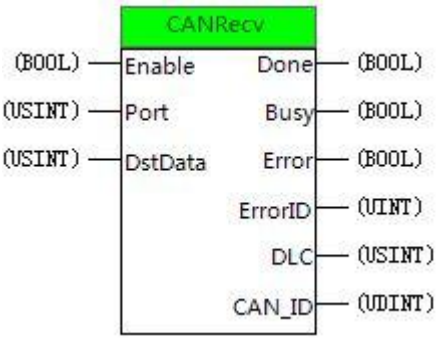
```
CANSend(Execute := ,Port := ,SendData := , DLC := , CAN_ID := ,Done
        => ,Busy => ,Error => ,ErrorID =>);
```

3.1.22 CANRecv

这个功能块是从串口输入缓冲区读取数据。

VAR_INPUT			
B	Enable	BOOL	非指针
B	Port	USINT	非指针，串口号
B	DstData	USINT	指针，存储接收数据的缓冲区
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号
B	DLC	USINT	非指针，数据长度代码
B	CAN_ID	UDINT	非指针，CAN ID

功能块图形形式



功能块文本形式

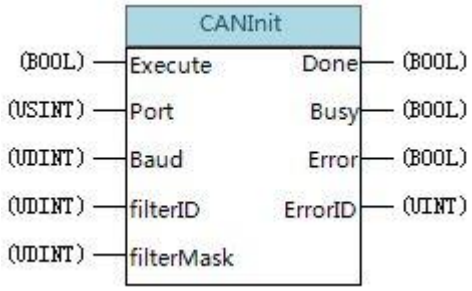
```
CANRecv(Enable := ,Port := ,DstData := ,Done => ,Busy => ,Error
        => ,ErrorID => , DLC => , CAN_ID =>);
```

3.1.23 CANInit

这个功能块是初始化串口。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Port	USINT	非指针，串口号
B	Baud	USINT	非指针，波特率
B	filterID	UDINT	非指针，过滤器 ID
B	filterMask	UDINT	非指针，过滤器掩码
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

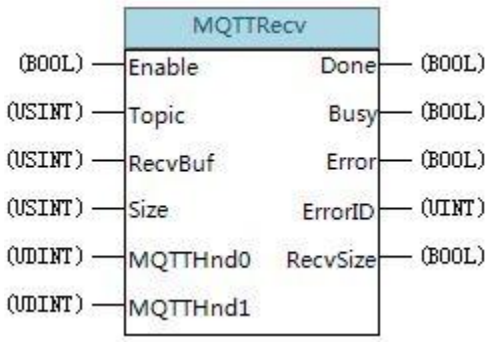
```
CANInit(Execute := ,Port := ,Baud := , filterID := , filterMask := ,Done
        => ,Busy => ,Error => ,ErrorID =>);
```

3.1.24 MQTTRecv

这个功能块是从 MQTT 缓冲区读取数据。

VAR_INPUT			
B	Enable	BOOL	非指针
B	Topic	USINT	指针，MQTT 主题
B	RecvBuf	USINT	指针，保存接收数据的缓冲区
B	Size	USINT	非指针，要读取的数据的大小
B	MQTTHnd0	UDINT	非指针，MQTT 句柄
B	MQTTHnd1	UDINT	非指针，MQTT 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号
B	RecvSize	BOOL	非指针，实际接收到的数据的大小

功能块图形形式



功能块文本形式

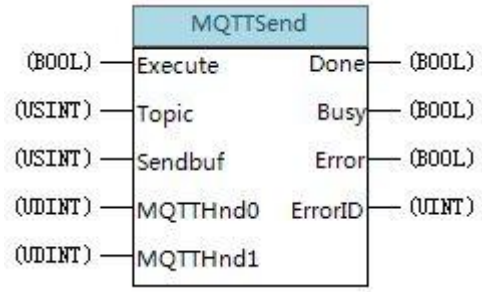
```
MQTTRecv(Enable := ,Topic := ,RecvBuf := , Size := , MQTTHnd0 := ,
         MQTTHnd1 := ,Done => ,Busy => ,Error => ,ErrorID => ,RecvSize
         =>);
```

3.1.25 MQTTSend

这个功能块是从 MQTT 缓冲区发送数据。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Topic	USINT	指针，MQTT 主题
B	SendBuf	USINT	指针，要发送的数据
B	MQTTHnd0	UDINT	非指针，MQTT 句柄
B	MQTTHnd1	UDINT	非指针，MQTT 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

```
MQTTSend(Execute := ,Topic := ,RecvBuf := , MQTTHnd0 := ,
         MQTTHnd1 := ,Done => ,Busy => ,Error => ,ErrorID =>);
```

3.1.26 MQTTSubscribe

这个功能块是订阅 MQTT 主题。

VAR_INPUT			
B	Execute	BOOL	非指针
B	Topic	USINT	指针，MQTT 主题
B	MQTTHnd0	UDINT	非指针，MQTT 句柄
B	MQTTHnd1	UDINT	非指针，MQTT 句柄
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号

功能块图形形式



功能块文本形式

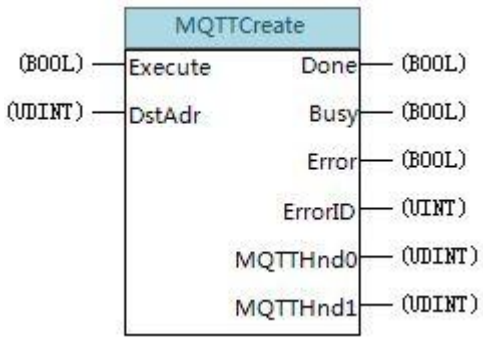
```
MQTTSubscribe(Execute := ,Topic := , MQTTHnd0 := , MQTTHnd1 := ,Done
              => ,Busy => ,Error => ,ErrorID =>);
```

3.1.27 MQTTCreate

这个功能块是创建 MQTT 服务。

VAR_INPUT			
B	Execute	BOOL	非指针
B	DstAdr	UDINT	非指针，目标地址
VAR_OUTPUT			
B	Done	BOOL	非指针
B	Busy	BOOL	非指针，是否被占用
B	Error	BOOL	非指针，是否被占用
B	ErrorID	UINT	非指针，错误编号
B	MQTTHnd0	UDINT	非指针，MQTT 句柄
B	MQTTHnd1	UDINT	非指针，MQTT 句柄

功能块图形形式



功能块文本形式

```
MQTTCreate(Execute := ,DstAdr := , Done => ,Busy => ,Error => ,ErrorID
          => ,MQTTHnd0 => ,MQTTHnd1 =>);
```